19 October 2026

# 32113 Advanced Database Assignment 2 Group 2

**Wattlebrook University Governed Data Platform: Solution Design and Working Prototype**

19 October 2026

## Student Names & UTS Student IDs

- Arya Mahesh Kaslikar (26184921)
- Abdul Mannan Mohammed (26215537)
- Aditi Mangroliya (26226264)
- Margi Govindbhai Vadodaria (26240402)
- Mihir Chauhan (25620699)
- Omkar Ravindran (25630423)

## Table of Contents

**Executive Summary** **1**

**1. Introduction** **2**

**2. Solution Design** **3**

2.1. Solution Architecture 3

2.2. Conceptual Model of Data Entities 4

2.3. Logical Data Model of Data Entities 6

2.4. Design Rationale 9

2.5. Design Trade-offs 10

**3. Working Prototype Implementation** **12**

3.1. Databases and Schemas of Source Systems 12

3.2. Integrated Data Warehouse Database and Schema 15

3.3. SQL Scripts and Mapping to the Solution Design 18

3.4. Reports and Dashboard per Data Use Case 22

3.5. End-to-end Testing 28

**4. Summary and Conclusion** **34**

**5. References** **35**

**6. Appendix** **37**

6.1. Individual Contribution Table 37

6.2. Individual Contributions (Evidence from Meeting Minutes) 37

6.3. Teamwork and Communication 39

6.4. Meeting Minutes 39

6.5. Source Code Listing 40

## Executive Summary

Wattlebrook University (WBU) holds the same people in three systems that share no identifier: the HR system (HRMS), the student information system (SIS) and the alumni and donor CRM. A 2026 audit also found 27,500 copies of production records that developers had made for testing since 2012. Of these, 94.1% were unmasked, 3.9% were catalogued and 92.2% were past their approved retention date.

This report presents the design and working prototype of a governed data platform that addresses the five problems identified in Assignment 1. The prototype runs on PostgreSQL 16 with a small Python orchestrator. Data flows from three source schemas through bronze (raw copies), silver (a Data Vault 2.0 model with identity resolution) and gold (a star schema without direct identifiers). A governance schema logs every step, every data movement and every table, and applies four retention policies. Masked copies of all three systems give developers realistic test data.

On a clean run on 15 October 2026, all 10 end-to-end checks passed. The 27,500 source records resolved into 20,399 people against a true 20,200, with 99.81% precision and 97.03% recall. Retention anonymised 8,970 expired person records, deleted 25,362 expired shadow copies and quarantined the 2,138 still within their approved date. The 18,530 masked rows contained no real names, emails, phone numbers or source IDs. The source code is at https://github.com/AryaK-31/wattlebrook-db and a recorded end-to-end demonstration is at https://www.youtube.com/watch?v=_rLFbb1bO5Y.

**Keywords:** Data Vault 2.0, identity resolution, data masking, data retention, data lineage, PostgreSQL

## 1. Introduction

Universities hold data about the same people for different purposes. A student may later join the staff and, after graduating, become an alumnus or a donor. Each role is recorded in a different system with its own identifier, so the university cannot easily say how many people it holds data about, or answer a privacy request or a breach enquiry with confidence (Henkoğlu & Uçak 2016). Privacy law adds pressure. Australian Privacy Principle 11 requires organisations to protect personal information and to destroy or de-identify it once it is no longer needed (Office of the Australian Information Commissioner 2025), and the NSW Privacy and Personal Information Protection Act 1998 places similar duties on NSW public sector agencies (NSW Parliamentary Counsel's Office 1998).

Wattlebrook University shows both problems. Its synthetic data set holds 5,000 HRMS staff records, 15,000 SIS student records and 7,500 CRM contacts, and each system formats names, emails and phone numbers differently. Over fourteen years developers also exported 27,500 production records to laptops, shared drives, sandboxes and Git repositories. None of these copies was registered, and most were never masked or deleted.

The prototype implemented in the lab environment has five high-level goals, one for each problem identified in Assignment 1:

- **Automate the data flow.** Replace manual extracts with one logged pipeline from source to bronze, silver and gold, where every step can be rerun safely.
- **Resolve identities.** Link the HRMS, SIS and CRM records of the same person, and keep the matching rule and confidence for every link.
- **Protect test data.** Give developers masked copies of all three systems, so real personal data stays in production.
- **Know where data is.** Keep a catalogue of every table, lineage for every data movement, and a scan that finds tables nobody registered.
- **Enforce retention.** Give every record a use-by date, anonymise expired personal data and delete expired copies.

All data is synthetic. The workbook includes an answer key that says which records belong to the same person, so the identity matching can be scored. Section 2 sets out the solution design, Section 3 the working prototype and its end-to-end testing, and Section 4 summarises the outcome.

# 2. Solution Design

## 2.1. Solution Architecture

Figure 1 shows the architecture. It follows the bronze, silver and gold (medallion) pattern inside one PostgreSQL 16 database (Bay Information Systems 2026), with a governance plane that every layer writes to. Each layer is a separate schema, so access can be granted per layer.

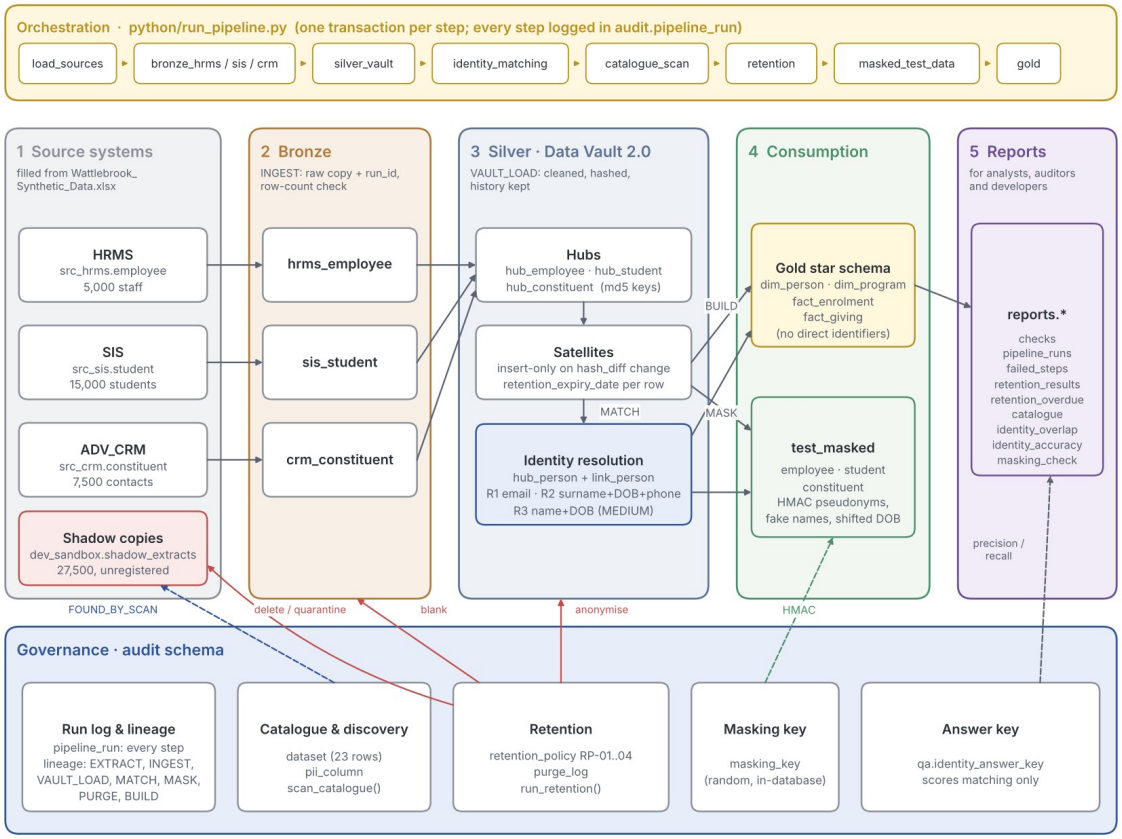



**Figure 1: Solution architecture of the Wattlebrook governed data platform**

- **Source systems** (`src_hrms`, `src_sis`, `src_crm`) stand in for the three operational systems. `dev_sandbox.shadow_extracts` holds the 27,500 test copies the audit found. `python/load_source_data.py` fills these tables from the Excel workbook.
- **Bronze** keeps a raw copy of each source table plus the run ID, load time and source label. `bronze.load_source()` copies by column name, then fails the step if the row counts differ or the source has gained a column bronze does not know.
- **Silver** is a Data Vault 2.0 model (Linstedt & Olschimke 2015). Hubs hold business keys, satellites hold the cleaned descriptive data, and a new satellite row is added only

when the data changes. Every satellite row carries its own use-by date. Identity resolution adds a person hub and a link that ties each source record to one person.

- **Consumption** has two outputs: a gold star schema for reporting, which holds no names, emails, phone numbers, dates of birth or addresses, and `test_masked`, which gives developers masked copies of all three systems.
- **Reports** are nine SQL views in the `reports` schema, including the end-to-end test `reports.checks`. A dashboard, drawn by `python/dashboard.py`, shows the main reports on one page.
- **Governance plane** (`audit` schema): the run log, lineage, catalogue, personal-data column register, retention policies, purge log and masking key. `qa.identity_answer_key` is used only to score the matching.
- **Orchestration**: `python/run_pipeline.py` runs `sql/01` to `sql/11`, which are safe to run again, then runs ten steps in order. Each step has its own transaction, and its RUNNING row is written in a separate transaction, so a failed step is rolled back but stays visible in the log.

## 2.2. Conceptual Model of Data Entities

Figure 2 shows the business entities and the rules that connect them, independent of how they are stored.

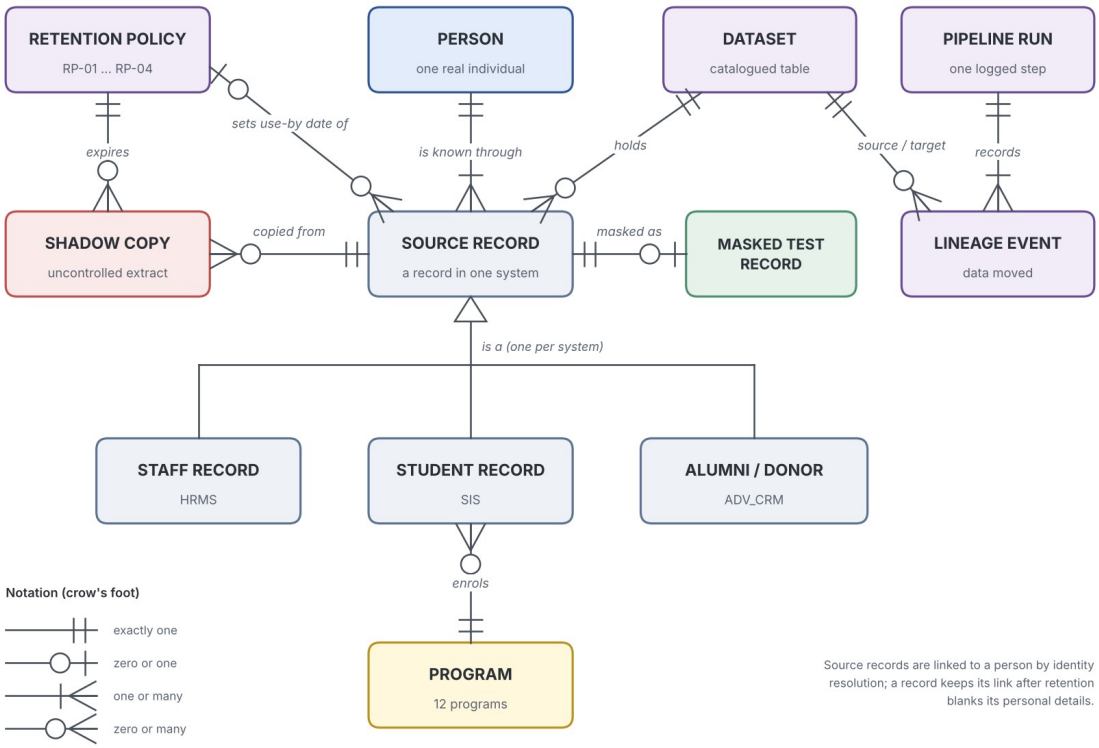



**Figure 2: Conceptual model of the data entities (crow's foot notation)**

- A **Person** is known through one or more **Source Records**, and each source record belongs to exactly one person. A source record is a staff record (HRMS), a student record (SIS) or an alumni or donor contact (ADV_CRM).
- Each **Student Record** is enrolled in exactly one **Program**. A program can have many students.
- A **Shadow Copy** is copied from one source record. A source record can have many uncontrolled copies.
- A **Retention Policy** sets the use-by date of a source record. Only some records have one: current staff, enrolled or deferred students and donors do not expire. Policy RP-04 expires shadow copies.
- A source record has at most one **Masked Test Record**. Records whose personal details were anonymised get none.
- A **Dataset** (a catalogued table) holds records. Each **Pipeline Run** step records **Lineage Events**, and each event names a source and a target dataset.

## 2.3. Logical Data Model of Data Entities

The logical model turns each conceptual entity into tables with keys and data types. Figure 3 shows the silver Data Vault, and Figure 4 shows the gold star schema and the governance tables. Table 1 maps the conceptual entities to these tables.

**Table 1: Conceptual entities and the logical tables that implement them**

| Conceptual entity | Logical table(s) | Primary key |
|---|---|---|
| Person | silver.hub_person; gold.dim_person | person_id; person_key (surrogate) |
| Source record | silver.hub_employee / hub_student / hub_constituent | md5 hash of the business key |
| Staff, student, contact details | silver.sat_employee / sat_student / sat_constituent | hash key + load_dts |
| Person ↔ source record | silver.link_person | system_name + source_record_id |
| Program | gold.dim_program | program_key (surrogate) |
| Enrolment, giving | gold.fact_enrolment, gold.fact_giving | (fact rows, FK to dimensions) |
| Shadow copy | dev_sandbox.shadow_extracts | shadow_record_id |
| Masked test record | test_masked.employee / student / constituent | HMAC pseudonym of the source ID |
| Retention policy | audit.retention_policy (+ audit.purge_log) | policy_id |
| Dataset | audit.dataset (+ audit.pii_column) | dataset (schema.table) |
| Pipeline run, lineage event | audit.pipeline_run, audit.lineage | run_id; lineage_id |

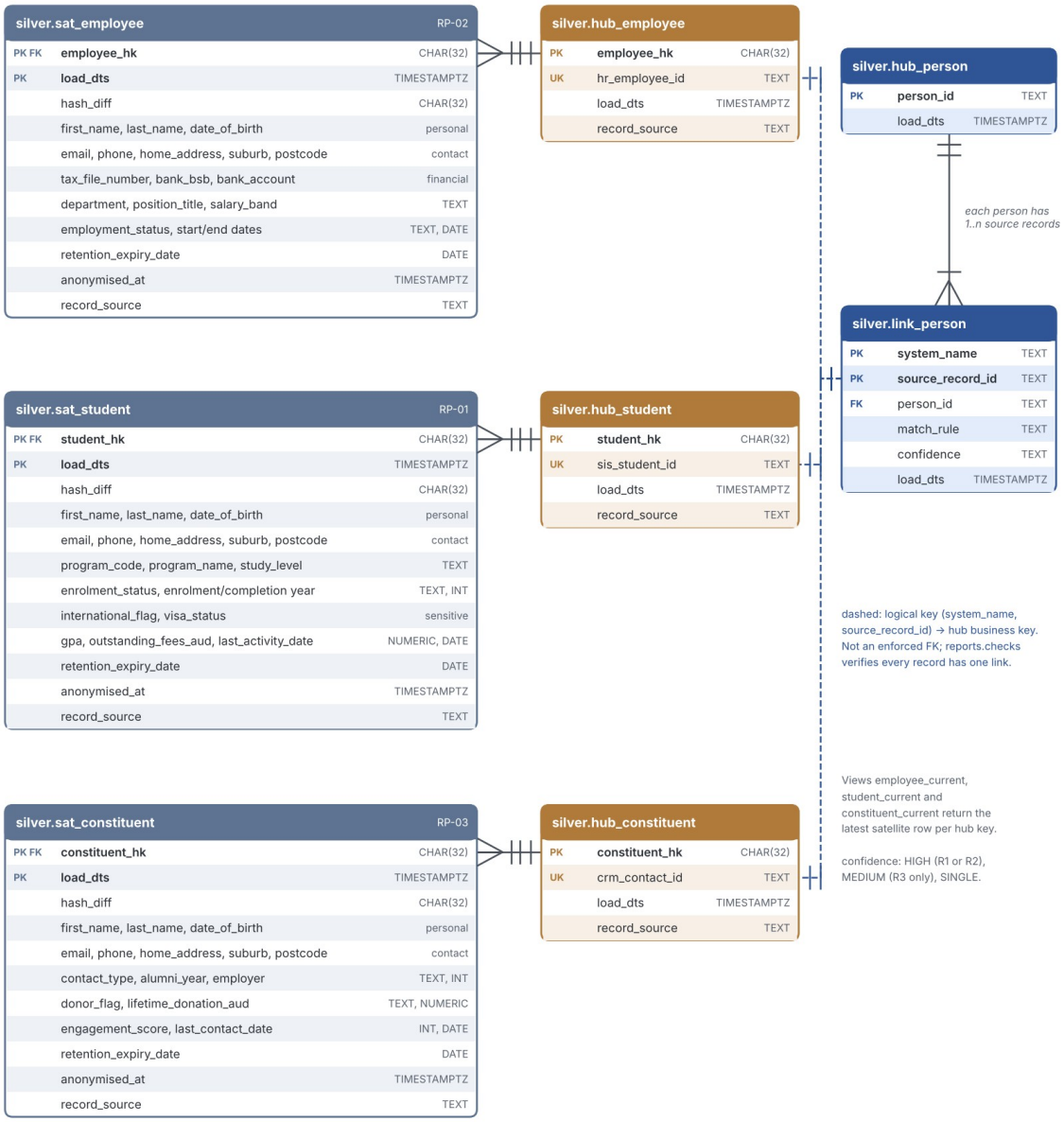



**Figure 3: Logical model of the silver layer (Data Vault 2.0 with person hub and link)**

The main decisions in the silver model are:

- **Hash keys.** Each hub's key is the md5 hash of the source system's ID, stored as `CHAR(32)`. The same record always gets the same key, so reloading never creates a duplicate hub row.
- **Insert-only satellites.** Satellites have a composite primary key (hash key, `load_dts`). A row is inserted only when `hash_diff`, an md5 of the descriptive columns, differs from the latest row. The views `employee_current`, `student_current` and `constituent_current` return the latest row.

- **Use-by date per row.** `retention_expiry_date` is calculated at load time: the end of employment, last activity or last contact plus 2,555 days (7 years). `anonymised_at` records when retention removed the details.
- **One link per source record.** The primary key of `silver.link_person` is (`system_name`, `source_record_id`), so a record can belong to only one person. `person_id` is 'P-' plus the first 10 hex characters of an md5 of the group's name, which keeps it stable between runs.

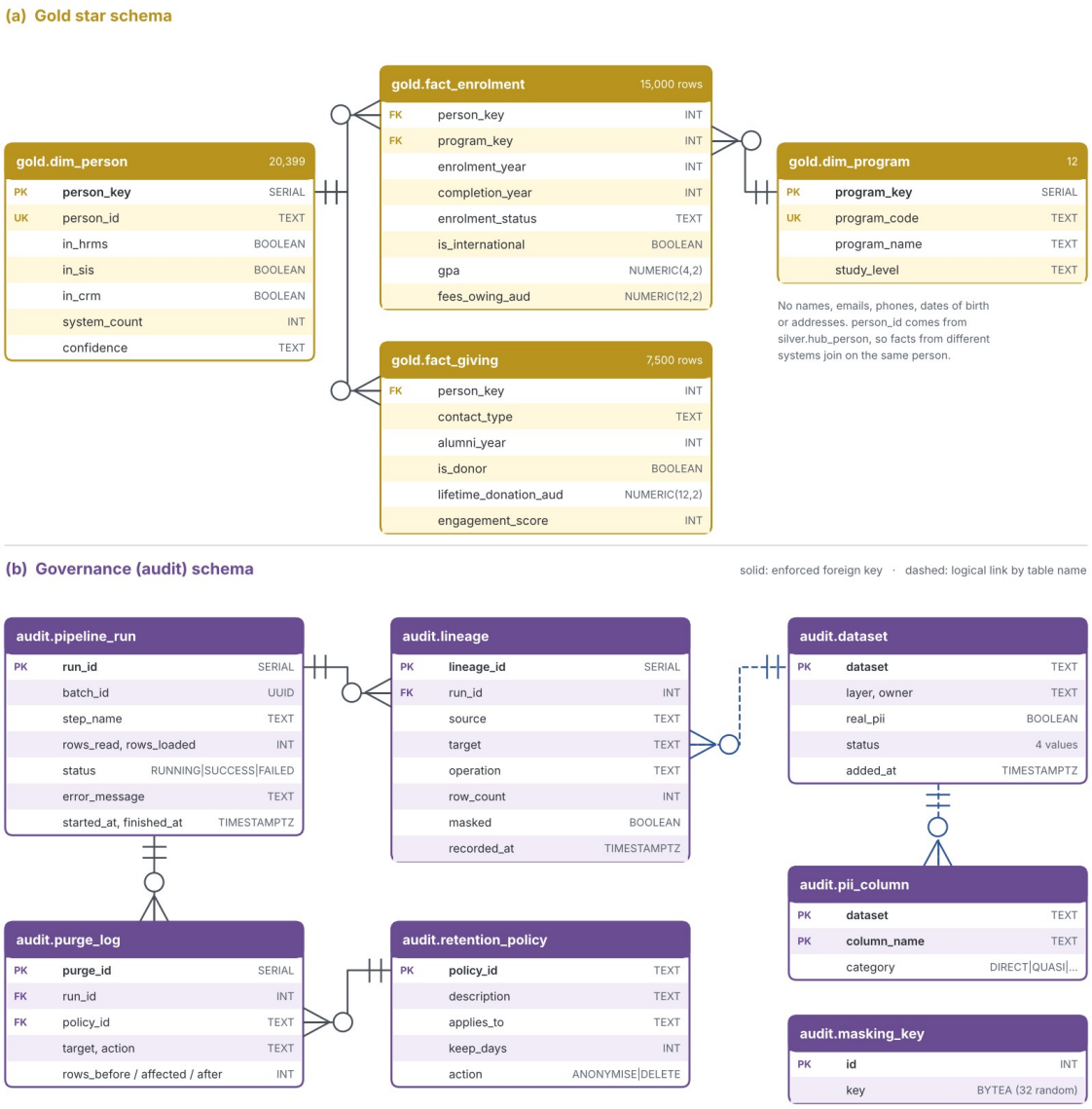



**Figure 4: Logical model of (a) the gold star schema and (b) the governance (audit) schema**

In gold, `dim_person` has one row per resolved person, with flags for the systems the person appears in and the weakest confidence used to build them. The two fact tables join to it through a surrogate key, so a donation and an enrolment from different

systems can be counted against the same person. In the audit schema, foreign keys link lineage and purge-log rows to the pipeline step that wrote them, and purge-log rows to their retention policy. CHECK constraints restrict status, category and action values to the allowed lists.

## 2.4. Design Rationale

**Use-case fit**

Each layer exists because one of the five goals needs it. Bronze gives a row-for-row copy that can be reconciled with the source, which turns manual copying into a logged, checked load. Silver's Data Vault keeps history without overwriting it, separates business keys from descriptive data, and lets retention blank a satellite while keeping the hub and link. The platform can therefore still report how many people were affected without keeping their details. A relational model in third normal form would overwrite history when an address changes (Arab 2026). Gold uses a star schema because reporting needs simple, fast joins (Kimball & Ross 2013; Hanumanthaiah 2023).

**Explainable identity matching**

Matching uses three deterministic rules rather than a probabilistic or machine-learning matcher. R1 (same email) and R2 (same surname, date of birth and phone number) are HIGH confidence. R3 (same first name, surname and date of birth, only between different systems) is MEDIUM confidence. Matches chain together, so if A matches B and B matches C, all three are one person. Every link stores the rules that produced it, so a data steward can see why two records were joined, and the answer key gives an exact precision and recall. Christen (2012) describes this trade-off between explainable rule-based linkage and more complex probabilistic methods.

**Governance inside the data platform**

Lineage is written by each step as it moves data, not inferred afterwards, so it cannot fall out of date. The discovery scan reads `information_schema`, so a table that nobody registered still appears in the catalogue, with owner UNKNOWN. Retention is driven by the use-by date stored on each row, so the overdue report and the purge procedure use the same rule. These follow privacy-by-design principles: controls are part of the pipeline, not added afterwards (Obiokafor & Aguboshim 2025; Gopa & Goel 2025).

### Masking that keeps data useful

Masked IDs use HMAC-SHA256 with a random 32-byte key from pgcrypto (PostgreSQL Global Development Group 2025). The same input always gives the same pseudonym, so masked tables still join, and a masked person reference links one person across all three masked systems. Names come from a list of invented names that do not occur in the data. Dates of birth move 15 to 350 days, because dates of birth combined with other details can re-identify people (Navandar 2024). Tax file numbers, bank details and street addresses are left out, and donations are rounded to the nearest $1,000.

### Performance, complexity and cost

At 27,500 source records, one PostgreSQL database is enough. A clean run took about 33 seconds in our test environment (Figure 19), with loading the workbook and building the masked test data taking the most time (Figure 30). Running the logic as stored procedures keeps it next to the data, and each step is one transaction. Indexes on satellite email columns support look-ups by email: one look-up uses an index scan and runs in 0.09 ms (Figure 37). A data lake or lakehouse suits very large or varied files (Azzabi et al. 2024) but would add services, cost and new skills without benefit at this scale. All components are open source and run in one Docker container, so the prototype has no licence cost.

## 2.5. Design Trade-offs

Table 2 lists the main trade-offs, what each one costs, and how the cost is handled now or could be handled next. Several limitations were found by testing the prototype.

**Table 2: Design trade-offs**

| Decision | Benefit | Cost or limitation | Mitigation / next step |
|---|---|---|---|
| Data Vault in silver instead of a 3NF model | Full history; new sources add hubs without redesign; retention can keep hubs and links | More tables and joins; satellites hold cleaned values, so this is a simplified vault | The *_current views hide the history for everyday queries |
| Deterministic rules R1–R3 instead of probabilistic matching | Explainable, repeatable and measurable | 99.81% precision, 97.03% recall: 15 wrong links (people who share a name and date of birth) and 238 missed links | Route MEDIUM (R3-only) links to a human reviewer; add fuzzy rules only where accuracy can be measured |

| Decision | Benefit | Cost or limitation | Mitigation / next step |
|---|---|---|---|
| One PostgreSQL database for every layer | Simple, cheap and transactional; one place to secure | Masking key stored in the same database; no separation between test and production | Move the key to a secrets manager and test_masked to its own database |
| Gold and test_masked rebuilt in full each run | Always consistent with silver; simple code | Run time grows with data volume | Switch to incremental loads when volume grows |
| Retention applied to bronze and silver, not to the source systems | The platform is compliant regardless of source timing | Sources still hold expired data; identity matching reads bronze before retention blanks it | Agree retention with source owners; keep existing links once records are anonymised |
| Remaining shadow copies quarantined, not dropped | Copies within their approved date are kept as agreed | Everything runs as one superuser, so REVOKE from PUBLIC has little real effect | Create developer and analyst roles with access only to test_masked and gold |
| Personal data found by column name | Fast and simple | Misses personal data in columns with unusual names | Add sampling of column contents |

## 3. Working Prototype Implementation

The prototype runs in one PostgreSQL 16 database, `wattlebrook_db`, and uses Python 3.11 or later with pandas, openpyxl, psycopg 3 and python-dotenv, plus matplotlib for the dashboard. The repository starts the database with Docker Compose (`postgres:16-alpine`). The end-to-end test shown in this section was run on a PostgreSQL 16.15 server with Python 3.13 (Figures 17 and 18). Credentials come from a local `.env` file that is not committed to Git. The source code is in the GitHub repository https://github.com/AryaK-31/wattlebrook-db, and a recorded demonstration of the end-to-end test is on YouTube at https://www.youtube.com/watch?v=_rLFbb1bO5Y. Figure 16 shows the repository being cloned for the test. Every screenshot below is marked with the step of the test it belongs to, and Table 9 in Section 3.5 lists all steps with their figures.

### 3.1. Databases and Schemas of Source Systems

Three source schemas stand in for the operational systems, and a fourth holds the shadow copies found by the audit (Table 3). Each schema is defined in `sql/02_source_tables.sql`, and `python/load_source_data.py` copies the matching sheet of `data/Wattlebrook_Synthetic_Data.xlsx` into it with PostgreSQL COPY. Each copy writes an EXTRACT row to the lineage table.

**Table 3: Source system schemas**

| Schema.table (system) | Rows | Business key | Personal data held | Use-by date driver | Evidence |
|---|---|---|---|---|---|
| src_hrms.employee (HRMS) | 5,000 | hr_employee_id (EMPnnnnnn) | Name, DOB, email, phone, address, tax file number, bank details, salary band | Former staff: employment end date + 7 years (RP-02) | Figures 6, 9 |
| src_sis.student (SIS) | 15,000 | sis_student_id (STUnnnnnnn) | Name, DOB, email, phone, address, visa status, GPA, fees owing | Completed or withdrawn: last activity + 7 years (RP-01) | Figures 7, 9 |
| src_crm.constituent (ADV_CRM) | 7,500 | crm_contact_id (CRMnnnnnnn) | Name, DOB, email, phone, address, employer, donations | Non-donors: last contact + 7 years (RP-03) | Figures 8, 9 |

| Schema.table (system) | Rows | Business key | Personal data held | Use-by date driver | Evidence |
|---|---|---|---|---|---|
| dev_sandbox.shadow_extracts | 27,500 | shadow_record_id | Copied fields from all three systems; 94.1% unmasked | Approved expiry date of each copy (RP-04) | Figures 9, 20 |

The three systems format the same details differently, which is what makes integration hard. For example, Anh Le has three IDs, three email addresses and the phone number written as `0439 986 987`, `0439986987` and `+61 439 986 987` (Figure 5). The source data also has a realistic status mix: 3,144 current and 1,856 former staff; 11,116 completed, 1,403 withdrawn, 2,271 enrolled and 210 deferred students; and 2,050 donors among the 7,500 CRM contacts.

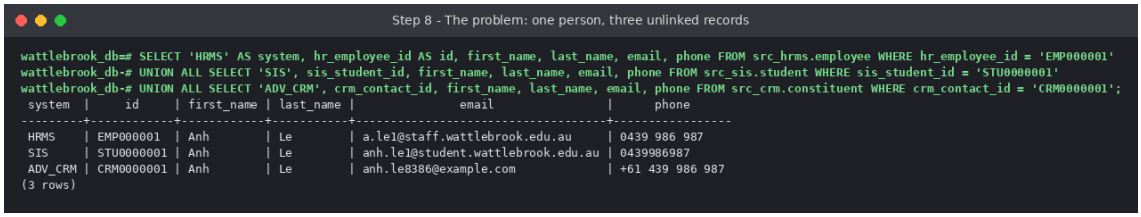



**Figure 5: Step 8 – the problem: one person (Anh Le) held in three systems with different IDs, emails and phone formats**

Figures 6 to 8 show the three source tables as created by `sql/02_source_tables.sql`, each with its business key as the primary key. Figure 9 shows the rows loaded from the workbook. The shadow-copy table shows 2,138 rows because the screenshot was taken after retention had deleted the 25,362 expired copies (UC1 in Section 3.4).

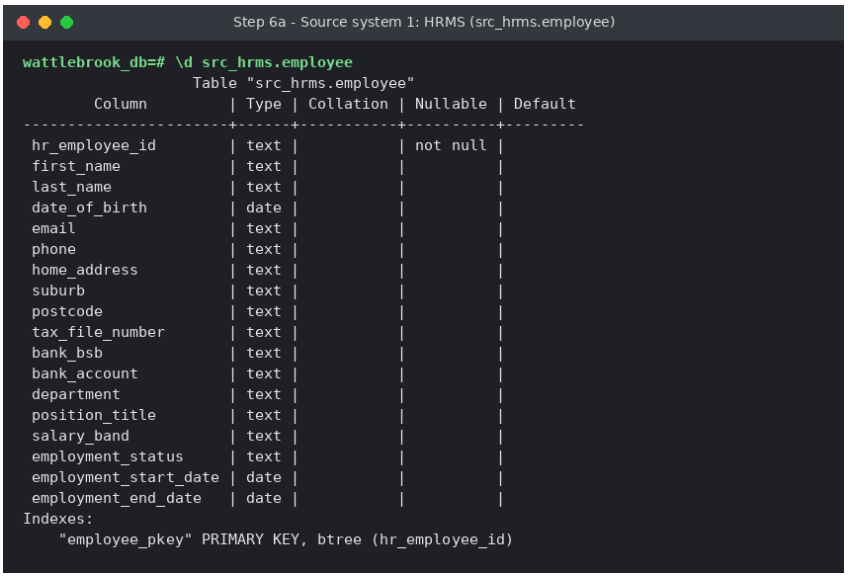



**Figure 6: Step 6a – source system 1: HRMS table src_hrms.employee**

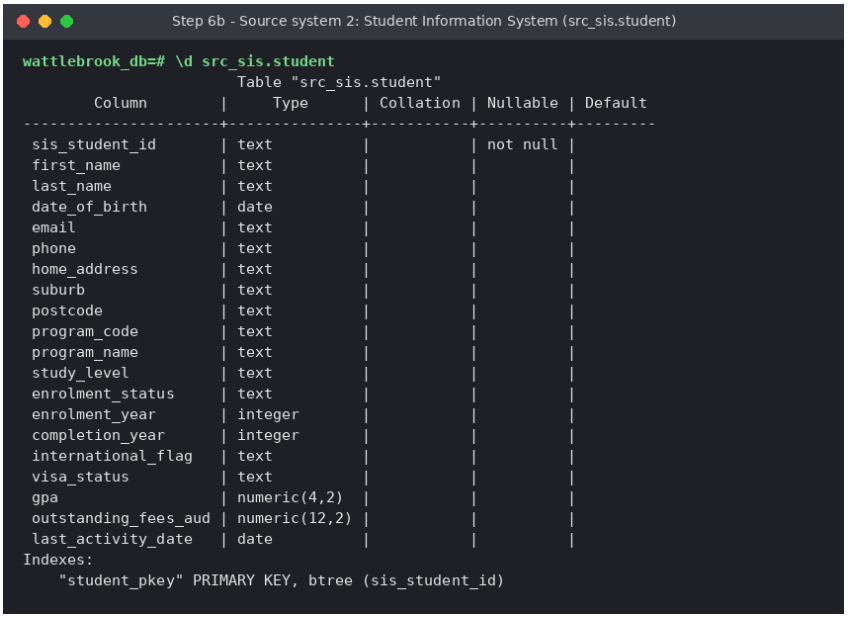



**Figure 7: Step 6b – source system 2: SIS table src_sis.student**

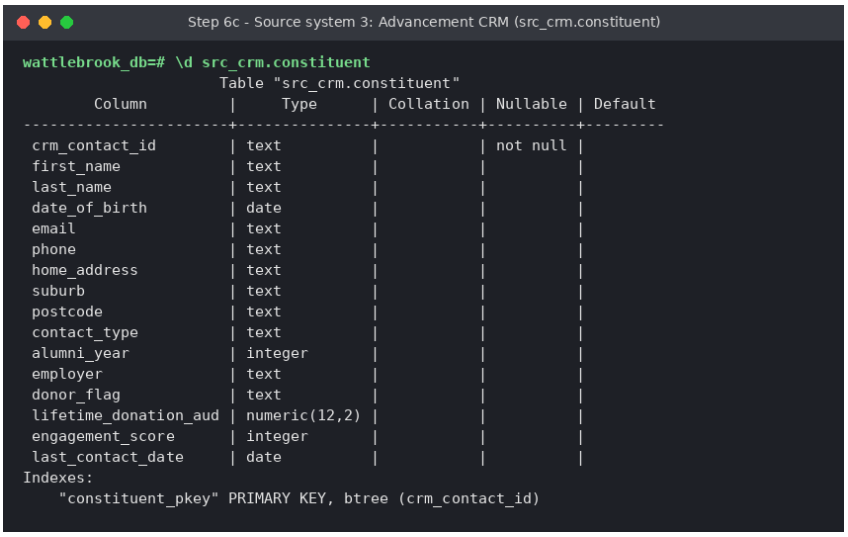



**Figure 8: Step 6c – source system 3: Advancement CRM table src_crm.constituent**

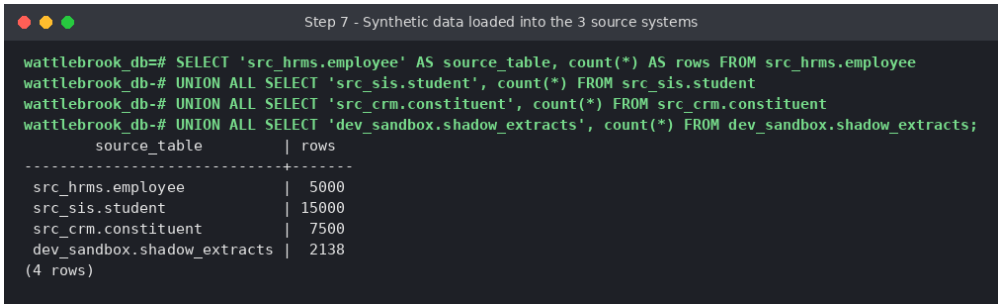



**Figure 9: Step 7 – rows in the three source systems, and the shadow copies left after retention**

## 3.2. Integrated Data Warehouse Database and Schema

The integrated warehouse sits in the same database, in seven schemas (Table 4). Bronze, silver and gold form the warehouse proper. `test_masked` is the developer copy, and `audit`, `qa` and `reports` support governance, testing and reporting.

**Table 4: Warehouse schemas and their contents after a clean run**

| Schema | Tables and views | Rows | Role | Evidence |
|---|---|---|---|---|
| bronze | hrms_employee, sis_student, crm_constituent | 5,000 / 15,000 / 7,500 | Raw copy, reloaded each run | Figure 11 |
| silver | hub_employee, hub_student, hub_constituent; sat_employee, sat_student, sat_constituent; hub_person, link_person; 3 *_current views | 27,500 hub keys; 27,500 satellite rows; 20,399 people; 27,500 links | Integrated Data Vault with identity | Figures 12, 21 |
| gold | dim_person, dim_program, fact_enrolment, fact_giving | 20,399 / 12 / 15,000 / 7,500 | Star schema for reporting | Figures 13, 14 |

| Schema | Tables and views | Rows | Role | Evidence |
|---|---|---|---|---|
| test_masked | employee, student, constituent | 4,439 / 8,242 / 5,849 | Masked test data | Figures 23, 24 |
| audit | pipeline_run, lineage, dataset, pii_column, retention_policy, purge_log, masking_key | 23 datasets; 129 personal-data columns | Governance plane | Figures 25, 26, 30 |
| qa | identity_answer_key | 27,500 | Scores the matching only | Figure 19 |
| reports | 9 views | – | Reports and end-to-end test | Figures 20, 22, 27, 28, 31 |

Figures 10 to 14 show the warehouse in the running database: the schemas, the bronze copy of every source row, the silver Data Vault with 27,500 hub keys and links grouped into 20,399 people, and the gold star schema with its foreign keys and row counts.

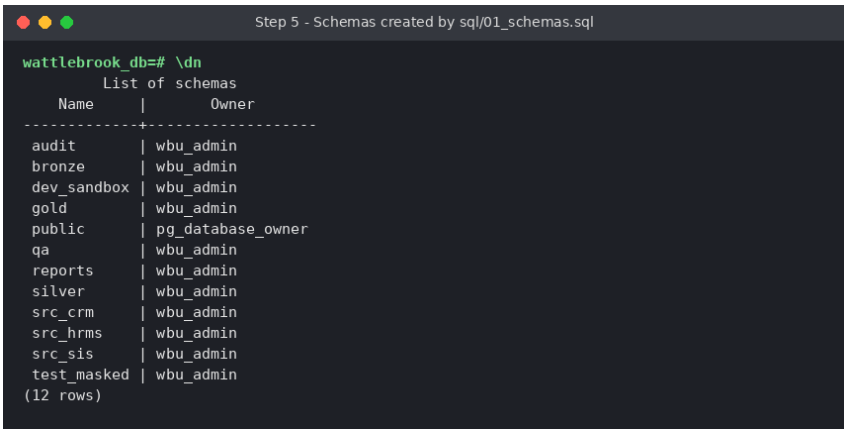



**Figure 10: Step 5 – schemas in wattlebrook_db created by sql/01_schemas.sql**

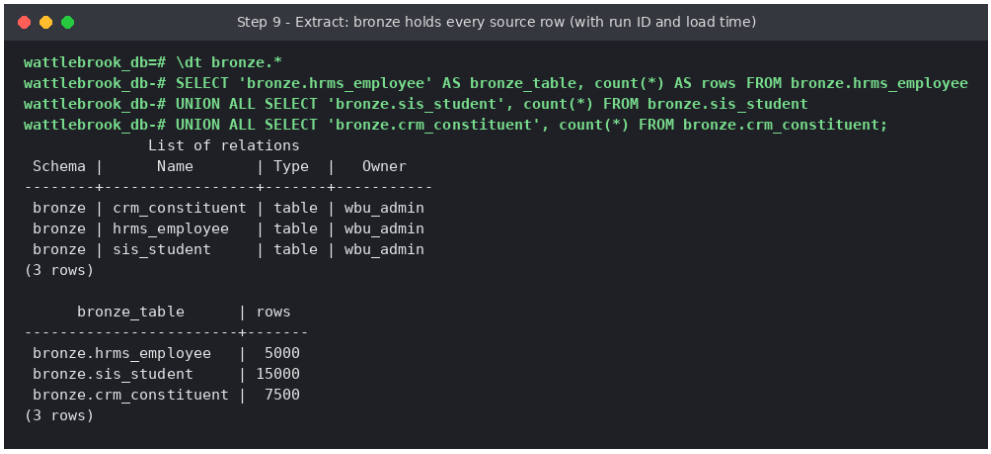



**Figure 11: Step 9 – bronze layer: one raw table per source with every source row**

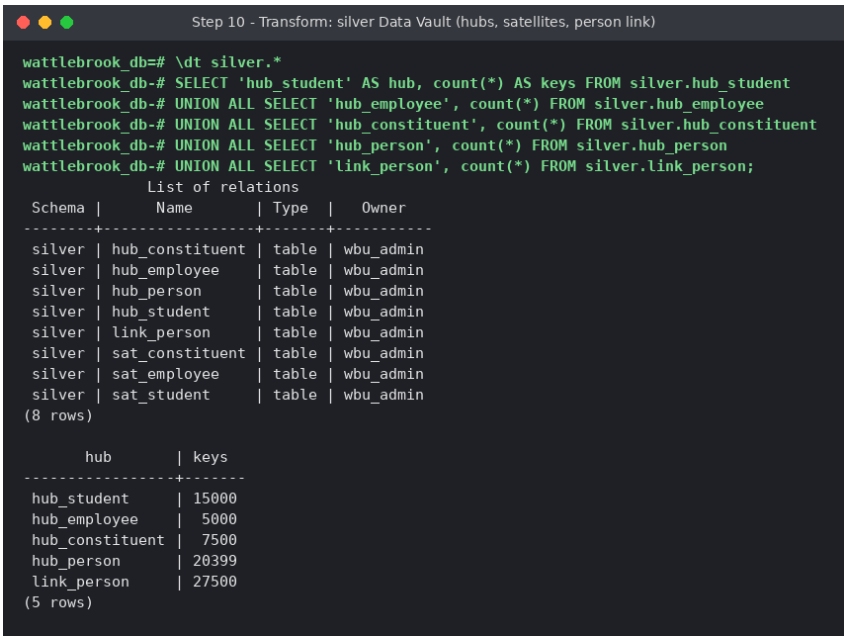



**Figure 12: Step 10 – silver Data Vault: hubs, satellites, person hub and link, with key counts**

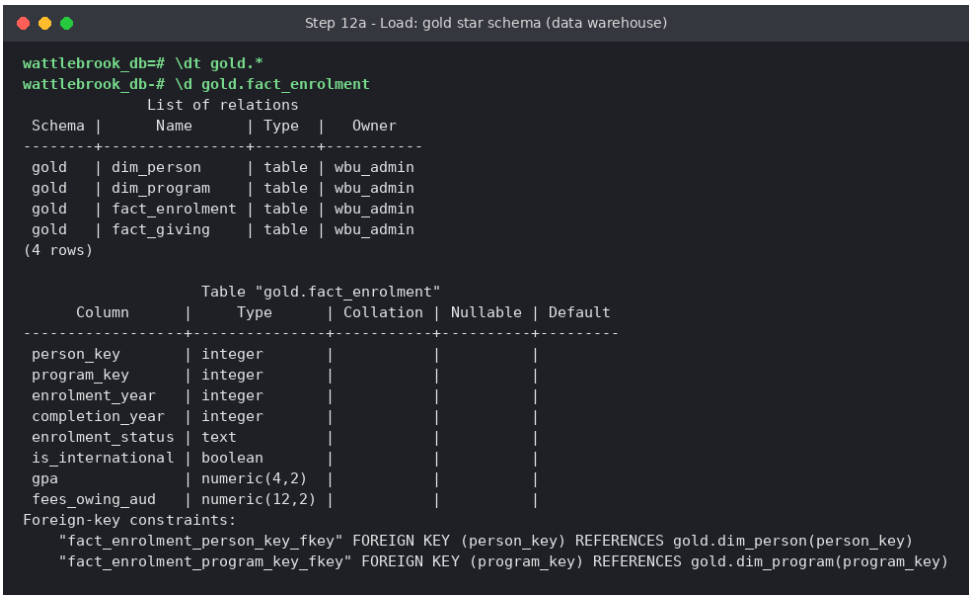



**Figure 13: Step 12a – gold star schema tables and the fact_enrolment foreign keys**

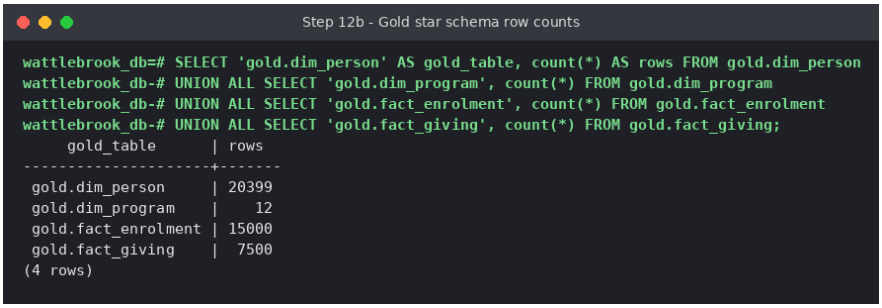



**Figure 14: Step 12b – gold star schema row counts**

### 3.3. SQL Scripts and Mapping to the Solution Design

The platform is built by eleven numbered SQL files, run in order by `run_pipeline.py`. Each file uses CREATE ... IF NOT EXISTS or CREATE OR REPLACE, so it can be run again on an existing database, and each file begins with a comment that names the design component it implements. Table 5 maps every script to that component, and Appendix 6.5 lists every script in full, with its comments.

**Table 5: Mapping of scripts to solution design components**

| Script | What it creates or does | Design component (Figure 1) |
|---|---|---|
| sql/01_schemas.sql | 11 schemas (source, layers, audit, qa, reports); pgcrypto extension | Layer structure |
| sql/02_source_tables.sql | src_hrms.employee, src_sis.student, src_crm.constituent, dev_sandbox.shadow_extracts, qa.identity_answer_key | 1 Source systems |
| sql/03_audit.sql | pipeline_run, dataset (22 registered datasets), pii_column, lineage, retention_policy (RP-01 to RP-04), purge_log | Governance plane |
| sql/04_bronze.sql | Bronze tables; bronze.load_source() with row-count check | 2 Bronze (INGEST) |
| sql/05_silver_vault.sql | Cleaning functions; hubs; satellites; *_current views; silver.load_vault() | 3 Silver (VAULT_LOAD) |
| sql/06_identity.sql | hub_person, link_person; silver.resolve_identities() with rules R1–R3 | 3 Identity resolution (MATCH) |
| sql/07_catalogue_scan.sql | audit.pii_category(); audit.scan_catalogue() | Catalogue and discovery |
| sql/08_retention.sql | audit.run_retention(): RP-01 to RP-03 anonymise, RP-04 delete and quarantine | Retention (PURGE) |
| sql/09_masking.sql | masking_key; pseudo(), fake_*() and shift_date(); test_masked tables; test_masked.provision() | 4 test_masked (MASK) |
| sql/10_gold.sql | Star schema; gold.build() | 4 Gold (BUILD) |
| sql/11_reports.sql | Nine report views, including reports.checks | 5 Reports and testing |
| python/ load_source_data.py | Copies the Excel sheets into the source tables (EXTRACT) | 1 Source systems |
| python/run_pipeline.py | Runs the SQL files, then the ten steps, each in its own transaction | Orchestration |
| python/dashboard.py | Reads the report views and gold, and draws the dashboard (Figure 29) | 5 Reports |
| sql/demo_queries.sql, sql/demo_hubs_and_link.sql | Read-only queries used in the demo, plus the failure demo | Demonstration |

Figure 15 shows the numbered SQL files in the repository and the start of `sql/04_bronze.sql`.

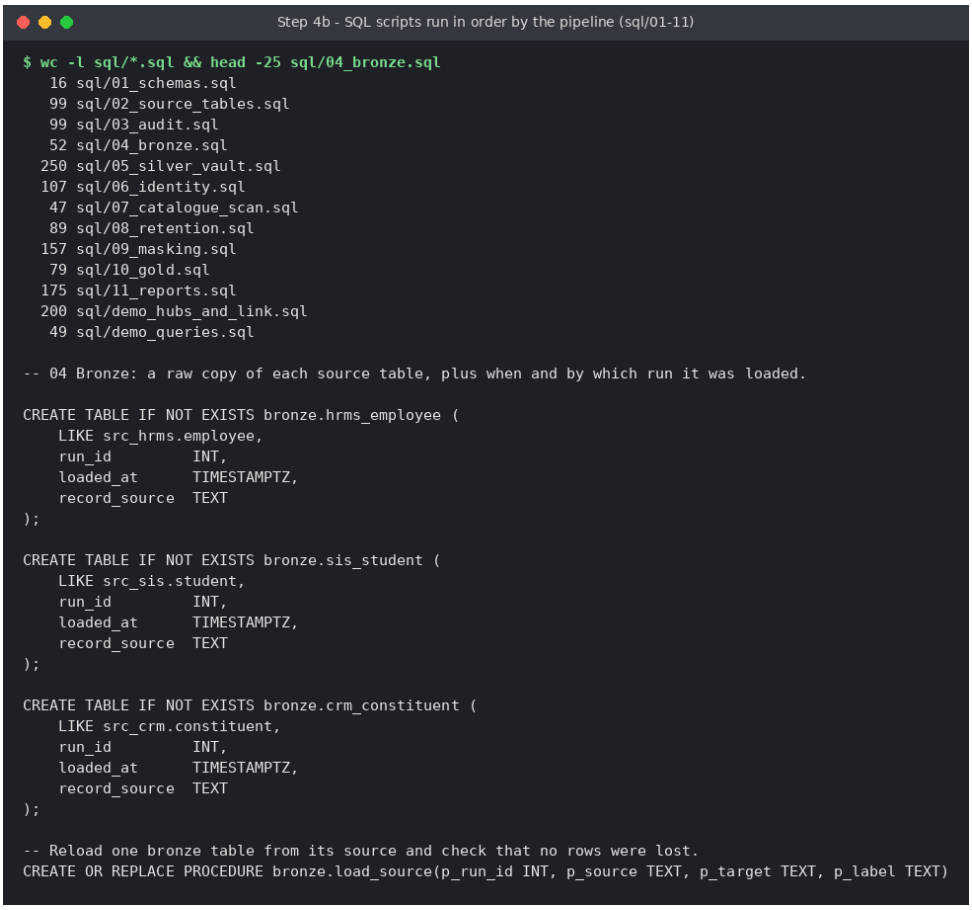



**Figure 15: Step 4b – SQL scripts run in order by the pipeline (sql/01–11), with the start of sql/04_bronze.sql (taken before the final comments were added; Appendix 6.5 lists the final scripts)**

The prototype is built and tested from an empty database with these commands:

```
cp .env.example .env
python -m venv .venv && .venv/bin/pip install -r requirements.txt
docker compose up -d --wait
.venv/bin/python python/run_pipeline.py --reset
docker exec -it wbu_postgres_db psql -U wbu_admin -d wattlebrook_db
SELECT * FROM reports.checks;
```

Figures 16 to 19 show these steps on the test machine: the repository is cloned, the `.env` file is created and the Python packages are installed, PostgreSQL 16 is started, and the full pipeline is run from an empty database with `--reset`, which took 32.8 seconds.

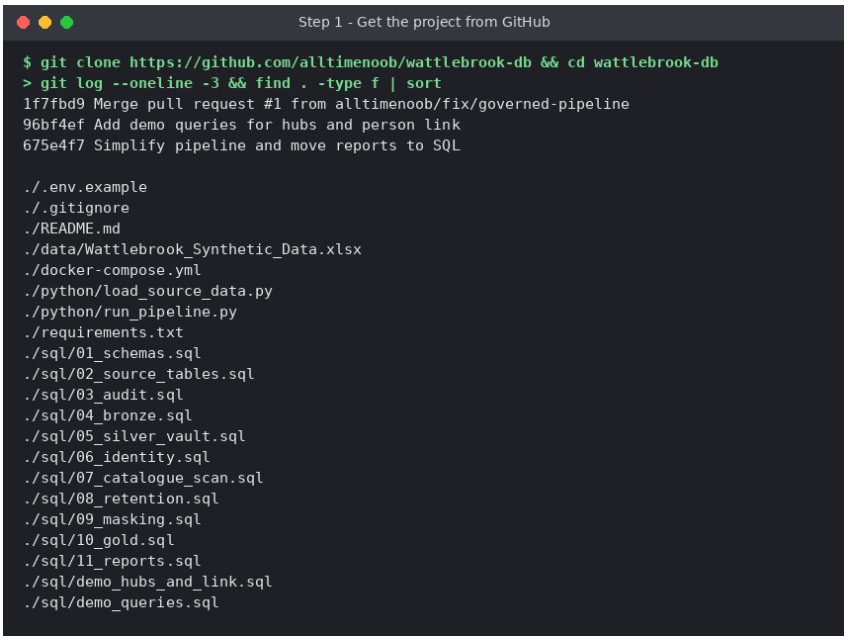



**Figure 16: Step 1 – cloning the project repository from GitHub (an earlier copy of the repository; the final code is at github.com/AryaK-31/wattlebrook-db)**

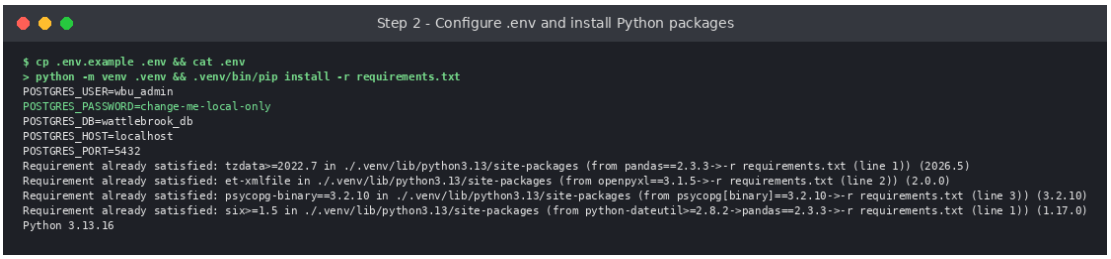



**Figure 17: Step 2 – creating .env and installing the Python packages**

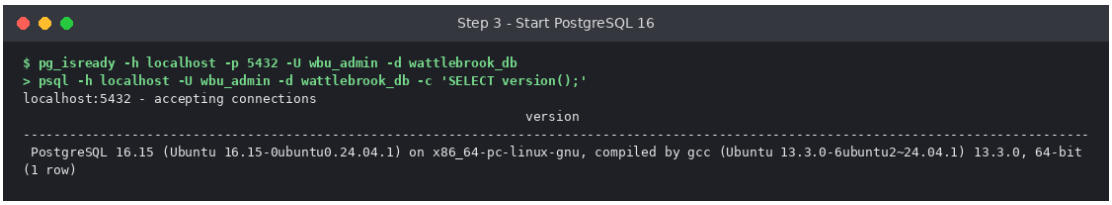



**Figure 18: Step 3 – PostgreSQL 16 accepting connections**

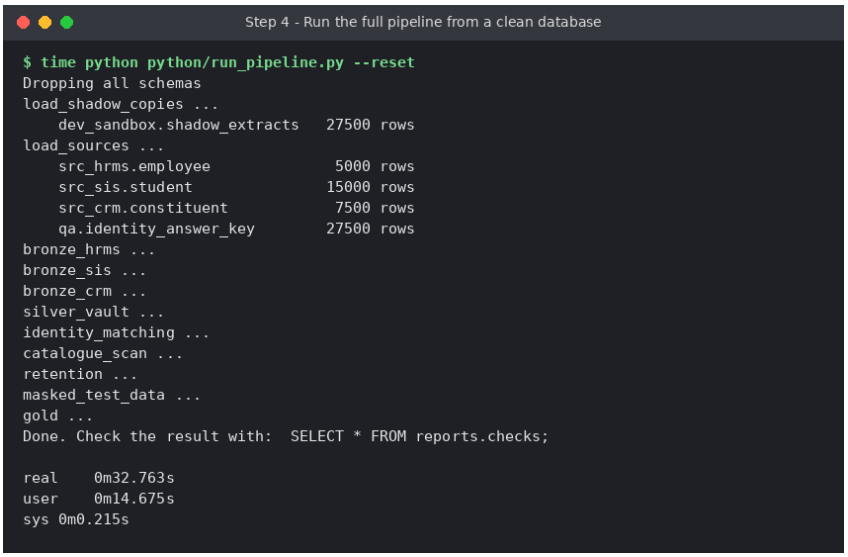



**Figure 19: Step 4 – full pipeline run from an empty database (--reset)**

Three short excerpts show how the design appears in the code. The first is from `silver.load_vault()`: a satellite row is inserted only for a new key or when the hash of the data changed, which keeps history and makes reruns safe.

```
INSERT INTO silver.sat_student
SELECT n.*
  FROM (SELECT md5(b.sis_student_id) AS student_hk, now() AS load_dts,
        md5(ROW(b.first_name, ..., b.last_activity_date)::TEXT) AS hash_diff, ...
          FROM bronze.sis_student b) n
  LEFT JOIN silver.student_current c ON c.student_hk = n.student_hk
 WHERE c.hash_diff IS DISTINCT FROM n.hash_diff;   -- new key, or the data changed
```

The second is from `silver.resolve_identities()`. Pairs that match a rule are collected, then each record repeatedly takes the smallest group name among the records it matches until nothing changes, which joins chains of matches into one person.

```
SELECT a.k, b.k, 'R3' FROM rec a JOIN rec b
    ON a.first_name = b.first_name AND a.last_name = b.last_name
   AND a.dob = b.dob AND a.sys < b.sys;              -- only across systems
LOOP
  UPDATE rec r SET grp = m.grp FROM (...) m WHERE r.k = m.k AND m.grp < r.grp;
  GET DIAGNOSTICS v_changed = ROW_COUNT;
  EXIT WHEN v_changed = 0;
END LOOP;
```

The third is from `audit.run_retention()`. Personal details of expired students are blanked in every satellite version, but the record, program and results are kept. Because `hash_diff` is not changed, the next load sees no change and does not copy the details back in.

```
UPDATE silver.sat_student
   SET first_name = NULL, last_name = NULL, date_of_birth = NULL,
       email = NULL, phone = NULL, home_address = NULL, visa_status = NULL,
       anonymised_at = now()
 WHERE anonymised_at IS NULL
   AND student_hk IN (SELECT student_hk FROM silver.student_current
                      WHERE retention_expiry_date < current_date
                        AND anonymised_at IS NULL);
```

### 3.4. Reports and Dashboard per Data Use Case

Reports are SQL views in the `reports` schema, so they always show the current state and any SQL client or BI tool can read them. Table 6 maps the five data use cases to their reports. The figures below are the actual output of these views and queries after the clean run on 15 October 2026.

**Table 6: Data use cases and their reports**

| Use case | Question answered | Report | Main user | Evidence |
|---|---|---|---|---|
| UC1 Retention compliance | Which personal data is past its use-by date, and what was removed? | reports.retention_results, reports.retention_overdue | Privacy officer | Figure 20 |
| UC2 Single view of a person | How many real people does WBU hold, in which systems, and how accurate is the matching? | reports.identity_overlap, reports.identity_accuracy | Data stewards, planning | Figures 21, 22 |
| UC3 Safe test data | Can developers test without real personal data? | reports.masking_check (+ test_masked.*) | Developers, security | Figures 23, 24 |
| UC4 Where data is | Which tables exist, who owns them, which hold personal data and where did the data come from? | reports.catalogue | Data governance | Table 7; Figures 25, 26 |
| UC5 Integrated analytics | How do enrolment and giving look when counted per person? | Queries on the gold star schema | Advancement, planning | Figures 27, 28 |
| Operations | Did the last run work, and what failed? | reports.pipeline_runs, reports.failed_steps, reports.checks | Data platform team | Figures 30, 31, 35 |

#### UC1 Retention compliance

Figure 20 shows retention on the first run. RP-01 anonymised 6,758 of 15,000 students, RP-02 anonymised 561 former staff and RP-03 anonymised 1,651 CRM contacts who are not donors. RP-04 deleted 25,362 of the 27,500 shadow copies. The 2,138 copies left are still within their approved date, so the table was quarantined rather than dropped. `reports.retention_overdue` then returns no rows. Retention compares

use-by dates with the current date, so the counts change slightly from day to day; the README's run on 13 October gave 6,755, 560, 1,651 and 25,343.

Report 3 - Retention results (reports.retention_results)

```
wattlebrook_db=# SELECT * FROM reports.retention_results;
wattlebrook_db-# SELECT count(*) AS still_overdue FROM reports.retention_overdue;
wattlebrook_db-# SELECT environment, count(*) AS copies, min(retention_expiry_date) AS next_expiry
wattlebrook_db-#   FROM dev_sandbox.shadow_extracts GROUP BY environment ORDER BY copies DESC;
 policy_id |             description              |           target            |  action   | rows_before | rows_affected | rows_after
-----------+--------------------------------------+-----------------------------+-----------+-------------+---------------+------------
 RP-01     | Students who completed or withdrew   | silver.sat_student          | ANONYMISE |       15000 |          6758 |       8242
 RP-02     | Former staff                         | silver.sat_employee         | ANONYMISE |        5000 |           561 |       4439
 RP-03     | CRM contacts who are not donors      | silver.sat_constituent      | ANONYMISE |        7500 |          1651 |       5849
 RP-04     | Test copies past their approved date | dev_sandbox.shadow_extracts | DELETE    |       27500 |         25362 |       2138
(4 rows)

 still_overdue
--------------
             0
(1 row)

        environment         | copies | next_expiry
----------------------------+--------+-------------
 shared_test_drive          |    524 | 2026-10-15
 sandbox_test_team_b        |    405 | 2026-10-15
 developer_laptop_export    |    345 | 2026-10-15
 sandbox_dev_team_a         |    324 | 2026-10-15
 git_repo_legacy_admissions |    234 | 2026-10-15
 git_repo_alumni_portal     |    154 | 2026-10-16
 staging_crm_integration    |    152 | 2026-10-16
(7 rows)
```

**Figure 20: Report 3 (UC1) – reports.retention_results, reports.retention_overdue and the remaining shadow copies by environment**

### UC2 Single view of a person

Figure 5 in Section 3.1 follows one person. Anh Le has three unrelated IDs and three different emails, yet `silver.link_person` ties all three records to person P-3EBCF1622F through rules R2 and R3 (Figure 21). Figure 22 shows the whole population: 13,974 people appear in one system, 5,749 in two and 676 in all three. Of the 7,777 links the matcher made, 7,762 are correct. The 15 wrong links join different people who share a name and date of birth, which is why R3 alone is only MEDIUM confidence. Precision is 99.81% and recall 97.03%.

Step 11 - Identity matching links the three records to one person

```
wattlebrook_db=# SELECT * FROM silver.link_person
wattlebrook_db-#  WHERE person_id = (SELECT person_id FROM silver.link_person WHERE source_record_id = 'STU0000001');
  person_id   | system_name | source_record_id | match_rule | confidence |           load_dts
--------------+-------------+------------------+------------+------------+-------------------------------
 P-3EBCF1622F | ADV_CRM     | CRM0000001       | R2+R3      | HIGH       | 2026-10-15 07:55:20.190686+00
 P-3EBCF1622F | HRMS        | EMP000001        | R2+R3      | HIGH       | 2026-10-15 07:55:20.190686+00
 P-3EBCF1622F | SIS         | STU0000001       | R2+R3      | HIGH       | 2026-10-15 07:55:20.190686+00
(3 rows)
```

**Figure 21: Step 11 (UC2) – silver.link_person ties the three records of Anh Le to one person_id**

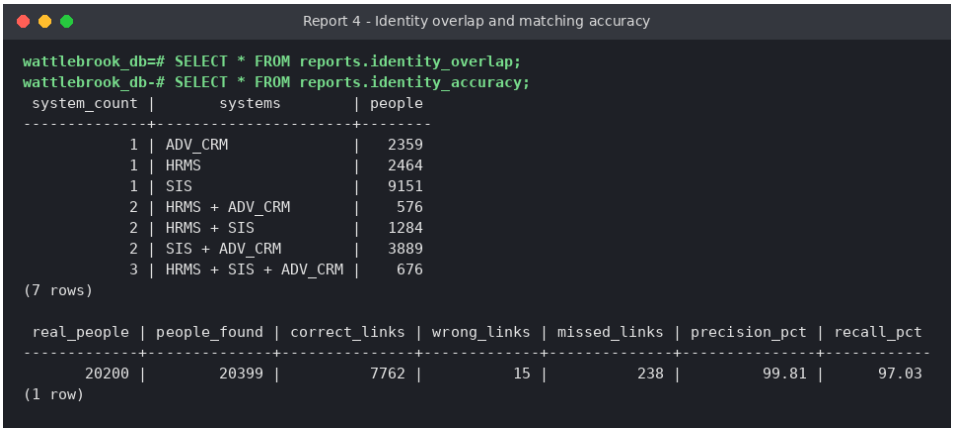



**Figure 22: Report 4 (UC2) – reports.identity_overlap and reports.identity_accuracy**

## UC3 Safe test data

Figure 23 shows the same person's HRMS and SIS records in the masked tables: invented names, `example.invalid` emails and phone numbers from an unused range, but one shared person reference, so test cases that span systems still work. `reports.masking_check` compares the 18,530 masked rows with the real source data and finds no real emails, phone numbers, first names, name and date-of-birth combinations, source IDs, real student dates of birth or anonymised people (Figure 24).

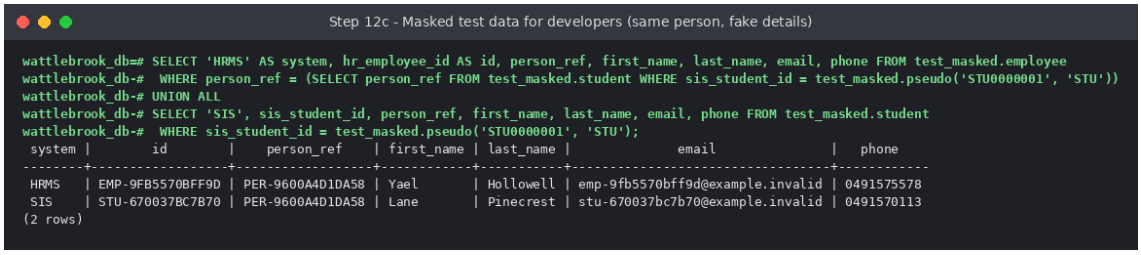



**Figure 23: Step 12c (UC3) – the same person in the masked test tables: invented details, one shared person_ref**

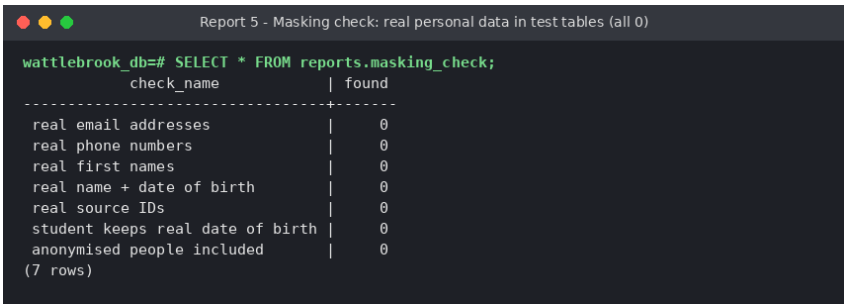



**Figure 24: Report 5 (UC3) – reports.masking_check finds no real personal data in the masked tables**

### UC4 Where data is

Table 7 is the output of `reports.catalogue`. All 23 tables are listed with their owner, whether they hold real personal data, how many personal-data columns the scan found and where their data came from. The shadow-copy table was not registered. The scan found it (owner UNKNOWN, no lineage), and retention then set its status to QUARANTINED. The scan classified 129 columns across 15 tables: 62 direct identifiers, 34 quasi-identifiers, 13 sensitive and 20 financial.

**Table 7: Output of reports.catalogue after the clean run**

| dataset | status | owner | real PII | PII cols | comes_from |
|---|---|---|---|---|---|
| bronze.crm_constituent | REGISTERED | Data Platform Team | t | 9 | src_crm.constituent |
| bronze.hrms_employee | REGISTERED | Data Platform Team | t | 12 | src_hrms.employee |
| bronze.sis_student | REGISTERED | Data Platform Team | t | 12 | src_sis.student |
| dev_sandbox.shadow_extracts | QUARANTINED | UNKNOWN | t | 7 | unknown (no lineage) |
| gold.dim_person | REGISTERED | Data Platform Team | f | 0 | silver.link_person |
| gold.dim_program | REGISTERED | Data Platform Team | f | 0 | silver.sat_student |
| gold.fact_enrolment | REGISTERED | Data Platform Team | f | 1 | silver.sat_student |
| gold.fact_giving | REGISTERED | Data Platform Team | f | 1 | silver.sat_constituent |
| qa.identity_answer_key | REGISTERED | Data Platform Team | f | 0 | Wattlebrook_Synthetic_Data.xlsx / Identity_Answer_Key |
| silver.hub_constituent | REGISTERED | Data Platform Team | f | 0 | bronze.crm_constituent |
| silver.hub_employee | REGISTERED | Data Platform Team | f | 0 | bronze.hrms_employee |
| silver.hub_person | REGISTERED | Data Platform Team | f | 0 | bronze (all three systems) |
| silver.hub_student | REGISTERED | Data Platform Team | f | 0 | bronze.sis_student |
| silver.link_person | REGISTERED | Data Platform Team | f | 0 | bronze (all three systems) |
| silver.sat_constituent | REGISTERED | Advancement Office | t | 9 | bronze.crm_constituent |
| silver.sat_employee | REGISTERED | People & Culture | t | 12 | bronze.hrms_employee |
| silver.sat_student | REGISTERED | Student Administration | t | 12 | bronze.sis_student |
| src_crm.constituent | REGISTERED | Advancement Office | t | 9 | Wattlebrook_Synthetic_Data.xlsx / ADV_CRM |
| src_hrms.employee | REGISTERED | People & Culture | t | 12 | Wattlebrook_Synthetic_Data.xlsx / HRMS |
| src_sis.student | REGISTERED | Student Administration | t | 12 | Wattlebrook_Synthetic_Data.xlsx / SIS |
| test_masked.constituent | REGISTERED | Data Platform Team | f | 6 | silver.sat_constituent |
| test_masked.employee | REGISTERED | Data Platform Team | f | 7 | silver.sat_employee |
| test_masked.student | REGISTERED | Data Platform Team | f | 8 | silver.sat_student |

Figure 25 shows the same view in psql. Figure 26 shows the 23 lineage rows written by the clean run, from the EXTRACT of each workbook sheet, through INGEST, VAULT_LOAD and MATCH, to the MASK and BUILD of each consumption table. The PURGE row records the 25,362 shadow copies deleted by retention.

Report 2 - Data catalogue and lineage (reports.catalogue)

```
wattlebrook_db=# SELECT * FROM reports.catalogue;
          dataset           |    layer    |   status    |        owner         | real_pii | pii_columns |                    comes_from
----------------------------+-------------+-------------+----------------------+----------+-------------+-----------------------------------------------
 bronze.crm_constituent     | bronze      | REGISTERED  | Data Platform Team   | t        |           9 | src_crm.constituent
 bronze.hrms_employee       | bronze      | REGISTERED  | Data Platform Team   | t        |          12 | src_hrms.employee
 bronze.sis_student         | bronze      | REGISTERED  | Data Platform Team   | t        |          12 | src_sis.student
 dev_sandbox.shadow_extracts | dev_sandbox | QUARANTINED | UNKNOWN              | t        |           7 | unknown (no lineage)
 gold.dim_person            | gold        | REGISTERED  | Data Platform Team   | f        |           0 | silver.link_person
 gold.dim_program           | gold        | REGISTERED  | Data Platform Team   | f        |           0 | silver.sat_student
 gold.fact_enrolment        | gold        | REGISTERED  | Data Platform Team   | f        |           1 | silver.sat_student
 gold.fact_giving           | gold        | REGISTERED  | Data Platform Team   | f        |           1 | silver.sat_constituent
 qa.identity_answer_key     | qa          | REGISTERED  | Data Platform Team   | f        |           0 | Wattlebrook_Synthetic_Data.xlsx / Identity_Answer_Key
 silver.hub_constituent     | silver      | REGISTERED  | Data Platform Team   | f        |           0 | bronze.crm_constituent
 silver.hub_employee        | silver      | REGISTERED  | Data Platform Team   | f        |           0 | bronze.hrms_employee
 silver.hub_person          | silver      | REGISTERED  | Data Platform Team   | f        |           0 | bronze (all three systems)
 silver.hub_student         | silver      | REGISTERED  | Data Platform Team   | f        |           0 | bronze.sis_student
 silver.link_person         | silver      | REGISTERED  | Data Platform Team   | f        |           0 | bronze (all three systems)
 silver.sat_constituent     | silver      | REGISTERED  | Advancement Office   | t        |           9 | bronze.crm_constituent
 silver.sat_employee        | silver      | REGISTERED  | People & Culture     | t        |          12 | bronze.hrms_employee
 silver.sat_student         | silver      | REGISTERED  | Student Administration | t      |          12 | bronze.sis_student
 src_crm.constituent        | source      | REGISTERED  | Advancement Office   | t        |           9 | Wattlebrook_Synthetic_Data.xlsx / ADV_CRM
 src_hrms.employee          | source      | REGISTERED  | People & Culture     | t        |          12 | Wattlebrook_Synthetic_Data.xlsx / HRMS
 src_sis.student            | source      | REGISTERED  | Student Administration | t      |          12 | Wattlebrook_Synthetic_Data.xlsx / SIS
 test_masked.constituent    | test_masked | REGISTERED  | Data Platform Team   | f        |           6 | silver.sat_constituent
 test_masked.employee       | test_masked | REGISTERED  | Data Platform Team   | f        |           7 | silver.sat_employee
 test_masked.student        | test_masked | REGISTERED  | Data Platform Team   | f        |           8 | silver.sat_student
(23 rows)
```

**Figure 25: Report 2 (UC4) – reports.catalogue in psql**

Report 2b - Lineage of the latest data movements (audit.lineage)

```
wattlebrook_db=# SELECT lineage_id, source, target, operation, row_count FROM audit.lineage ORDER BY lineage_id DESC LIMIT 25;
 lineage_id |                          source                          |         target          | operation  | row_count
------------+----------------------------------------------------------+-------------------------+------------+-----------
         23 | silver.sat_constituent                                   | gold.fact_giving        | BUILD      |      7500
         22 | silver.sat_student                                       | gold.fact_enrolment     | BUILD      |     15000
         21 | silver.sat_student                                       | gold.dim_program        | BUILD      |        12
         20 | silver.link_person                                       | gold.dim_person         | BUILD      |     20399
         19 | silver.sat_constituent                                   | test_masked.constituent | MASK       |      5849
         18 | silver.sat_student                                       | test_masked.student     | MASK       |      8242
         17 | silver.sat_employee                                      | test_masked.employee    | MASK       |      4439
         16 | dev_sandbox.shadow_extracts                              | (deleted)               | PURGE      |     25362
         15 | bronze (all three systems)                               | silver.link_person      | MATCH      |     27500
         14 | bronze (all three systems)                               | silver.hub_person       | MATCH      |     20399
         13 | bronze.crm_constituent                                   | silver.sat_constituent  | VAULT_LOAD |      7500
         12 | bronze.crm_constituent                                   | silver.hub_constituent  | VAULT_LOAD |      7500
         11 | bronze.sis_student                                       | silver.sat_student      | VAULT_LOAD |     15000
         10 | bronze.sis_student                                       | silver.hub_student      | VAULT_LOAD |     15000
          9 | bronze.hrms_employee                                     | silver.sat_employee     | VAULT_LOAD |      5000
          8 | bronze.hrms_employee                                     | silver.hub_employee     | VAULT_LOAD |      5000
          7 | src_crm.constituent                                      | bronze.crm_constituent  | INGEST     |      7500
          6 | src_sis.student                                          | bronze.sis_student      | INGEST     |     15000
          5 | src_hrms.employee                                        | bronze.hrms_employee    | INGEST     |      5000
          4 | Wattlebrook_Synthetic_Data.xlsx / Identity_Answer_Key    | qa.identity_answer_key  | EXTRACT    |     27500
          3 | Wattlebrook_Synthetic_Data.xlsx / ADV_CRM                | src_crm.constituent     | EXTRACT    |      7500
          2 | Wattlebrook_Synthetic_Data.xlsx / SIS                    | src_sis.student         | EXTRACT    |     15000
          1 | Wattlebrook_Synthetic_Data.xlsx / HRMS                   | src_hrms.employee       | EXTRACT    |      5000
(23 rows)
```

**Figure 26: Report 2b (UC4) – lineage of every data movement in the clean run (audit.lineage)**

## UC5 Integrated analytics

The gold star schema answers questions that need more than one system without exposing personal details. Figure 28 shows that 1,376 of the 2,050 donors (801 alumni who donate and 575 other donors) are also current or former students or staff. This can only be counted once identities are resolved. Figure 27 shows enrolments by study level and status, with the average GPA and the fees owing. These are ad hoc queries on gold; they could be saved as views in `reports` in the same way as the other reports.

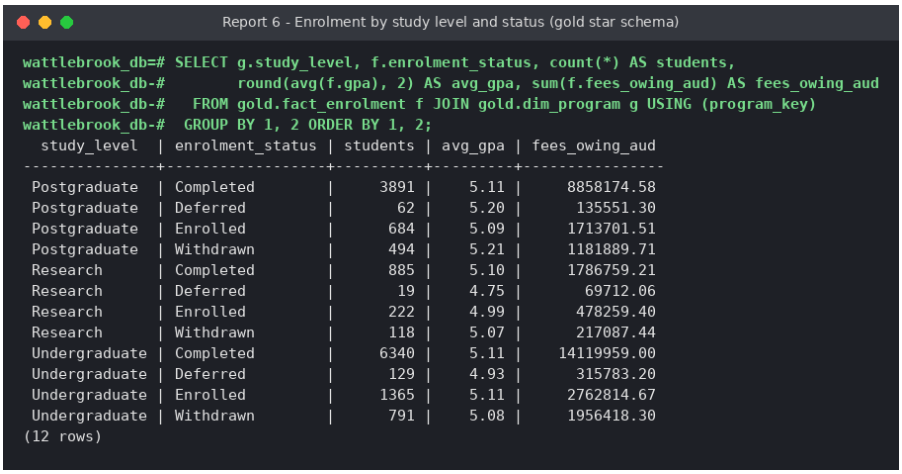



**Figure 27: Report 6 (UC5) – enrolments by study level and status from the gold star schema**

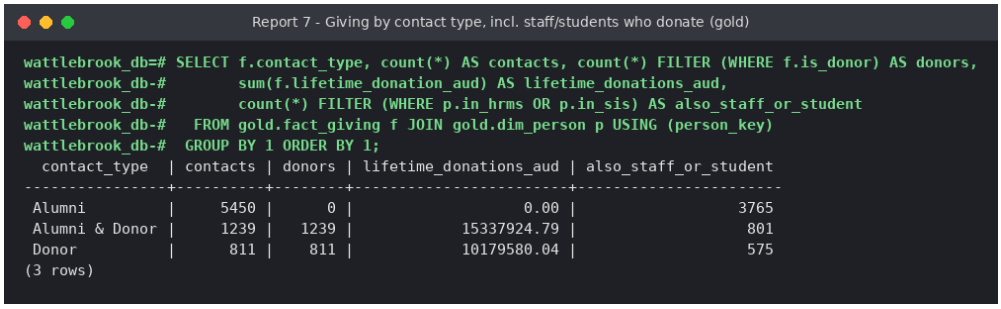



**Figure 28: Report 7 (UC5) – giving by contact type, including donors who are also staff or students**

**Dashboard.** `python/dashboard.py` reads the report views and the gold star schema and draws them as one dashboard page (Figure 29). Five tiles at the top show the state of the platform after the last run: the end-to-end checks, the number of people resolved, the matching accuracy, personal data past its use-by date and the number of masked test rows. Six charts below them cover UC1 (retention), UC2 (people by system), UC4 (catalogue by layer), UC5 (enrolments and donors) and the run time of each pipeline step. The script only reads data and shows no names, emails, phone numbers or dates of birth. The same views can also be connected to a BI tool such as Power BI or Metabase without copying data.

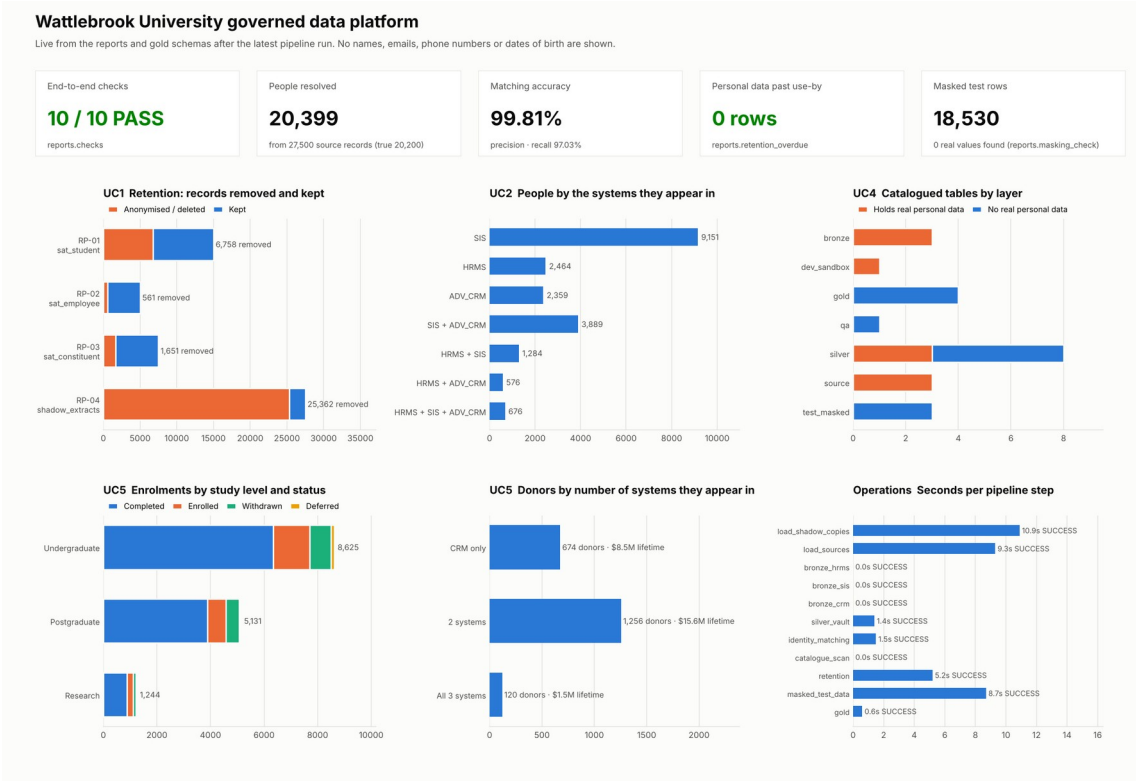



**Figure 29: Dashboard drawn by python/dashboard.py from the report views and the gold star schema after a separate clean run (step times vary between runs)**

## 3.5. End-to-end Testing

End-to-end testing is built into the platform as the view `reports.checks`, which runs ten checks across every layer (Table 8). Three tests were run: a clean rebuild, a second run on top of it, and a deliberate failure.

**Table 8: End-to-end checks in reports.checks**

| Check | What it proves |
|---|---|
| every step of the last run succeeded | The pipeline ran from start to finish |
| bronze has every source row | No rows were lost in ingestion |
| silver has a hub row for every source record | Every business key reached the vault |
| every source record is linked to one person | Identity resolution covered all 27,500 records |
| matching precision is at least 99% | Few false merges, scored against the answer key |
| masked data holds no real personal data | All seven masking checks return 0 |
| no personal data past its retention date | Retention removed everything that was overdue |
| shadow copies are no longer FOUND_BY_SCAN | The discovered copies were dealt with |
| every table is in the catalogue | No unregistered tables remain |
| every registered dataset has lineage | Every dataset's source is recorded |

### Test 1: clean rebuild

`run_pipeline.py --reset` drops all schemas and rebuilds from the state the audit found (Figure 19). All eleven steps succeeded and all ten checks passed (Figures 30 and 31). The `load_sources` step loaded 55,000 rows: the 27,500 source records plus the 27,500 rows of the answer key.

Report 1 - Pipeline run log (reports.pipeline_runs)

```
wattlebrook_db=# SELECT * FROM reports.pipeline_runs;
 run_id |     step_name      | status  | rows_read | rows_loaded | error_message | seconds
--------+--------------------+---------+-----------+-------------+---------------+---------
      1 | load_shadow_copies | SUCCESS |           |       27500 |               |     6.8
      2 | load_sources       | SUCCESS |           |       55000 |               |     7.6
      3 | bronze_hrms        | SUCCESS |      5000 |        5000 |               |     0.0
      4 | bronze_sis         | SUCCESS |     15000 |       15000 |               |     0.1
      5 | bronze_crm         | SUCCESS |      7500 |        7500 |               |     0.1
      6 | silver_vault       | SUCCESS |           |       27500 |               |     1.3
      7 | identity_matching  | SUCCESS |     27500 |       20399 |               |     1.4
      8 | catalogue_scan     | SUCCESS |           |           1 |               |     0.1
      9 | retention          | SUCCESS |           |             |               |     4.8
     10 | masked_test_data   | SUCCESS |           |             |               |     9.2
     11 | gold               | SUCCESS |           |             |               |     0.6
(11 rows)
```

**Figure 30: Report 1 – reports.pipeline_runs for the clean run: all eleven steps succeeded**

Step 13 - End-to-end test: reports.checks (every row PASS)

```
wattlebrook_db=# SELECT * FROM reports.checks;
                  check_name                  | result |             detail
----------------------------------------------+--------+--------------------------------
 every step of the last run succeeded         | PASS   | 11 steps
 bronze has every source row                  | PASS   |
 silver has a hub row for every source record | PASS   |
 every source record is linked to one person  | PASS   |
 matching precision is at least 99%           | PASS   | 99.81% precision, 97.03% recall
 masked data holds no real personal data      | PASS   | 18530 masked rows
 no personal data past its retention date     | PASS   |
 shadow copies are no longer FOUND_BY_SCAN    | PASS   | QUARANTINED
 every table is in the catalogue              | PASS   |
 every registered dataset has lineage         | PASS   |
(10 rows)
```

**Figure 31: Step 13 – reports.checks after the clean run: 10 of 10 PASS**

### Test 2: second run (rerun safety)

Running the pipeline again without `--reset` reloaded bronze from the sources but added no satellite rows and no people, retention had nothing left to remove, and all ten checks still passed (Figures 32 and 33). Anonymised people stayed anonymised, because their unchanged `hash_diff` stops the source data being copied back in.

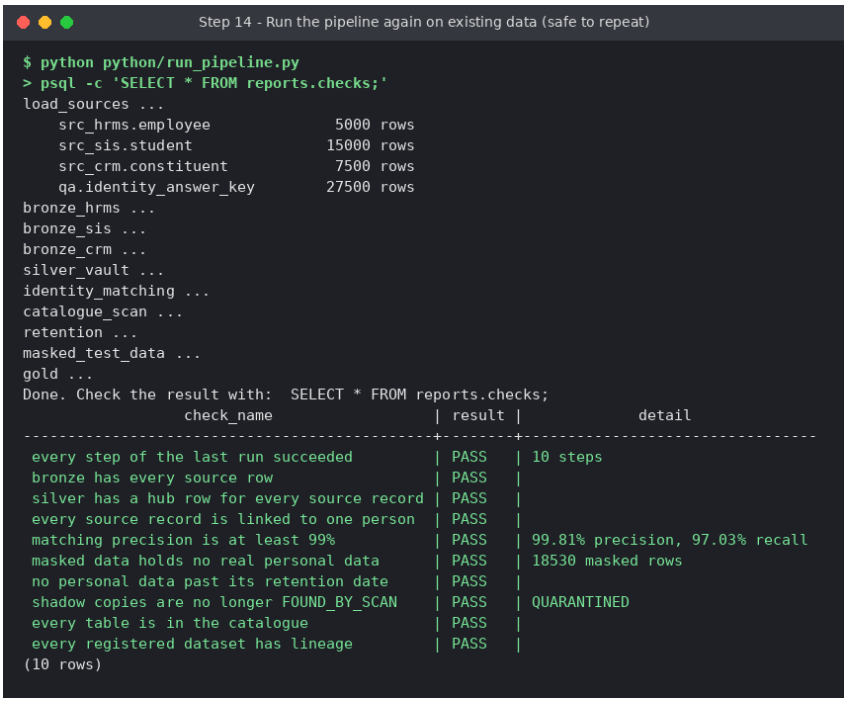



**Figure 32: Step 14 – second run on the existing data: all ten checks still pass**

```
-- satellite rows and people BEFORE the second run
 sat_student | sat_employee | sat_constituent | people
-------------+--------------+-----------------+--------
       15000 |         5000 |            7500 |  20399
(1 row)

-- AFTER the second run (no new history, no new people)
 sat_student | sat_employee | sat_constituent | people
-------------+--------------+-----------------+--------
       15000 |         5000 |            7500 |  20399
(1 row)

wattlebrook_db=# SELECT policy_id, rows_before, rows_affected, rows_after FROM
wattlebrook_db-#   reports.retention_results;
 policy_id | rows_before | rows_affected | rows_after
-----------+-------------+---------------+------------
 RP-01     |        8242 |             0 |       8242
 RP-02     |        4439 |             0 |       4439
 RP-03     |        5849 |             0 |       5849
 RP-04     |        2138 |             0 |       2138
(4 rows)

wattlebrook_db=# SELECT result, count(*) FROM reports.checks GROUP BY result;
 result | count
--------+-------
 PASS   |    10
(1 row)
```

**Figure 33: Step 14 – satellite rows and people unchanged by the second run, and nothing more for retention to remove**

### Test 3: failure handling

Following `sql/demo_queries.sql`, a column the pipeline does not expect was added to `src_sis.student` and the pipeline was run. The `bronze_sis` step failed, its work was rolled back and the pipeline stopped. The failure stays in `reports.failed_steps`, the first check turns to FAIL, and `bronze.sis_student`

still holds the 15,000 rows from the previous good load (Figures 34 and 35). Dropping the column and running again restores a full pass (Figure 36).

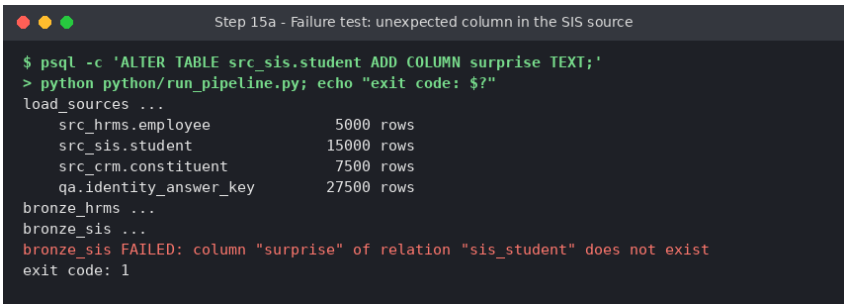



**Figure 34: Step 15a – an unexpected column in the SIS source makes the bronze_sis step fail**

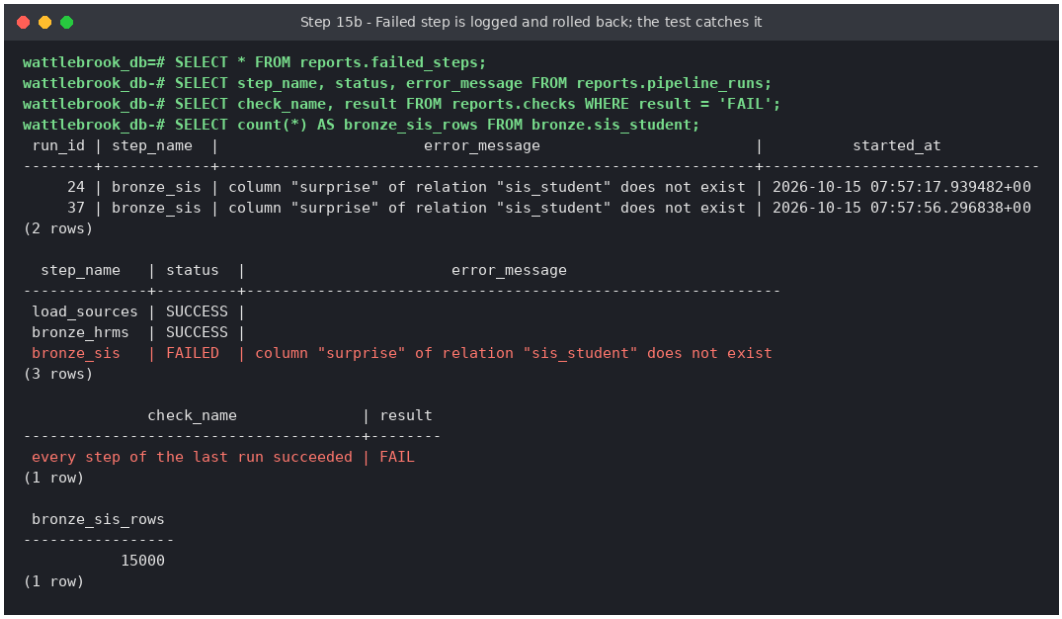



**Figure 35: Step 15b – the failed step is logged and rolled back, bronze keeps 15,000 rows and reports.checks shows FAIL**

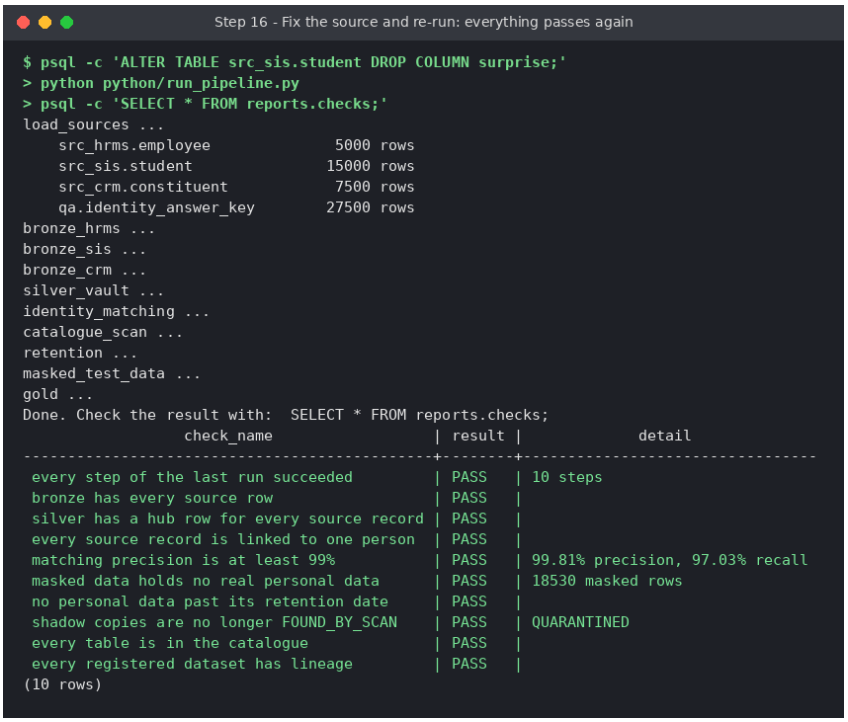



**Figure 36: Step 16 – after the column is dropped, a new run passes all ten checks again**

### Test 4: index look-up

`EXPLAIN ANALYZE` on a look-up by email in `silver.sat_student` uses the index `sat_student_email_idx` and runs in 0.09 ms (Figure 37), which supports the performance argument in Section 2.4.

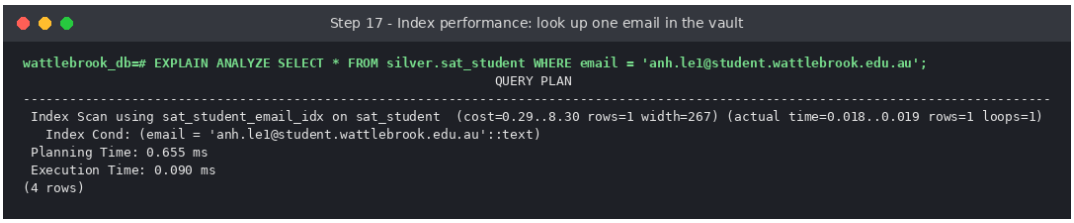



**Figure 37: Step 17 – a look-up by email uses the satellite email index (0.09 ms)**

Table 9 lists every step of the end-to-end test in the order it was run, what it shows and the screenshot that proves it. The step labels match the title bar of each screenshot.

### Table 9: End-to-end test steps and screenshot evidence

| Step | What was run | Result | Evidence |
|---|---|---|---|
| Step 1 | git clone of the project repository | 11 numbered SQL files, 2 Python scripts and the data workbook | Figure 16 |
| Step 2 | cp .env.example .env; pip install -r requirements.txt | Environment set; packages installed (Python 3.13) | Figure 17 |
| Step 3 | pg_isready; SELECT version() | PostgreSQL 16.15 accepting connections | Figure 18 |

| Step | What was run | Result | Evidence |
|---|---|---|---|
| Step 4 | python python/run_pipeline.py --reset | All steps run from an empty database in 32.8 s | Figure 19 |
| Step 4b | wc -l sql/*.sql; head of sql/04_bronze.sql | Scripts sql/01 to 11 present and commented | Figure 15 |
| Step 5 | \dn | All project schemas exist | Figure 10 |
| Step 6a–6c | \d on the three source tables | HRMS, SIS and CRM schemas with primary keys | Figures 6, 7, 8 |
| Step 7 | Row counts of the source tables | 5,000 / 15,000 / 7,500 rows; 2,138 shadow copies left | Figure 9 |
| Step 8 | The same person in the three sources | Three IDs, emails and phone formats for Anh Le | Figure 5 |
| Step 9 | Bronze tables and row counts | Every source row is in bronze | Figure 11 |
| Step 10 | Silver tables and key counts | 27,500 hub keys and links; 20,399 people | Figure 12 |
| Step 11 | silver.link_person for STU0000001 | Three records linked to P-3EBCF1622F (R2+R3, HIGH) | Figure 21 |
| Step 12a–12b | Gold tables, fact_enrolment and row counts | Star schema with foreign keys; 20,399 / 12 / 15,000 / 7,500 rows | Figures 13, 14 |
| Step 12c | Masked records for the same person | Invented details, one shared person_ref | Figure 23 |
| Report 1 | reports.pipeline_runs | 11 steps, all SUCCESS | Figure 30 |
| Report 2, 2b | reports.catalogue; audit.lineage | 23 catalogued tables; 23 lineage rows | Figures 25, 26 |
| Report 3 | reports.retention_results; reports.retention_overdue | 8,970 people anonymised, 25,362 copies deleted, 0 overdue | Figure 20 |
| Report 4 | reports.identity_overlap; reports.identity_accuracy | 20,399 people; 99.81% precision, 97.03% recall | Figure 22 |
| Report 5 | reports.masking_check | All seven checks find 0 | Figure 24 |
| Report 6, 7 | Enrolment and giving queries on gold | 15,000 enrolments; 1,376 donors also staff or students | Figures 27, 28 |
| Step 13 | SELECT * FROM reports.checks | 10 of 10 PASS | Figure 31 |
| Step 14 | python python/run_pipeline.py (second run) | 10 of 10 PASS; no new satellite rows or people | Figures 32, 33 |
| Step 15a–15b | Add a column to src_sis.student, then run | bronze_sis FAILED and rolled back; check FAIL; 15,000 rows kept | Figures 34, 35 |
| Step 16 | Drop the column, then run again | 10 of 10 PASS | Figure 36 |
| Step 17 | EXPLAIN ANALYZE look-up by email | Index scan, 0.09 ms | Figure 37 |

### Demo video

A recording of the end-to-end test is available at: https://www.youtube.com/watch?v=_rLFbb1bO5Y. Source code: https://github.com/AryaK-31/wattlebrook-db. The video follows the steps in Table 9 and `sql/demo_queries.sql`. It shows the clean rebuild with `--reset`, then the problem and the identity link for Anh Le, the masked test data, the run log, catalogue and lineage, retention, the identity and masking reports, and `reports.checks`, and finishes with the failure demo.

## 4. Summary and Conclusion

This report set out the design and working prototype of a governed data platform for Wattlebrook University. The design separates the data flow into source, bronze, silver and gold layers, with a governance plane that every layer writes to. The logical model implements the conceptual model as a Data Vault 2.0 silver layer with a person hub and link, a gold star schema without direct identifiers, and audit tables for the run log, lineage, catalogue and retention. PostgreSQL 16 was chosen because it suits a data set of this size, keeps the logic next to the data and costs nothing to run.

The prototype addresses each problem from Assignment 1. Manual copying is replaced by a logged pipeline that reconciles row counts and can be rerun safely. Identity resolution groups 27,500 records into 20,399 people with 99.81% precision, and every link records its rule and confidence. Developers get 18,530 masked rows that contain no real personal data. The catalogue lists every table, and the scan found the unregistered shadow copies. Retention anonymised 8,970 expired person records and deleted 25,362 expired copies. All ten end-to-end checks pass, a second run changes nothing, and a failed step is rolled back but stays visible.

The trade-offs are known. Rule-based matching misses about 3% of true links and makes a few false ones. Retention does not yet reach the source systems. The single superuser and the in-database masking key would not be acceptable in production. The next steps are therefore governance steps rather than new technology: review the MEDIUM matches, agree retention with source owners, add proper roles, and move the masking key to a secrets manager.

## 5. References


Arab, I. 2026, *Enterprise data modelling methodologies: a comparative analysis of Inmon, Kimball, and Data Vault*, arXiv preprint, viewed 15 October 2026, <https://doi.org/10.48550/arXiv.2606.29355>.

Azzabi, S., Alfughi, Z. & Ouda, A. 2024, 'Data lakes: a survey of concepts and architectures', *Computers*, vol. 13, no. 7, art. 183, viewed 15 October 2026, <https://doi.org/10.3390/computers13070183>.

Bay Information Systems 2026, *Bronze silver gold data architecture: building layered pipelines in PostgreSQL*, Bay Information Systems, viewed 15 October 2026, <https://www.bayis.co.uk/library/57-medallion.html>.

Christen, P. 2012, *Data matching: concepts and techniques for record linkage, entity resolution, and duplicate detection*, Springer, Berlin, viewed 15 October 2026, <https://doi.org/10.1007/978-3-642-31164-2>.

Gopa, R. & Goel, P. 2025, 'Data quality and governance for ensuring trustworthy and compliant data', *Journal of Quantum Science and Technology*, vol. 2, no. 2, viewed 15 October 2026, <https://doi.org/10.63345/jqst.v2i2.285>.

Hanumanthaiah, S. 2023, 'Data modeler's guide to implement Kimball dimensional modeling on Google BigQuery', *International Scientific Journal of Engineering and Management*, vol. 2, pp. 1–8, viewed 15 October 2026, <https://doi.org/10.55041/ISJEM00136>.

Henkoğlu, T. & Uçak, N. 2016, 'Information security and the protection of personal data in universities', *International Journal of Business and Management Invention*, vol. 5, pp. 30–43, viewed 15 October 2026, <https://www.researchgate.net/publication/315777031_Information_Security_and_the_Protection_of_Personal_Data_in_Universities>.

Kimball, R. & Ross, M. 2013, *The data warehouse toolkit: the definitive guide to dimensional modeling*, 3rd edn, Wiley, Indianapolis, IN.

Linstedt, D. & Olschimke, M. 2015, *Building a scalable data warehouse with Data Vault 2.0*, Morgan Kaufmann, Waltham, MA.

Navandar, P. 2024, *Unveiling the power of data masking: safeguarding sensitive information in the digital age*, ResearchGate, viewed 15 October 2026, <https://www.researchgate.net/publication/386874644_UNVEILING_THE_POWER_OF_DATA_MASKING_SAFEGUARDING_SENSITIVE_INFORMATION_IN_THE_DIGITAL_AGE>.

NSW Parliamentary Counsel's Office 1998, *Privacy and Personal Information Protection Act 1998* (NSW), NSW Legislation, viewed 15 October 2026, <https://legislation.nsw.gov.au/view/whole/html/inforce/current/act-1998-133>.

Obiokafor, C.I.I. & Aguboshim, F. 2025, 'Integrating privacy by design (PbD) in the system development life cycle for enhanced data protection', *World Journal of Advanced Research and Reviews*, vol. 26, no. 1, pp. 1233–40, viewed 15 October 2026, <https://doi.org/10.30574/wjarr.2025.26.1.0538>.

Office of the Australian Information Commissioner 2025, *The Privacy Act*, OAIC, Sydney, viewed 15 October 2026, <https://www.oaic.gov.au/privacy/privacy-legislation/the-privacy-act>.

PostgreSQL Global Development Group 2025, *pgcrypto: cryptographic functions*, PostgreSQL documentation, viewed 15 October 2026, <https://www.postgresql.org/docs/current/pgcrypto.html>.

# 6. Appendix

## 6.1. Individual Contribution Table

Tasks were first allocated at Meeting 2 on 23 September 2026 and confirmed at Meeting 4 on 3 October 2026. Table 10 lists each member's student ID, allocated task, the report sections they worked on and the contribution rating (0–100%) agreed by the group, using the rating scale in the assignment brief. The same ratings are recorded on the individual contribution form attached to this report.

**Table 10: Individual contribution table**

| Student name | Student ID | Allocated task | Report section | Contribution (0–100%) |
|---|---|---|---|---|
| Arya Mahesh Kaslikar | 26184921 | Source load, shadow copies, bronze and silver pipeline (load_source_data.py, sql/02, 04, 05) | 2.4 (technology), 3.1 | 100% |
| Abdul Mannan Mohammed | 26215537 | Gold star schema and use-case reports (sql/10, 11) | 2.3, 3.4 | 100% |
| Aditi Mangroliya | 26226264 | Masked test data and leak check (sql/09, reports.masking_check) | 2.4 (masking), 3.4 UC3 | 100% |
| Margi Govindbhai Vadodaria | 26240402 | Catalogue, lineage and discovery scan (sql/03, 07) | 2.4 (governance), 3.4 UC4 | 100% |
| Mihir Chauhan | 25620699 | Person hub, identity matching and accuracy (sql/06, reports.identity_*) | 2.4 (identity), 3.4 UC2 | 100% |
| Omkar Ravindran | 25630423 | Retention and end-to-end test (sql/08, reports.checks, run_pipeline.py) | 3.4 UC1, 3.5, 4 | 100% |

The completed individual contribution form (Canvas template) is attached at the end of this report, before the meeting minutes. It gives every member 100% for both the project and the presentation.

## 6.2. Individual Contributions (Evidence from Meeting Minutes)

The group recorded each member's work in the meeting minutes and on a shared Notion board rather than in separate logbooks. The evidence for each member below therefore refers to the minutes listed in Section 6.4, which are attached at the end of this report.

**Arya Mahesh Kaslikar**

Source load, shadow copies and the bronze and silver layers: `python/load_source_data.py`, `sql/02_source_tables.sql`, `sql/04_bronze.sql` and `sql/05_silver_vault.sql`. Evidence: Meeting 1 minutes (Notion board and architecture flowchart), Meetings 2 to 4 Assigned Responsibilities, and the Meeting 5 round-table notes.

**Abdul Mannan Mohammed**

Gold star schema and the use-case reports: `sql/10_gold.sql` and `sql/11_reports.sql`, and the action to connect the reports to a dashboard. Evidence: Meeting 1 minutes (report outline), Meetings 2 to 4 Assigned Responsibilities and Action Items, and Meeting 5 (report with Omkar).

**Aditi Mangroliya**

Masked test data and the leak check: `sql/09_masking.sql` and `reports.masking_check`. Evidence: Meeting 1 minutes (dashboard tool shortlist), Meetings 2 to 4 Assigned Responsibilities, and Meeting 5 (dashboard with Margi).

**Margi Govindbhai Vadodaria**

Catalogue, lineage and discovery scan: `sql/03_audit.sql` and `sql/07_catalogue_scan.sql`. Evidence: Meeting 1 minutes (dashboard tool shortlist), Meetings 2 to 4 Assigned Responsibilities, and Meeting 5 (dashboard with Aditi).

**Mihir Chauhan**

Person hub, identity matching and accuracy: `sql/06_identity.sql` and `reports.identity_accuracy`, and the action to review the MEDIUM identity matches. Evidence: Meeting 1 minutes (report outline), Meetings 3 and 4 Action Items (MEDIUM match review), and Meeting 5 (test video).

**Omkar Ravindran**

Retention and the end-to-end test: `sql/08_retention.sql`, `reports.checks` and `python/run_pipeline.py`, with the action to rebuild from empty using `--reset` until all checks pass. Evidence: Meeting 1 minutes (architecture flowchart), Meetings 2

to 4 Assigned Responsibilities and Action Items, and Meeting 5 (end-to-end checks and report with Abdul).

### 6.3. Teamwork and Communication

The group held five 45-minute meetings between 5 September and 10 October 2026, and all six members attended every one. Work was tracked on a shared Notion board that Arya set up after Meeting 1, and all code was kept in one GitHub repository (Section 3).

Each meeting followed the same pattern: we reviewed the action items from the previous meeting, agreed the decisions that were needed and gave every new action item an owner. After the tutor's feedback at Meeting 4, Meeting 5 went around the table so that each member said what they had done, where they were stuck and what they would do next.

Several tasks were done in pairs: the architecture flowchart (Arya and Omkar), the report outline (Mihir and Abdul), the dashboard (Aditi and Margi), the test video script (Mihir and Omkar) and the final report (Omkar and Abdul). The meeting minutes in Section 6.4 are the record of this communication and of the tasks given to each member.

### 6.4. Meeting Minutes

The minutes of all five group meetings are attached at the end of this report, after the individual contribution form. Meetings 1, 4 and 5 are signed by the tutor.

- **Meeting 1, 5 September 2026:** structure of Assignment 2, architecture flowchart and the three work streams (signed by the tutor, 8 September 2026).
- **Meeting 2, 23 September 2026:** solution for each of the five problems and the prototype work split across all six members.
- **Meeting 3, 30 September 2026:** run order, retention policies, matching rules, core reports and script owners.
- **Meeting 4, 3 October 2026:** confirmed responsibilities and action items (signed by the tutor, 6 October 2026).
- **Meeting 5, 10 October 2026:** round-table progress check and final tasks for the report and test video (signed by the tutor, 13 October 2026).

## 6.5. Source Code Listing

This appendix lists every script of the working prototype in full, in the order run_pipeline.py runs them, followed by the dashboard, demo and setup files. The first comment in each SQL file names the design component it implements (Table 5 and Figure 1). The scripts were run from an empty PostgreSQL 16 database with run_pipeline.py --reset, then run a second time, and all ten checks in reports.checks passed both times.

**`sql/01_schemas.sql`** – Schemas for every layer

```
-- 01 Schemas: one per source system, one per platform layer.
-- Design component: Layer structure (Figure 1): one schema per source system and per
platform layer.

CREATE SCHEMA IF NOT EXISTS src_hrms;     -- HR system
CREATE SCHEMA IF NOT EXISTS src_sis;      -- student system
CREATE SCHEMA IF NOT EXISTS src_crm;      -- alumni and donor CRM
CREATE SCHEMA IF NOT EXISTS dev_sandbox;  -- where the old shadow copies were found

CREATE SCHEMA IF NOT EXISTS bronze;       -- raw copy of each source, reloaded every run
CREATE SCHEMA IF NOT EXISTS silver;       -- Data Vault: hubs, satellites, person link
CREATE SCHEMA IF NOT EXISTS gold;         -- star schema for reporting, no personal data
CREATE SCHEMA IF NOT EXISTS test_masked;  -- masked data for developers
CREATE SCHEMA IF NOT EXISTS audit;        -- run log, catalogue, lineage, retention
CREATE SCHEMA IF NOT EXISTS qa;           -- identity answer key, only used to score
matching
CREATE SCHEMA IF NOT EXISTS reports;      -- report views

CREATE EXTENSION IF NOT EXISTS pgcrypto;  -- hmac() for masking
```

**`sql/02_source_tables.sql`** – Source systems and shadow copies

```
-- 02 Source tables. These stand in for the three operational systems and the
-- Design component: Figure 1, box 1 - Source systems (HRMS, SIS, ADV_CRM) and the shadow
copies.
-- shadow copies. load_source_data.py fills them from the Excel workbook.

-- HRMS: one row per staff member, keyed by the HR employee ID (EMPnnnnnn).
CREATE TABLE IF NOT EXISTS src_hrms.employee (
    hr_employee_id         TEXT PRIMARY KEY,
    first_name             TEXT,
    last_name              TEXT,
    date_of_birth          DATE,
    email                  TEXT,
    phone                  TEXT,
    home_address           TEXT,
    suburb                 TEXT,
    postcode               TEXT,
    tax_file_number        TEXT,
    bank_bsb               TEXT,
    bank_account           TEXT,
    department             TEXT,
    position_title         TEXT,
    salary_band            TEXT,
    employment_status      TEXT,    -- Current / Former
    employment_start_date  DATE,
    employment_end_date    DATE
);

-- SIS: one row per student, keyed by the student ID (STUnnnnnnn).
CREATE TABLE IF NOT EXISTS src_sis.student (
    sis_student_id         TEXT PRIMARY KEY,
    first_name             TEXT,
    last_name              TEXT,
    date_of_birth          DATE,
```

```
    email                 TEXT,
    phone                 TEXT,
    home_address          TEXT,
    suburb                TEXT,
    postcode              TEXT,
    program_code          TEXT,
    program_name          TEXT,
    study_level           TEXT,
    enrolment_status      TEXT,    -- Enrolled / Deferred / Completed / Withdrawn
    enrolment_year        INT,
    completion_year       INT,
    international_flag    TEXT,
    visa_status           TEXT,
    gpa                   NUMERIC(4, 2),
    outstanding_fees_aud  NUMERIC(12, 2),
    last_activity_date    DATE
);

-- ADV_CRM: one row per alumni or donor contact, keyed by the CRM contact ID (CRMnnnnnnn).
CREATE TABLE IF NOT EXISTS src_crm.constituent (
    crm_contact_id        TEXT PRIMARY KEY,
    first_name            TEXT,
    last_name             TEXT,
    date_of_birth         DATE,
    email                 TEXT,
    phone                 TEXT,
    home_address          TEXT,
    suburb                TEXT,
    postcode              TEXT,
    contact_type          TEXT,    -- Alumni / Donor / Alumni & Donor
    alumni_year           INT,
    employer              TEXT,
    donor_flag            TEXT,
    lifetime_donation_aud NUMERIC(12, 2),
    engagement_score      INT,
    last_contact_date     DATE
);

-- The copies developers made between 2012 and 2026. Nobody registered this table.
CREATE TABLE IF NOT EXISTS dev_sandbox.shadow_extracts (
    shadow_record_id        TEXT PRIMARY KEY,
    environment             TEXT,    -- where the copy was found
    source_system           TEXT,
    source_record_id        TEXT,
    extracted_date          DATE,
    purpose                 TEXT,
    approved_retention_days INT,
    retention_expiry_date   DATE,
    pii_classification      TEXT,
    masked                  TEXT,
    catalogued              TEXT,
    lineage_tracked         TEXT,
    access_group            TEXT,
    last_accessed_date      DATE,
    first_name              TEXT,
    last_name               TEXT,
    email                   TEXT,
    date_of_birth           DATE,
    phone                   TEXT,
    home_address            TEXT,
    sensitive_value         TEXT
);

-- Which real person each source record belongs to. Only used to score identity matching.
CREATE TABLE IF NOT EXISTS qa.identity_answer_key (
    source_system      TEXT,
    source_record_id   TEXT,
    true_person_id     TEXT,
    false_friend_of    TEXT,
    PRIMARY KEY (source_system, source_record_id)
);
```

**sql/03_audit.sql** – Governance plane

```
-- 03 Audit tables: run log, catalogue, lineage and retention policies.
-- Design component: Figure 1 - Governance plane (run log, catalogue, lineage, retention
policies, purge log).

-- One row per pipeline step. run_pipeline.py writes it.
CREATE TABLE IF NOT EXISTS audit.pipeline_run (
    run_id        SERIAL PRIMARY KEY,
    batch_id      UUID NOT NULL,            -- all steps of one pipeline run share this
    step_name     TEXT NOT NULL,
    rows_read     INT,
    rows_loaded   INT,
    status        TEXT NOT NULL CHECK (status IN ('RUNNING', 'SUCCESS', 'FAILED')),
    error_message TEXT,
    started_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    finished_at   TIMESTAMPTZ
);

-- Catalogue: every dataset we know about, who owns it and whether it holds real personal
data.
-- The discovery scan (07) adds any table that is not listed here as FOUND_BY_SCAN.
CREATE TABLE IF NOT EXISTS audit.dataset (
    dataset       TEXT PRIMARY KEY,          -- schema.table
    layer         TEXT NOT NULL,
    owner         TEXT NOT NULL,
    real_pii      BOOLEAN NOT NULL,
    status        TEXT NOT NULL CHECK (status IN ('REGISTERED', 'FOUND_BY_SCAN',
'QUARANTINED', 'PURGED')),
    added_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

INSERT INTO audit.dataset (dataset, layer, owner, real_pii, status) VALUES
('src_hrms.employee',            'source',      'People & Culture',       TRUE,
'REGISTERED'),
('src_sis.student',              'source',      'Student Administration', TRUE,
'REGISTERED'),
('src_crm.constituent',          'source',      'Advancement Office',     TRUE,
'REGISTERED'),
('bronze.hrms_employee',          'bronze',      'Data Platform Team',     TRUE,
'REGISTERED'),
('bronze.sis_student',           'bronze',      'Data Platform Team',     TRUE,
'REGISTERED'),
('bronze.crm_constituent',       'bronze',      'Data Platform Team',     TRUE,
'REGISTERED'),
('silver.hub_employee',          'silver',      'Data Platform Team',     FALSE,
'REGISTERED'),
('silver.hub_student',           'silver',      'Data Platform Team',     FALSE,
'REGISTERED'),
('silver.hub_constituent',       'silver',      'Data Platform Team',     FALSE,
'REGISTERED'),
('silver.sat_employee',          'silver',      'People & Culture',       TRUE,
'REGISTERED'),
('silver.sat_student',           'silver',      'Student Administration', TRUE,
'REGISTERED'),
('silver.sat_constituent',       'silver',      'Advancement Office',     TRUE,
'REGISTERED'),
('silver.hub_person',            'silver',      'Data Platform Team',     FALSE,
'REGISTERED'),
('silver.link_person',           'silver',      'Data Platform Team',     FALSE,
'REGISTERED'),
('test_masked.employee',         'test_masked', 'Data Platform Team',     FALSE,
'REGISTERED'),
('test_masked.student',          'test_masked', 'Data Platform Team',     FALSE,
'REGISTERED'),
('test_masked.constituent',      'test_masked', 'Data Platform Team',     FALSE,
'REGISTERED'),
('gold.dim_person',              'gold',        'Data Platform Team',     FALSE,
'REGISTERED'),
('gold.dim_program',             'gold',        'Data Platform Team',     FALSE,
'REGISTERED'),
('gold.fact_enrolment',          'gold',        'Data Platform Team',     FALSE,
'REGISTERED'),
('gold.fact_giving',             'gold',        'Data Platform Team',     FALSE,
'REGISTERED'),
```

```
('qa.identity_answer_key',       'qa',          'Data Platform Team',     FALSE,
'REGISTERED')
ON CONFLICT (dataset) DO NOTHING;

-- Columns the discovery scan classified as personal data.
CREATE TABLE IF NOT EXISTS audit.pii_column (
    dataset       TEXT NOT NULL,
    column_name   TEXT NOT NULL,
    category      TEXT NOT NULL CHECK (category IN ('DIRECT', 'QUASI', 'SENSITIVE',
'FINANCIAL')),
    PRIMARY KEY (dataset, column_name)
);

-- Lineage: one row every time a step moves data, written by that step.
CREATE TABLE IF NOT EXISTS audit.lineage (
    lineage_id    SERIAL PRIMARY KEY,
    run_id        INT REFERENCES audit.pipeline_run (run_id),
    source        TEXT NOT NULL,
    target        TEXT NOT NULL,
    operation     TEXT NOT NULL,             -- INGEST, VAULT_LOAD, MATCH, MASK, PURGE,
BUILD
    row_count     INT,
    masked        BOOLEAN NOT NULL DEFAULT FALSE,
    recorded_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Retention policies. 2555 days = 7 years after the person stopped being active.
CREATE TABLE IF NOT EXISTS audit.retention_policy (
    policy_id     TEXT PRIMARY KEY,
    description   TEXT NOT NULL,
    applies_to    TEXT NOT NULL,
    keep_days     INT,                       -- NULL: the record carries its own expiry
date
    action        TEXT NOT NULL CHECK (action IN ('ANONYMISE', 'DELETE'))
);

INSERT INTO audit.retention_policy VALUES
('RP-01', 'Students who completed or withdrew',    'silver.sat_student',          2555,
'ANONYMISE'),
('RP-02', 'Former staff',                          'silver.sat_employee',         2555,
'ANONYMISE'),
('RP-03', 'CRM contacts who are not donors',       'silver.sat_constituent',      2555,
'ANONYMISE'),
('RP-04', 'Test copies past their approved date',  'dev_sandbox.shadow_extracts', NULL,
'DELETE')
ON CONFLICT (policy_id) DO NOTHING;

-- What each retention run did.
CREATE TABLE IF NOT EXISTS audit.purge_log (
    purge_id      SERIAL PRIMARY KEY,
    run_id        INT REFERENCES audit.pipeline_run (run_id),
    policy_id     TEXT REFERENCES audit.retention_policy (policy_id),
    target        TEXT NOT NULL,
    action        TEXT NOT NULL,
    rows_before   INT,                       -- rows still holding personal data before
the run
    rows_affected INT NOT NULL,
    rows_after    INT,
    executed_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

**`sql/04_bronze.sql`** – Bronze layer

```
-- 04 Bronze: a raw copy of each source table, plus when and by which run it was loaded.
-- Design component: Figure 1, box 2 - Bronze layer (pipeline step INGEST).


-- Each bronze table has the same columns as its source (LIKE), plus three load columns.
CREATE TABLE IF NOT EXISTS bronze.hrms_employee (
    LIKE src_hrms.employee,
    run_id         INT,
    loaded_at      TIMESTAMPTZ,
    record_source  TEXT
);
```

```
CREATE TABLE IF NOT EXISTS bronze.sis_student (
    LIKE src_sis.student,
    run_id         INT,
    loaded_at      TIMESTAMPTZ,
    record_source  TEXT
);

CREATE TABLE IF NOT EXISTS bronze.crm_constituent (
    LIKE src_crm.constituent,
    run_id         INT,
    loaded_at      TIMESTAMPTZ,
    record_source  TEXT
);

-- Reload one bronze table from its source and check that no rows were lost.
CREATE OR REPLACE PROCEDURE bronze.load_source(p_run_id INT, p_source TEXT, p_target TEXT,
p_label TEXT)
LANGUAGE plpgsql AS $$
DECLARE
    v_columns TEXT;
    v_read    INT;
    v_loaded  INT;
BEGIN
    -- Copy column by column name. If the source has a column bronze does not know, this
fails.
    SELECT string_agg(quote_ident(column_name), ', ' ORDER BY ordinal_position) INTO
v_columns
      FROM information_schema.columns
     WHERE table_schema || '.' || table_name = p_source;

    -- Bronze is a full reload: empty the table, then copy every source row with the run
ID.
    EXECUTE format('TRUNCATE bronze.%I', p_target);
    EXECUTE format('INSERT INTO bronze.%I (%s, run_id, loaded_at, record_source)
                    SELECT %s, %s, now(), %L FROM %s',
                   p_target, v_columns, v_columns, p_run_id, p_label, p_source);
    GET DIAGNOSTICS v_loaded = ROW_COUNT;

    -- Reconcile: the step fails (and is rolled back) if any row was lost.
    EXECUTE format('SELECT count(*) FROM %s', p_source) INTO v_read;
    IF v_read <> v_loaded THEN
        RAISE EXCEPTION 'Row count mismatch: read %, loaded %', v_read, v_loaded;
    END IF;

    -- Record the counts in the run log and the movement in the lineage table.
    UPDATE audit.pipeline_run SET rows_read = v_read, rows_loaded = v_loaded WHERE run_id
= p_run_id;
    INSERT INTO audit.lineage (run_id, source, target, operation, row_count)
    VALUES (p_run_id, p_source, 'bronze.' || p_target, 'INGEST', v_loaded);
END;
$$;
```

### **sql/05_silver_vault.sql** – Silver Data Vault

```
-- 05 Silver: Data Vault 2.0.
-- Design component: Figure 1, box 3 - Silver Data Vault 2.0: hubs and satellites (step
VAULT_LOAD); Figure 3.
-- Hub      = one row per business key (the ID used by the source system).
-- Satellite = the descriptive data for that key. Rows are only added, never
--             overwritten: a new row is inserted when hash_diff shows the data changed.
-- Every satellite row gets a retention_expiry_date (its "use-by date").

-- ------------------------------------------------------------ cleaning helpers

-- '  PRIYA ' -> 'Priya'. Mixed-case names such as O'Brien are left as they are.
CREATE OR REPLACE FUNCTION silver.clean_name(p TEXT) RETURNS TEXT
LANGUAGE sql IMMUTABLE AS $$
    SELECT CASE WHEN btrim(p) IN (upper(btrim(p)), lower(btrim(p))) THEN initcap(btrim(p))
                ELSE btrim(p) END;
$$;
```

```
CREATE OR REPLACE FUNCTION silver.clean_email(p TEXT) RETURNS TEXT
LANGUAGE sql IMMUTABLE AS $$
    SELECT nullif(lower(btrim(p)), '');
$$;

-- '+61 412 345 678' and '0412 345 678' -> '0412345678'
CREATE OR REPLACE FUNCTION silver.clean_phone(p TEXT) RETURNS TEXT
LANGUAGE sql IMMUTABLE AS $$
    SELECT CASE WHEN d LIKE '61%' AND length(d) = 11 THEN '0' || substr(d, 3) ELSE
nullif(d, '') END
      FROM (SELECT regexp_replace(p, '[^0-9]', '', 'g') AS d) x;
$$;

-- ------------------------------------------------------------------ hubs

CREATE TABLE IF NOT EXISTS silver.hub_employee (
    employee_hk     CHAR(32) PRIMARY KEY,      -- md5 of the business key
    hr_employee_id  TEXT NOT NULL UNIQUE,
    load_dts        TIMESTAMPTZ NOT NULL,
    record_source   TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS silver.hub_student (
    student_hk      CHAR(32) PRIMARY KEY,
    sis_student_id  TEXT NOT NULL UNIQUE,
    load_dts        TIMESTAMPTZ NOT NULL,
    record_source   TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS silver.hub_constituent (
    constituent_hk  CHAR(32) PRIMARY KEY,
    crm_contact_id  TEXT NOT NULL UNIQUE,
    load_dts        TIMESTAMPTZ NOT NULL,
    record_source   TEXT NOT NULL
);

-- ------------------------------------------------------------- satellites

CREATE TABLE IF NOT EXISTS silver.sat_employee (
    employee_hk            CHAR(32) NOT NULL REFERENCES silver.hub_employee,
    load_dts               TIMESTAMPTZ NOT NULL,
    hash_diff              CHAR(32) NOT NULL,
    first_name             TEXT,
    last_name              TEXT,
    date_of_birth          DATE,
    email                  TEXT,
    phone                  TEXT,
    home_address           TEXT,
    suburb                 TEXT,
    postcode               TEXT,
    tax_file_number        TEXT,
    bank_bsb               TEXT,
    bank_account           TEXT,
    department             TEXT,
    position_title         TEXT,
    salary_band            TEXT,
    employment_status      TEXT,
    employment_start_date  DATE,
    employment_end_date    DATE,
    retention_expiry_date  DATE,               -- RP-02: end date + 7 years, former staff
only
    anonymised_at          TIMESTAMPTZ,
    record_source          TEXT NOT NULL,
    PRIMARY KEY (employee_hk, load_dts)
);

CREATE TABLE IF NOT EXISTS silver.sat_student (
    student_hk             CHAR(32) NOT NULL REFERENCES silver.hub_student,
    load_dts               TIMESTAMPTZ NOT NULL,
    hash_diff              CHAR(32) NOT NULL,
    first_name             TEXT,
    last_name              TEXT,
    date_of_birth          DATE,
```

```
    email                 TEXT,
    phone                 TEXT,
    home_address          TEXT,
    suburb                TEXT,
    postcode              TEXT,
    program_code          TEXT,
    program_name          TEXT,
    study_level           TEXT,
    enrolment_status      TEXT,
    enrolment_year        INT,
    completion_year       INT,
    international_flag    TEXT,
    visa_status           TEXT,
    gpa                   NUMERIC(4, 2),
    outstanding_fees_aud  NUMERIC(12, 2),
    last_activity_date    DATE,
    retention_expiry_date DATE,              -- RP-01: last activity + 7 years,
completed/withdrawn only
    anonymised_at         TIMESTAMPTZ,
    record_source         TEXT NOT NULL,
    PRIMARY KEY (student_hk, load_dts)
);

CREATE TABLE IF NOT EXISTS silver.sat_constituent (
    constituent_hk        CHAR(32) NOT NULL REFERENCES silver.hub_constituent,
    load_dts              TIMESTAMPTZ NOT NULL,
    hash_diff             CHAR(32) NOT NULL,
    first_name            TEXT,
    last_name             TEXT,
    date_of_birth         DATE,
    email                 TEXT,
    phone                 TEXT,
    home_address          TEXT,
    suburb                TEXT,
    postcode              TEXT,
    contact_type          TEXT,
    alumni_year           INT,
    employer              TEXT,
    donor_flag            TEXT,
    lifetime_donation_aud NUMERIC(12, 2),
    engagement_score      INT,
    last_contact_date     DATE,
    retention_expiry_date DATE,              -- RP-03: last contact + 7 years, non-
donors only
    anonymised_at         TIMESTAMPTZ,
    record_source         TEXT NOT NULL,
    PRIMARY KEY (constituent_hk, load_dts)
);

CREATE INDEX IF NOT EXISTS sat_employee_email_idx    ON silver.sat_employee (email);
CREATE INDEX IF NOT EXISTS sat_student_email_idx     ON silver.sat_student (email);
CREATE INDEX IF NOT EXISTS sat_constituent_email_idx ON silver.sat_constituent (email);

-- ------------------------------------------------ latest satellite row per key

CREATE OR REPLACE VIEW silver.employee_current AS
SELECT DISTINCT ON (h.employee_hk) h.hr_employee_id, s.*
  FROM silver.hub_employee h JOIN silver.sat_employee s USING (employee_hk)
 ORDER BY h.employee_hk, s.load_dts DESC;

CREATE OR REPLACE VIEW silver.student_current AS
SELECT DISTINCT ON (h.student_hk) h.sis_student_id, s.*
  FROM silver.hub_student h JOIN silver.sat_student s USING (student_hk)
 ORDER BY h.student_hk, s.load_dts DESC;

CREATE OR REPLACE VIEW silver.constituent_current AS
SELECT DISTINCT ON (h.constituent_hk) h.crm_contact_id, s.*
  FROM silver.hub_constituent h JOIN silver.sat_constituent s USING (constituent_hk)
 ORDER BY h.constituent_hk, s.load_dts DESC;

-- ------------------------------------------------------------------ load step

CREATE OR REPLACE PROCEDURE silver.load_vault(p_run_id INT)
```

```
LANGUAGE plpgsql AS $$
DECLARE
    v_rows  INT;
    v_total INT := 0;
BEGIN
    -- Employees -------------------------------------------------------------
    INSERT INTO silver.hub_employee
    SELECT md5(hr_employee_id), hr_employee_id, now(), 'HRMS'
      FROM bronze.hrms_employee
    ON CONFLICT DO NOTHING;

    INSERT INTO silver.sat_employee
    SELECT n.*
      FROM (SELECT md5(b.hr_employee_id) AS employee_hk, now() AS load_dts,
                   md5(ROW(b.first_name, b.last_name, b.date_of_birth, b.email, b.phone,
b.home_address,
                           b.suburb, b.postcode, b.tax_file_number, b.bank_bsb,
b.bank_account, b.department,
                           b.position_title, b.salary_band, b.employment_status,
b.employment_start_date,
                           b.employment_end_date)::TEXT) AS hash_diff,
                   silver.clean_name(b.first_name), silver.clean_name(b.last_name),
b.date_of_birth,
                   silver.clean_email(b.email), silver.clean_phone(b.phone),
btrim(b.home_address),
                   b.suburb, b.postcode, b.tax_file_number, b.bank_bsb, b.bank_account,
b.department,
                   b.position_title, b.salary_band, b.employment_status,
b.employment_start_date,
                   b.employment_end_date,
                   CASE WHEN b.employment_status = 'Former' THEN b.employment_end_date +
2555 END,
                   NULL::TIMESTAMPTZ, 'HRMS'
              FROM bronze.hrms_employee b) n
      LEFT JOIN silver.employee_current c ON c.employee_hk = n.employee_hk
     WHERE c.hash_diff IS DISTINCT FROM n.hash_diff;    -- new key, or the data changed
    GET DIAGNOSTICS v_rows = ROW_COUNT;
    v_total := v_total + v_rows;
    INSERT INTO audit.lineage (run_id, source, target, operation, row_count) VALUES
    (p_run_id, 'bronze.hrms_employee', 'silver.hub_employee', 'VAULT_LOAD', (SELECT
count(*) FROM silver.hub_employee)),
    (p_run_id, 'bronze.hrms_employee', 'silver.sat_employee', 'VAULT_LOAD', v_rows);

    -- Students --------------------------------------------------------------
    INSERT INTO silver.hub_student
    SELECT md5(sis_student_id), sis_student_id, now(), 'SIS'
      FROM bronze.sis_student
    ON CONFLICT DO NOTHING;

    INSERT INTO silver.sat_student
    SELECT n.*
      FROM (SELECT md5(b.sis_student_id) AS student_hk, now() AS load_dts,
                   md5(ROW(b.first_name, b.last_name, b.date_of_birth, b.email, b.phone,
b.home_address,
                           b.suburb, b.postcode, b.program_code, b.enrolment_status,
b.enrolment_year,
                           b.completion_year, b.international_flag, b.visa_status, b.gpa,
                           b.outstanding_fees_aud, b.last_activity_date)::TEXT) AS
hash_diff,
                   silver.clean_name(b.first_name), silver.clean_name(b.last_name),
b.date_of_birth,
                   silver.clean_email(b.email), silver.clean_phone(b.phone),
btrim(b.home_address),
                   b.suburb, b.postcode, b.program_code, b.program_name, b.study_level,
b.enrolment_status,
                   b.enrolment_year, b.completion_year, b.international_flag,
b.visa_status, b.gpa,
                   b.outstanding_fees_aud, b.last_activity_date,
                   CASE WHEN b.enrolment_status IN ('Completed', 'Withdrawn') THEN
b.last_activity_date + 2555 END,
                   NULL::TIMESTAMPTZ, 'SIS'
              FROM bronze.sis_student b) n
      LEFT JOIN silver.student_current c ON c.student_hk = n.student_hk
     WHERE c.hash_diff IS DISTINCT FROM n.hash_diff;
    GET DIAGNOSTICS v_rows = ROW_COUNT;
    v_total := v_total + v_rows;
```

```
    INSERT INTO audit.lineage (run_id, source, target, operation, row_count) VALUES
    (p_run_id, 'bronze.sis_student', 'silver.hub_student', 'VAULT_LOAD', (SELECT count(*)
FROM silver.hub_student)),
    (p_run_id, 'bronze.sis_student', 'silver.sat_student', 'VAULT_LOAD', v_rows);

    -- CRM contacts ----------------------------------------------------------
    INSERT INTO silver.hub_constituent
    SELECT md5(crm_contact_id), crm_contact_id, now(), 'ADV_CRM'
      FROM bronze.crm_constituent
    ON CONFLICT DO NOTHING;

    INSERT INTO silver.sat_constituent
    SELECT n.*
      FROM (SELECT md5(b.crm_contact_id) AS constituent_hk, now() AS load_dts,
                   md5(ROW(b.first_name, b.last_name, b.date_of_birth, b.email, b.phone,
b.home_address,
                           b.suburb, b.postcode, b.contact_type, b.alumni_year,
b.employer, b.donor_flag,
                           b.lifetime_donation_aud, b.engagement_score,
b.last_contact_date)::TEXT) AS hash_diff,
                   silver.clean_name(b.first_name), silver.clean_name(b.last_name),
b.date_of_birth,
                   silver.clean_email(b.email), silver.clean_phone(b.phone),
btrim(b.home_address),
                   b.suburb, b.postcode, b.contact_type, b.alumni_year, b.employer,
b.donor_flag,
                   b.lifetime_donation_aud, b.engagement_score, b.last_contact_date,
                   CASE WHEN b.donor_flag = 'N' THEN b.last_contact_date + 2555 END,
                   NULL::TIMESTAMPTZ, 'ADV_CRM'
              FROM bronze.crm_constituent b) n
      LEFT JOIN silver.constituent_current c ON c.constituent_hk = n.constituent_hk
     WHERE c.hash_diff IS DISTINCT FROM n.hash_diff;
    GET DIAGNOSTICS v_rows = ROW_COUNT;
    v_total := v_total + v_rows;
    INSERT INTO audit.lineage (run_id, source, target, operation, row_count) VALUES
    (p_run_id, 'bronze.crm_constituent', 'silver.hub_constituent', 'VAULT_LOAD', (SELECT
count(*) FROM silver.hub_constituent)),
    (p_run_id, 'bronze.crm_constituent', 'silver.sat_constituent', 'VAULT_LOAD', v_rows);

    UPDATE audit.pipeline_run SET rows_loaded = v_total WHERE run_id = p_run_id;
END;
$$;
```

**`sql/06_identity.sql`** – Identity resolution

```
-- 06 Identity resolution: work out which HRMS, SIS and CRM records are the same person.
-- Design component: Figure 1, box 3 - Identity resolution: person hub and link (step
MATCH); Figure 3.
-- No record is deleted. hub_person holds one row per person and link_person says
-- which source records belong to them and which rule matched them.
--
-- Rules (on cleaned values from the latest bronze load):
--   R1  same email address                                          HIGH
--   R2  same surname + date of birth + phone number                 HIGH
--   R3  same first name + surname + date of birth,
--       only between records from different systems                 MEDIUM
-- Matches chain together: if A matches B and B matches C, all three are one person.

CREATE TABLE IF NOT EXISTS silver.hub_person (
    person_id   TEXT PRIMARY KEY,
    load_dts    TIMESTAMPTZ NOT NULL
);

CREATE TABLE IF NOT EXISTS silver.link_person (
    person_id         TEXT NOT NULL REFERENCES silver.hub_person,
    system_name       TEXT NOT NULL,       -- HRMS, SIS or ADV_CRM
    source_record_id  TEXT NOT NULL,
    match_rule        TEXT NOT NULL,       -- e.g. 'R1+R2', or 'NONE' for a person found
in one system only
    confidence        TEXT NOT NULL CHECK (confidence IN ('HIGH', 'MEDIUM', 'SINGLE')),
    load_dts          TIMESTAMPTZ NOT NULL,
    PRIMARY KEY (system_name, source_record_id)
);
```

```
CREATE INDEX IF NOT EXISTS link_person_person_idx ON silver.link_person (person_id);

CREATE OR REPLACE PROCEDURE silver.resolve_identities(p_run_id INT)
LANGUAGE plpgsql AS $$
DECLARE
    v_changed INT;
BEGIN
    -- 1. One row per source record, with the values used for matching.
    DROP TABLE IF EXISTS rec, pair;
    CREATE TEMP TABLE rec AS
    SELECT 'HRMS:' || hr_employee_id AS k, 'HRMS' AS sys, hr_employee_id AS id,
silver.clean_email(email) AS email,
           silver.clean_phone(phone) AS phone, lower(btrim(first_name)) AS first_name,
           lower(btrim(last_name)) AS last_name, date_of_birth AS dob
      FROM bronze.hrms_employee
    UNION ALL
    SELECT 'SIS:' || sis_student_id, 'SIS', sis_student_id, silver.clean_email(email),
silver.clean_phone(phone),
           lower(btrim(first_name)), lower(btrim(last_name)), date_of_birth
      FROM bronze.sis_student
    UNION ALL
    SELECT 'ADV_CRM:' || crm_contact_id, 'ADV_CRM', crm_contact_id,
silver.clean_email(email), silver.clean_phone(phone),
           lower(btrim(first_name)), lower(btrim(last_name)), date_of_birth
      FROM bronze.crm_constituent;

    -- Every record starts in its own group, named after itself.
    ALTER TABLE rec ADD COLUMN grp TEXT;
    UPDATE rec SET grp = k;
    CREATE INDEX ON rec (k);
    ANALYZE rec;

    -- 2. Pairs of records that match a rule.
    CREATE TEMP TABLE pair AS
    SELECT a.k AS a, b.k AS b, 'R1' AS rule
      FROM rec a JOIN rec b ON a.email = b.email AND a.k < b.k
    UNION ALL
    SELECT a.k, b.k, 'R2'
      FROM rec a JOIN rec b ON a.last_name = b.last_name AND a.dob = b.dob AND a.phone =
b.phone
                           AND a.k < b.k
    UNION ALL
    SELECT a.k, b.k, 'R3'
      FROM rec a JOIN rec b ON a.first_name = b.first_name AND a.last_name = b.last_name
                           AND a.dob = b.dob AND a.sys < b.sys;

    -- 3. Merge groups: keep giving each record the smallest group name among the
    --    records it matches, until nothing changes.
    LOOP
        UPDATE rec r SET grp = m.grp
          FROM (SELECT p.a AS k, min(other.grp) AS grp
                  FROM (SELECT a, b FROM pair UNION ALL SELECT b, a FROM pair) p
                  JOIN rec other ON other.k = p.b
                 GROUP BY p.a) m
         WHERE r.k = m.k AND m.grp < r.grp;
        GET DIAGNOSTICS v_changed = ROW_COUNT;
        EXIT WHEN v_changed = 0;
    END LOOP;

    -- 4. Save. person_id comes from the group name, so it stays the same between runs.
    TRUNCATE silver.link_person;
    DELETE FROM silver.hub_person;

    INSERT INTO silver.hub_person
    SELECT DISTINCT 'P-' || upper(left(md5(grp), 10)), now() FROM rec;

    INSERT INTO silver.link_person
    SELECT 'P-' || upper(left(md5(r.grp), 10)), r.sys, r.id,
           coalesce(m.rules, 'NONE'),
           CASE WHEN m.rules IS NULL THEN 'SINGLE'
                WHEN m.rules LIKE '%R1%' OR m.rules LIKE '%R2%' THEN 'HIGH'
                ELSE 'MEDIUM' END,
           now()
      FROM rec r
```

```
      LEFT JOIN (SELECT k, string_agg(DISTINCT rule, '+' ORDER BY rule) AS rules
                   FROM (SELECT a AS k, rule FROM pair UNION ALL SELECT b, rule FROM pair)
x
                  GROUP BY k) m ON m.k = r.k;

    UPDATE audit.pipeline_run SET rows_read = (SELECT count(*) FROM rec),
                                  rows_loaded = (SELECT count(*) FROM silver.hub_person)
     WHERE run_id = p_run_id;
    INSERT INTO audit.lineage (run_id, source, target, operation, row_count) VALUES
    (p_run_id, 'bronze (all three systems)', 'silver.hub_person',  'MATCH', (SELECT
count(*) FROM silver.hub_person)),
    (p_run_id, 'bronze (all three systems)', 'silver.link_person', 'MATCH', (SELECT
count(*) FROM silver.link_person));
END;
$$;
```

**`sql/07_catalogue_scan.sql`** – Catalogue and discovery scan

```
-- 07 Discovery scan: look at every table in the database, record which columns
-- Design component: Figure 1 - Governance plane: catalogue and discovery scan.
-- hold personal data, and add any table nobody registered to the catalogue.
-- This is how the shadow copies in dev_sandbox show up.

CREATE OR REPLACE FUNCTION audit.pii_category(p_column TEXT) RETURNS TEXT
LANGUAGE sql IMMUTABLE AS $$
    SELECT CASE
        WHEN p_column IN ('first_name', 'last_name', 'email', 'phone', 'home_address')
THEN 'DIRECT'
        WHEN p_column IN ('date_of_birth', 'suburb', 'postcode')
THEN 'QUASI'
        WHEN p_column IN ('visa_status', 'gpa', 'international_flag', 'sensitive_value')
THEN 'SENSITIVE'
        WHEN p_column IN ('tax_file_number', 'bank_bsb', 'bank_account', 'salary_band',
                          'lifetime_donation_aud', 'outstanding_fees_aud')            THEN
'FINANCIAL'
    END;
$$;

CREATE OR REPLACE PROCEDURE audit.scan_catalogue(p_run_id INT)
LANGUAGE plpgsql AS $$
DECLARE
    v_found INT;
BEGIN
    -- Personal-data columns in every table
    INSERT INTO audit.pii_column (dataset, column_name, category)
    SELECT c.table_schema || '.' || c.table_name, c.column_name,
audit.pii_category(c.column_name)
      FROM information_schema.columns c
      JOIN information_schema.tables t USING (table_schema, table_name)
     WHERE t.table_type = 'BASE TABLE'
       AND c.table_schema IN ('src_hrms', 'src_sis', 'src_crm', 'dev_sandbox',
                              'bronze', 'silver', 'gold', 'test_masked')
       AND audit.pii_category(c.column_name) IS NOT NULL
    ON CONFLICT DO NOTHING;

    -- Tables that are not in the catalogue
    INSERT INTO audit.dataset (dataset, layer, owner, real_pii, status)
    SELECT t.table_schema || '.' || t.table_name, t.table_schema, 'UNKNOWN',
           EXISTS (SELECT 1 FROM audit.pii_column p
                    WHERE p.dataset = t.table_schema || '.' || t.table_name AND p.category
= 'DIRECT'),
           'FOUND_BY_SCAN'
      FROM information_schema.tables t
     WHERE t.table_type = 'BASE TABLE'
       AND t.table_schema IN ('src_hrms', 'src_sis', 'src_crm', 'dev_sandbox',
                              'bronze', 'silver', 'gold', 'test_masked')
    ON CONFLICT (dataset) DO NOTHING;
    GET DIAGNOSTICS v_found = ROW_COUNT;

    UPDATE audit.pipeline_run SET rows_loaded = v_found WHERE run_id = p_run_id;
END;
$$;
```

**sql/08_retention.sql** – Retention

```
-- 08 Retention: remove personal data that is past its use-by date.
-- Design component: Figure 1 - Governance plane: retention policies RP-01 to RP-04 (step
PURGE).
--   RP-01..RP-03  anonymise the person's details in silver (every version of the row)
--                 and blank the same details in bronze.
--   RP-04         delete shadow copies past their approved date. Whatever is left
--                 is still within its approved date, so the table is quarantined
--                 (nobody can read it) instead of dropped.
-- hash_diff is not changed when a row is anonymised, so the next load sees "no change"
-- and does not copy the personal data back in from the source system.

CREATE OR REPLACE PROCEDURE audit.run_retention(p_run_id INT)
LANGUAGE plpgsql AS $$
DECLARE
    v_before INT;
    v_done   INT;
BEGIN
    -- RP-01 students ------------------------------------------------------
    SELECT count(*) INTO v_before FROM silver.student_current WHERE anonymised_at IS NULL;

    UPDATE silver.sat_student
       SET first_name = NULL, last_name = NULL, date_of_birth = NULL, email = NULL, phone
= NULL,
           home_address = NULL, visa_status = NULL, anonymised_at = now()
     WHERE anonymised_at IS NULL
       AND student_hk IN (SELECT student_hk FROM silver.student_current
                          WHERE retention_expiry_date < current_date AND anonymised_at IS
NULL);
    GET DIAGNOSTICS v_done = ROW_COUNT;

    INSERT INTO audit.purge_log (run_id, policy_id, target, action, rows_before,
rows_affected, rows_after)
    VALUES (p_run_id, 'RP-01', 'silver.sat_student', 'ANONYMISE', v_before, v_done,
v_before - v_done);

    -- RP-02 former staff --------------------------------------------------
    SELECT count(*) INTO v_before FROM silver.employee_current WHERE anonymised_at IS
NULL;

    UPDATE silver.sat_employee
       SET first_name = NULL, last_name = NULL, date_of_birth = NULL, email = NULL, phone
= NULL,
           home_address = NULL, tax_file_number = NULL, bank_bsb = NULL, bank_account =
NULL,
           anonymised_at = now()
     WHERE anonymised_at IS NULL
       AND employee_hk IN (SELECT employee_hk FROM silver.employee_current
                           WHERE retention_expiry_date < current_date AND anonymised_at
IS NULL);
    GET DIAGNOSTICS v_done = ROW_COUNT;

    INSERT INTO audit.purge_log (run_id, policy_id, target, action, rows_before,
rows_affected, rows_after)
    VALUES (p_run_id, 'RP-02', 'silver.sat_employee', 'ANONYMISE', v_before, v_done,
v_before - v_done);

    -- RP-03 CRM contacts who are not donors -------------------------------
    SELECT count(*) INTO v_before FROM silver.constituent_current WHERE anonymised_at IS
NULL;

    UPDATE silver.sat_constituent
       SET first_name = NULL, last_name = NULL, date_of_birth = NULL, email = NULL, phone
= NULL,
           home_address = NULL, employer = NULL, anonymised_at = now()
     WHERE anonymised_at IS NULL
       AND constituent_hk IN (SELECT constituent_hk FROM silver.constituent_current
                              WHERE retention_expiry_date < current_date AND
anonymised_at IS NULL);
    GET DIAGNOSTICS v_done = ROW_COUNT;
```

```
    INSERT INTO audit.purge_log (run_id, policy_id, target, action, rows_before,
rows_affected, rows_after)
    VALUES (p_run_id, 'RP-03', 'silver.sat_constituent', 'ANONYMISE', v_before, v_done,
v_before - v_done);

    -- Bronze is reloaded from the sources on every run, so blank the same people there
too.
    UPDATE bronze.sis_student SET first_name = NULL, last_name = NULL, date_of_birth =
NULL, email = NULL,
                                  phone = NULL, home_address = NULL, visa_status = NULL
     WHERE sis_student_id IN (SELECT sis_student_id FROM silver.student_current WHERE
anonymised_at IS NOT NULL);

    UPDATE bronze.hrms_employee SET first_name = NULL, last_name = NULL, date_of_birth =
NULL, email = NULL,
                                    phone = NULL, home_address = NULL, tax_file_number =
NULL,
                                    bank_bsb = NULL, bank_account = NULL
     WHERE hr_employee_id IN (SELECT hr_employee_id FROM silver.employee_current WHERE
anonymised_at IS NOT NULL);

    UPDATE bronze.crm_constituent SET first_name = NULL, last_name = NULL, date_of_birth =
NULL, email = NULL,
                                     phone = NULL, home_address = NULL, employer = NULL
     WHERE crm_contact_id IN (SELECT crm_contact_id FROM silver.constituent_current WHERE
anonymised_at IS NOT NULL);

    -- RP-04 shadow copies ---------------------------------------------------
    SELECT count(*) INTO v_before FROM dev_sandbox.shadow_extracts;

    DELETE FROM dev_sandbox.shadow_extracts WHERE retention_expiry_date < current_date;
    GET DIAGNOSTICS v_done = ROW_COUNT;

    INSERT INTO audit.purge_log (run_id, policy_id, target, action, rows_before,
rows_affected, rows_after)
    VALUES (p_run_id, 'RP-04', 'dev_sandbox.shadow_extracts', 'DELETE', v_before, v_done,
v_before - v_done);
    INSERT INTO audit.lineage (run_id, source, target, operation, row_count)
    VALUES (p_run_id, 'dev_sandbox.shadow_extracts', '(deleted)', 'PURGE', v_done);

    REVOKE ALL ON dev_sandbox.shadow_extracts FROM PUBLIC;
    UPDATE audit.dataset
       SET status = CASE WHEN v_before - v_done = 0 THEN 'PURGED' ELSE 'QUARANTINED' END
     WHERE dataset = 'dev_sandbox.shadow_extracts';
END;
$$;
```

### **sql/09_masking.sql** – Masked test data

```
-- 09 Masked test data for developers, for all three systems.
-- Design component: Figure 1, box 4 - test_masked: masked test data for developers (step
MASK).
--   - IDs are replaced with a keyed hash (HMAC). The same ID always gets the same
--     pseudonym, so the masked tables still join, but nobody can work backwards
--     without the key. The key is created randomly inside the database, never in git.
--   - Names come from a list of made-up names that do not appear in the real data.
--   - Dates of birth move 15 to 350 days earlier or later.
--   - Emails and phone numbers are replaced. Tax file numbers, bank details and
--     street addresses are left out.
--   - People whose details were anonymised by retention are not included.

CREATE TABLE IF NOT EXISTS audit.masking_key (
    id   INT PRIMARY KEY DEFAULT 1,
    key  BYTEA NOT NULL
);
INSERT INTO audit.masking_key (key) SELECT gen_random_bytes(32)
 WHERE NOT EXISTS (SELECT 1 FROM audit.masking_key);

-- 'STU0000001' -> 'STU-4F0A9C21B7E3'
CREATE OR REPLACE FUNCTION test_masked.pseudo(p_value TEXT, p_prefix TEXT) RETURNS TEXT
LANGUAGE sql STABLE AS $$
```

```
    SELECT p_prefix || '-' || upper(left(encode(hmac(convert_to(p_value, 'UTF8'), (SELECT
key FROM audit.masking_key), 'sha256'), 'hex'), 12));
$$;

-- A number from 1 to p_max that is always the same for the same input.
CREATE OR REPLACE FUNCTION test_masked.pick(p_value TEXT, p_max INT) RETURNS INT
LANGUAGE sql STABLE AS $$
    SELECT 1 + abs(hashtext(test_masked.pseudo(p_value, ''))) % p_max;
$$;

CREATE OR REPLACE FUNCTION test_masked.fake_first_name(p_id TEXT) RETURNS TEXT
LANGUAGE sql STABLE AS $$
    SELECT (ARRAY['Ari', 'Bo', 'Cass', 'Dana', 'Eli', 'Frankie', 'Gale', 'Hollis', 'Indy',
'Jules',
                  'Kit', 'Lane', 'Morgan', 'Noor', 'Oakley', 'Parker', 'Quinn', 'Reese',
'Sage', 'Tatum',
                  'Uma', 'Vale', 'Wren', 'Yael', 'Zion', 'Rowan', 'Sky', 'Jordan',
'Emery', 'Remy'])
           [test_masked.pick(p_id || ':first', 30)];
$$;

CREATE OR REPLACE FUNCTION test_masked.fake_last_name(p_id TEXT) RETURNS TEXT
LANGUAGE sql STABLE AS $$
    SELECT (ARRAY['Ashdown', 'Birchwood', 'Coldwater', 'Dunmore', 'Eastwick', 'Fernhill',
'Glenrock',
                  'Hollowell', 'Ironbark', 'Juniper', 'Kestrel', 'Larkspur', 'Millbrook',
'Northcote',
                  'Oakridge', 'Pinecrest', 'Quarrington', 'Riverside', 'Stonebridge',
'Thornbury',
                  'Underwood', 'Westbrook', 'Yarrow', 'Zephyr', 'Ambleside', 'Bellmount',
'Copperfield',
                  'Driftwood', 'Elmstead', 'Foxglove'])
           [test_masked.pick(p_id || ':last', 30)];
$$;

CREATE OR REPLACE FUNCTION test_masked.shift_date(p_date DATE, p_id TEXT) RETURNS DATE
LANGUAGE sql STABLE AS $$
    SELECT p_date + CASE WHEN test_masked.pick(p_id || ':sign', 2) = 1 THEN -1 ELSE 1 END
                  * (14 + test_masked.pick(p_id || ':days', 336));
$$;

-- 0491 57x xxx is a range the source systems never use, so a masked number is never real.
CREATE OR REPLACE FUNCTION test_masked.fake_phone(p_id TEXT) RETURNS TEXT
LANGUAGE sql STABLE AS $$
    SELECT '049157' || lpad(test_masked.pick(p_id || ':phone', 9999)::TEXT, 4, '0');
$$;

CREATE TABLE IF NOT EXISTS test_masked.employee (
    hr_employee_id         TEXT PRIMARY KEY,
    person_ref             TEXT,       -- masked person_id: links the same person across
masked tables
    first_name             TEXT,
    last_name              TEXT,
    date_of_birth          DATE,
    email                  TEXT,
    phone                  TEXT,
    postcode               TEXT,
    department             TEXT,
    position_title         TEXT,
    salary_band            TEXT,
    employment_status      TEXT,
    employment_start_date  DATE,
    employment_end_date    DATE
);

CREATE TABLE IF NOT EXISTS test_masked.student (
    sis_student_id         TEXT PRIMARY KEY,
    person_ref             TEXT,
    first_name             TEXT,
    last_name              TEXT,
    date_of_birth          DATE,
    email                  TEXT,
    phone                  TEXT,
    postcode               TEXT,
```

```
    program_code           TEXT,
    study_level            TEXT,
    enrolment_status       TEXT,
    enrolment_year         INT,
    completion_year        INT,
    international_flag     TEXT,
    gpa                    NUMERIC(4, 2)
);

CREATE TABLE IF NOT EXISTS test_masked.constituent (
    crm_contact_id         TEXT PRIMARY KEY,
    person_ref             TEXT,
    first_name             TEXT,
    last_name              TEXT,
    date_of_birth          DATE,
    email                  TEXT,
    phone                  TEXT,
    postcode               TEXT,
    contact_type           TEXT,
    alumni_year            INT,
    donor_flag             TEXT,
    donation_band_aud      NUMERIC(12, 0),  -- rounded to the nearest $1,000
    engagement_score       INT
);

CREATE OR REPLACE PROCEDURE test_masked.provision(p_run_id INT)
LANGUAGE plpgsql AS $$
BEGIN
    TRUNCATE test_masked.employee, test_masked.student, test_masked.constituent;

    INSERT INTO test_masked.employee
    SELECT test_masked.pseudo(e.hr_employee_id, 'EMP'), test_masked.pseudo(l.person_id,
'PER'),
           test_masked.fake_first_name(e.hr_employee_id),
test_masked.fake_last_name(e.hr_employee_id),
           test_masked.shift_date(e.date_of_birth, e.hr_employee_id),
           lower(test_masked.pseudo(e.hr_employee_id, 'emp')) || '@example.invalid',
           test_masked.fake_phone(e.hr_employee_id),
           e.postcode, e.department, e.position_title, e.salary_band, e.employment_status,
           e.employment_start_date, e.employment_end_date
      FROM silver.employee_current e
      JOIN silver.link_person l ON l.system_name = 'HRMS' AND l.source_record_id =
e.hr_employee_id
     WHERE e.anonymised_at IS NULL;

    INSERT INTO test_masked.student
    SELECT test_masked.pseudo(s.sis_student_id, 'STU'), test_masked.pseudo(l.person_id,
'PER'),
           test_masked.fake_first_name(s.sis_student_id),
test_masked.fake_last_name(s.sis_student_id),
           test_masked.shift_date(s.date_of_birth, s.sis_student_id),
           lower(test_masked.pseudo(s.sis_student_id, 'stu')) || '@example.invalid',
           test_masked.fake_phone(s.sis_student_id),
           s.postcode, s.program_code, s.study_level, s.enrolment_status,
s.enrolment_year,
           s.completion_year, s.international_flag, s.gpa
      FROM silver.student_current s
      JOIN silver.link_person l ON l.system_name = 'SIS' AND l.source_record_id =
s.sis_student_id
     WHERE s.anonymised_at IS NULL;

    INSERT INTO test_masked.constituent
    SELECT test_masked.pseudo(c.crm_contact_id, 'CRM'), test_masked.pseudo(l.person_id,
'PER'),
           test_masked.fake_first_name(c.crm_contact_id),
test_masked.fake_last_name(c.crm_contact_id),
           test_masked.shift_date(c.date_of_birth, c.crm_contact_id),
           lower(test_masked.pseudo(c.crm_contact_id, 'crm')) || '@example.invalid',
           test_masked.fake_phone(c.crm_contact_id),
           c.postcode, c.contact_type, c.alumni_year, c.donor_flag,
           round(c.lifetime_donation_aud, -3), c.engagement_score
      FROM silver.constituent_current c
      JOIN silver.link_person l ON l.system_name = 'ADV_CRM' AND l.source_record_id =
c.crm_contact_id
     WHERE c.anonymised_at IS NULL;
```

```
    INSERT INTO audit.lineage (run_id, source, target, operation, row_count, masked)
VALUES
    (p_run_id, 'silver.sat_employee',    'test_masked.employee',    'MASK', (SELECT
count(*) FROM test_masked.employee),    TRUE),
    (p_run_id, 'silver.sat_student',     'test_masked.student',     'MASK', (SELECT
count(*) FROM test_masked.student),     TRUE),
    (p_run_id, 'silver.sat_constituent', 'test_masked.constituent', 'MASK', (SELECT
count(*) FROM test_masked.constituent), TRUE);
END;
$$;
```

**`sql/10_gold.sql`** – Gold star schema

```
-- 10 Gold: a small star schema for reporting. No names, emails, phones or dates of birth.
-- Design component: Figure 1, box 4 - Gold star schema for reporting (step BUILD); Figure
4(a).

-- Person dimension: one row per resolved person, with flags for the systems they appear
in.
CREATE TABLE IF NOT EXISTS gold.dim_person (
    person_key     SERIAL PRIMARY KEY,
    person_id      TEXT UNIQUE NOT NULL,
    in_hrms        BOOLEAN NOT NULL,
    in_sis         BOOLEAN NOT NULL,
    in_crm         BOOLEAN NOT NULL,
    system_count   INT NOT NULL,
    confidence     TEXT NOT NULL            -- weakest match used to build this person
);

-- Program dimension: one row per study program.
CREATE TABLE IF NOT EXISTS gold.dim_program (
    program_key    SERIAL PRIMARY KEY,
    program_code   TEXT UNIQUE NOT NULL,
    program_name   TEXT,
    study_level    TEXT
);

-- Enrolment fact: one row per SIS student record, linked to the person and the program.
CREATE TABLE IF NOT EXISTS gold.fact_enrolment (
    person_key        INT REFERENCES gold.dim_person,
    program_key       INT REFERENCES gold.dim_program,
    enrolment_year    INT,
    completion_year   INT,
    enrolment_status  TEXT,
    is_international  BOOLEAN,
    gpa               NUMERIC(4, 2),
    fees_owing_aud    NUMERIC(12, 2)
);

-- Giving fact: one row per CRM contact, linked to the person.
CREATE TABLE IF NOT EXISTS gold.fact_giving (
    person_key            INT REFERENCES gold.dim_person,
    contact_type          TEXT,
    alumni_year           INT,
    is_donor              BOOLEAN,
    lifetime_donation_aud NUMERIC(12, 2),
    engagement_score      INT
);

-- Rebuild the whole star schema from silver. Called by the pipeline step 'gold'.
CREATE OR REPLACE PROCEDURE gold.build(p_run_id INT)
LANGUAGE plpgsql AS $$
BEGIN
    -- Gold is rebuilt in full each run, so it always matches silver (see Table 2 trade-
offs).
    TRUNCATE gold.fact_enrolment, gold.fact_giving, gold.dim_person, gold.dim_program
RESTART IDENTITY;

    -- One person per person_id in the link; confidence is the weakest rule used for that
person.
```

```
    INSERT INTO gold.dim_person (person_id, in_hrms, in_sis, in_crm, system_count,
confidence)
    SELECT person_id,
           bool_or(system_name = 'HRMS'), bool_or(system_name = 'SIS'),
bool_or(system_name = 'ADV_CRM'),
           count(DISTINCT system_name),
           CASE WHEN bool_or(confidence = 'MEDIUM') THEN 'MEDIUM'
                WHEN bool_or(confidence = 'HIGH')   THEN 'HIGH'
                ELSE 'SINGLE' END
      FROM silver.link_person
     GROUP BY person_id;

    -- Programs come from the current student satellite rows.
    INSERT INTO gold.dim_program (program_code, program_name, study_level)
    SELECT DISTINCT program_code, program_name, study_level FROM silver.student_current;

    -- Enrolments: SIS records joined to their person and program. No names or contact
details are copied.
    INSERT INTO gold.fact_enrolment
    SELECT p.person_key, g.program_key, s.enrolment_year, s.completion_year,
s.enrolment_status,
           s.international_flag = 'Y', s.gpa, s.outstanding_fees_aud
      FROM silver.student_current s
      JOIN silver.link_person l ON l.system_name = 'SIS' AND l.source_record_id =
s.sis_student_id
      JOIN gold.dim_person p ON p.person_id = l.person_id
      JOIN gold.dim_program g ON g.program_code = s.program_code;

    -- Giving: CRM records joined to their person, so donors can be counted across
systems.
    INSERT INTO gold.fact_giving
    SELECT p.person_key, c.contact_type, c.alumni_year, c.donor_flag = 'Y',
c.lifetime_donation_aud,
           c.engagement_score
      FROM silver.constituent_current c
      JOIN silver.link_person l ON l.system_name = 'ADV_CRM' AND l.source_record_id =
c.crm_contact_id
      JOIN gold.dim_person p ON p.person_id = l.person_id;

    -- Lineage: one BUILD row per gold table.
    INSERT INTO audit.lineage (run_id, source, target, operation, row_count) VALUES
    (p_run_id, 'silver.link_person',  'gold.dim_person',     'BUILD', (SELECT count(*)
FROM gold.dim_person)),
    (p_run_id, 'silver.sat_student',  'gold.dim_program',    'BUILD', (SELECT count(*)
FROM gold.dim_program)),
    (p_run_id, 'silver.sat_student',  'gold.fact_enrolment', 'BUILD', (SELECT count(*)
FROM gold.fact_enrolment)),
    (p_run_id, 'silver.sat_constituent', 'gold.fact_giving', 'BUILD', (SELECT count(*)
FROM gold.fact_giving));
END;
$$;
```

**`sql/11_reports.sql`** – Report views and end-to-end checks

```
-- 11 Reports. Each view answers one question. Run them in psql, e.g.
-- Design component: Figure 1, box 5 - Reports and end-to-end testing (data use cases UC1
to UC5).
--     SELECT * FROM reports.checks;
-- reports.checks is the end-to-end test: every row should say PASS.

-- Every step of the most recent pipeline run.
CREATE OR REPLACE VIEW reports.pipeline_runs AS
SELECT run_id, step_name, status, rows_read, rows_loaded, error_message,
       round(extract(epoch FROM finished_at - started_at)::NUMERIC, 1) AS seconds
  FROM audit.pipeline_run
 WHERE batch_id = (SELECT batch_id FROM audit.pipeline_run ORDER BY run_id DESC LIMIT 1)
 ORDER BY run_id;

-- Every failed step ever (they stay logged even though their work was rolled back).
CREATE OR REPLACE VIEW reports.failed_steps AS
SELECT run_id, step_name, error_message, started_at
  FROM audit.pipeline_run WHERE status = 'FAILED' ORDER BY run_id;
```

```
-- What retention did on its most recent run.
CREATE OR REPLACE VIEW reports.retention_results AS
SELECT l.policy_id, p.description, l.target, l.action, l.rows_before, l.rows_affected,
l.rows_after
  FROM audit.purge_log l JOIN audit.retention_policy p USING (policy_id)
 WHERE l.run_id = (SELECT max(run_id) FROM audit.purge_log)
 ORDER BY l.policy_id;

-- Personal data that is past its use-by date right now (should be empty after a run).
CREATE OR REPLACE VIEW reports.retention_overdue AS
SELECT 'RP-01' AS policy_id, sis_student_id AS record_id, retention_expiry_date
  FROM silver.student_current WHERE retention_expiry_date < current_date AND anonymised_at
IS NULL
UNION ALL
SELECT 'RP-02', hr_employee_id, retention_expiry_date
  FROM silver.employee_current WHERE retention_expiry_date < current_date AND
anonymised_at IS NULL
UNION ALL
SELECT 'RP-03', crm_contact_id, retention_expiry_date
  FROM silver.constituent_current WHERE retention_expiry_date < current_date AND
anonymised_at IS NULL
UNION ALL
SELECT 'RP-04', shadow_record_id, retention_expiry_date
  FROM dev_sandbox.shadow_extracts WHERE retention_expiry_date < current_date;

-- The catalogue: every dataset, its owner, its personal-data columns and where its data
came from.
CREATE OR REPLACE VIEW reports.catalogue AS
SELECT d.dataset, d.layer, d.status, d.owner, d.real_pii,
       (SELECT count(*) FROM audit.pii_column c WHERE c.dataset = d.dataset) AS
pii_columns,
       coalesce((SELECT string_agg(DISTINCT l.source, ', ') FROM audit.lineage l WHERE
l.target = d.dataset),
                'unknown (no lineage)') AS comes_from
  FROM audit.dataset d
 ORDER BY d.layer, d.dataset;

-- How many people appear in one, two or three systems.
CREATE OR REPLACE VIEW reports.identity_overlap AS
SELECT system_count,
       concat_ws(' + ', CASE WHEN in_hrms THEN 'HRMS' END, CASE WHEN in_sis THEN 'SIS'
END,
                        CASE WHEN in_crm THEN 'ADV_CRM' END) AS systems,
       count(*) AS people
  FROM gold.dim_person
 GROUP BY 1, 2
 ORDER BY 1, 2;

-- How accurate the matching is, compared with the answer key.
-- A "link" is a pair of records put in the same person.
--   precision = of the links we made, how many were right
--   recall    = of the links that should exist, how many we found
CREATE OR REPLACE VIEW reports.identity_accuracy AS
WITH r AS (
    SELECT l.person_id, k.true_person_id
      FROM silver.link_person l
      JOIN qa.identity_answer_key k ON k.source_system = l.system_name AND
k.source_record_id = l.source_record_id
),
made    AS (SELECT sum(n * (n - 1) / 2) AS links FROM (SELECT count(*) n FROM r GROUP BY
person_id) x),
should  AS (SELECT sum(n * (n - 1) / 2) AS links FROM (SELECT count(*) n FROM r GROUP BY
true_person_id) x),
correct AS (SELECT sum(n * (n - 1) / 2) AS links FROM (SELECT count(*) n FROM r GROUP BY
person_id, true_person_id) x)
SELECT (SELECT count(DISTINCT true_person_id) FROM r) AS real_people,
       (SELECT count(DISTINCT person_id) FROM r)      AS people_found,
       correct.links                                   AS correct_links,
       made.links - correct.links                      AS wrong_links,
       should.links - correct.links                    AS missed_links,
       round(100.0 * correct.links / made.links, 2)    AS precision_pct,
       round(100.0 * correct.links / should.links, 2)  AS recall_pct
  FROM made, should, correct;
```

```
-- Is any real personal data in the masked test tables? Every count should be 0.
CREATE OR REPLACE VIEW reports.masking_check AS
WITH real AS (
    SELECT silver.clean_email(email) AS email, silver.clean_phone(phone) AS phone,
           lower(first_name) AS first_name, lower(last_name) AS last_name, date_of_birth
      FROM src_hrms.employee
    UNION ALL
    SELECT silver.clean_email(email), silver.clean_phone(phone), lower(first_name),
lower(last_name), date_of_birth
      FROM src_sis.student
    UNION ALL
    SELECT silver.clean_email(email), silver.clean_phone(phone), lower(first_name),
lower(last_name), date_of_birth
      FROM src_crm.constituent
),
masked AS (
    SELECT hr_employee_id AS id, email, phone, lower(first_name) AS first_name,
lower(last_name) AS last_name,
           date_of_birth FROM test_masked.employee
    UNION ALL
    SELECT sis_student_id, email, phone, lower(first_name), lower(last_name),
date_of_birth FROM test_masked.student
    UNION ALL
    SELECT crm_contact_id, email, phone, lower(first_name), lower(last_name),
date_of_birth FROM test_masked.constituent
)
SELECT 'real email addresses' AS check_name,
       (SELECT count(*) FROM masked WHERE email IN (SELECT email FROM real)) AS found
UNION ALL
SELECT 'real phone numbers', (SELECT count(*) FROM masked WHERE phone IN (SELECT phone
FROM real))
UNION ALL
SELECT 'real first names', (SELECT count(*) FROM masked WHERE first_name IN (SELECT
first_name FROM real))
UNION ALL
SELECT 'real name + date of birth', (SELECT count(*) FROM masked m JOIN real r USING
(first_name, last_name, date_of_birth))
UNION ALL
SELECT 'real source IDs', (SELECT count(*) FROM masked
                           WHERE id IN (SELECT hr_employee_id FROM src_hrms.employee
                                        UNION ALL SELECT sis_student_id FROM
src_sis.student
                                        UNION ALL SELECT crm_contact_id FROM
src_crm.constituent))
UNION ALL
SELECT 'student keeps real date of birth',
       (SELECT count(*) FROM test_masked.student m JOIN src_sis.student s
            ON test_masked.pseudo(s.sis_student_id, 'STU') = m.sis_student_id
         WHERE m.date_of_birth = s.date_of_birth)
UNION ALL
SELECT 'anonymised people included',
       (SELECT count(*) FROM test_masked.student m JOIN silver.student_current s
            ON test_masked.pseudo(s.sis_student_id, 'STU') = m.sis_student_id
         WHERE s.anonymised_at IS NOT NULL);

-- End-to-end test. Every row should say PASS.
CREATE OR REPLACE VIEW reports.checks AS
WITH c (check_name, ok, detail) AS (
    SELECT 'every step of the last run succeeded',
           NOT EXISTS (SELECT 1 FROM reports.pipeline_runs WHERE status <> 'SUCCESS'),
           (SELECT count(*) FROM reports.pipeline_runs)::TEXT || ' steps'
    UNION ALL
    SELECT 'bronze has every source row',
           (SELECT count(*) FROM bronze.hrms_employee) = (SELECT count(*) FROM
src_hrms.employee)
       AND (SELECT count(*) FROM bronze.sis_student) = (SELECT count(*) FROM
src_sis.student)
       AND (SELECT count(*) FROM bronze.crm_constituent) = (SELECT count(*) FROM
src_crm.constituent), ''
    UNION ALL
    SELECT 'silver has a hub row for every source record',
           (SELECT count(*) FROM silver.hub_employee) = (SELECT count(*) FROM
src_hrms.employee)
       AND (SELECT count(*) FROM silver.hub_student) = (SELECT count(*) FROM
src_sis.student)
```

```
       AND (SELECT count(*) FROM silver.hub_constituent) = (SELECT count(*) FROM
src_crm.constituent), ''
    UNION ALL
    SELECT 'every source record is linked to one person',
           (SELECT count(*) FROM silver.link_person) = (SELECT count(*) FROM
src_hrms.employee)
                                                       + (SELECT count(*) FROM
src_sis.student)
                                                       + (SELECT count(*) FROM
src_crm.constituent), ''
    UNION ALL
    SELECT 'matching precision is at least 99%',
           (SELECT precision_pct >= 99 FROM reports.identity_accuracy),
           (SELECT precision_pct || '% precision, ' || recall_pct || '% recall' FROM
reports.identity_accuracy)
    UNION ALL
    SELECT 'masked data holds no real personal data',
           (SELECT sum(found) = 0 FROM reports.masking_check),
           (SELECT count(*) FROM test_masked.student) + (SELECT count(*) FROM
test_masked.employee)
           + (SELECT count(*) FROM test_masked.constituent) || ' masked rows'
    UNION ALL
    SELECT 'no personal data past its retention date',
           NOT EXISTS (SELECT 1 FROM reports.retention_overdue), ''
    UNION ALL
    SELECT 'shadow copies are no longer FOUND_BY_SCAN',
           NOT EXISTS (SELECT 1 FROM audit.dataset WHERE layer = 'dev_sandbox' AND status
= 'FOUND_BY_SCAN'),
           (SELECT status FROM audit.dataset WHERE dataset =
'dev_sandbox.shadow_extracts')
    UNION ALL
    SELECT 'every table is in the catalogue',
           NOT EXISTS (SELECT 1 FROM information_schema.tables t
                       WHERE t.table_type = 'BASE TABLE'
                         AND t.table_schema IN ('src_hrms', 'src_sis', 'src_crm',
'dev_sandbox',
                                                'bronze', 'silver', 'gold',
'test_masked')
                         AND t.table_schema || '.' || t.table_name NOT IN (SELECT dataset
FROM audit.dataset)), ''
    UNION ALL
    SELECT 'every registered dataset has lineage',
           NOT EXISTS (SELECT 1 FROM audit.dataset d
                       WHERE d.status = 'REGISTERED' AND d.layer <> 'qa'
                         AND d.dataset NOT IN (SELECT target FROM audit.lineage)), ''
)
SELECT check_name, CASE WHEN ok THEN 'PASS' ELSE 'FAIL' END AS result, detail FROM c;
```

**`python/load_source_data.py`** – Source extract

```
"""Copy the sheets of the Excel workbook into the source tables.

The tables themselves are defined in sql/02_source_tables.sql.
"""

from pathlib import Path

import pandas as pd

# Design component: Figure 1, box 1 - Source systems (pipeline step EXTRACT).

# The synthetic data workbook, relative to the project folder.
WORKBOOK = Path(__file__).resolve().parent.parent / "data" /
"Wattlebrook_Synthetic_Data.xlsx"

# Workbook sheet -> source table it is copied into.
SOURCE_SHEETS = {
    "HRMS": "src_hrms.employee",
    "SIS": "src_sis.student",
    "ADV_CRM": "src_crm.constituent",
    "Identity_Answer_Key": "qa.identity_answer_key",
}
```

```
def copy_sheet(conn, sheet, table):
    """Replace the rows of a table with the rows of one sheet. Returns the number of
rows."""
    df = pd.read_excel(WORKBOOK, sheet_name=sheet, dtype=str)
    df = df.astype(object).where(df.notna(), None)  # empty cells become NULL

    # Full reload with PostgreSQL COPY, which is much faster than row-by-row INSERTs.
    conn.execute(f"TRUNCATE {table}")
    columns = ", ".join(df.columns)
    with conn.cursor().copy(f"COPY {table} ({columns}) FROM STDIN") as copy:
        for row in df.itertuples(index=False):
            copy.write_row(row)
    print(f"    {table:<28} {len(df):>6} rows")
    return len(df)



def load_sources(conn, run_id):
    """Fill the three source systems and the answer key."""
    total = 0
    for sheet, table in SOURCE_SHEETS.items():
        rows = copy_sheet(conn, sheet, table)
        total += rows
        # Lineage: record where each source table came from.
        conn.execute(
            "INSERT INTO audit.lineage (run_id, source, target, operation, row_count)
VALUES (%s, %s, %s, 'EXTRACT', %s)",
            (run_id, f"{WORKBOOK.name} / {sheet}", table, rows),
        )
    conn.execute("UPDATE audit.pipeline_run SET rows_loaded = %s WHERE run_id = %s",
(total, run_id))



def load_shadow_copies(conn, run_id):
    """Recreate the copies found by the audit. Not registered in the catalogue, on
purpose."""
    rows = copy_sheet(conn, "Shadow_Extracts", "dev_sandbox.shadow_extracts")
    conn.execute("UPDATE audit.pipeline_run SET rows_loaded = %s WHERE run_id = %s",
(rows, run_id))
```

### **python/run_pipeline.py** – Orchestration

```
"""Run the Wattlebrook pipeline from start to finish.

    python python/run_pipeline.py            run the pipeline
    python python/run_pipeline.py --reset    delete everything first and start again

Each step runs in its own transaction. If a step fails, its changes are rolled
back, but the FAILED row stays in audit.pipeline_run because it is written in a
separate transaction.
"""

import os
import sys
import uuid
from pathlib import Path

import psycopg
from dotenv import load_dotenv

from load_source_data import load_shadow_copies, load_sources

# Design component: Figure 1 - Orchestration (runs sql/01-11, then the ten pipeline
steps).

ROOT = Path(__file__).resolve().parent.parent
load_dotenv(ROOT / ".env")
```

```
DB = dict(
    host=os.getenv("POSTGRES_HOST", "localhost"),
    port=os.getenv("POSTGRES_PORT", "5432"),
    dbname=os.getenv("POSTGRES_DB"),
    user=os.getenv("POSTGRES_USER"),
    password=os.getenv("POSTGRES_PASSWORD"),
)

SCHEMAS = ["reports", "gold", "test_masked", "silver", "bronze", "dev_sandbox",
           "src_hrms", "src_sis", "src_crm", "qa", "audit"]

# (step name, SQL to run). %s is replaced with the step's run_id.
STEPS = [
    ("load_sources",       load_sources),
    ("bronze_hrms",        "CALL bronze.load_source(%s, 'src_hrms.employee',
'hrms_employee', 'HRMS')"),
    ("bronze_sis",         "CALL bronze.load_source(%s, 'src_sis.student', 'sis_student',
'SIS')"),
    ("bronze_crm",         "CALL bronze.load_source(%s, 'src_crm.constituent',
'crm_constituent', 'ADV_CRM')"),
    ("silver_vault",       "CALL silver.load_vault(%s)"),
    ("identity_matching",  "CALL silver.resolve_identities(%s)"),
    ("catalogue_scan",     "CALL audit.scan_catalogue(%s)"),
    ("retention",          "CALL audit.run_retention(%s)"),
    ("masked_test_data",   "CALL test_masked.provision(%s)"),
    ("gold",               "CALL gold.build(%s)"),
]



def run_step(batch_id, name, work):
    with psycopg.connect(**DB) as conn:
        run_id = conn.execute(
            "INSERT INTO audit.pipeline_run (batch_id, step_name, status) VALUES (%s, %s,
'RUNNING') RETURNING run_id",
            (batch_id, name),
        ).fetchone()[0]
    print(f"{name} ...")

    try:
        with psycopg.connect(**DB) as conn:  # commits if the block finishes, rolls back
on an error
            if callable(work):
                work(conn, run_id)
            else:
                conn.execute(work, (run_id,))
    except psycopg.Error as error:
        message = str(error).splitlines()[0]
        with psycopg.connect(**DB) as conn:
            conn.execute(
                "UPDATE audit.pipeline_run SET status = 'FAILED', error_message = %s,
finished_at = now() WHERE run_id = %s",
                (message, run_id),
            )
        sys.exit(f"{name} FAILED: {message}")

    with psycopg.connect(**DB) as conn:
        conn.execute("UPDATE audit.pipeline_run SET status = 'SUCCESS', finished_at =
now() WHERE run_id = %s", (run_id,))



def main():
    if not DB["dbname"]:
        sys.exit("Copy .env.example to .env first.")

    with psycopg.connect(**DB) as conn:
        if "--reset" in sys.argv:
            print("Dropping all schemas")
            for schema in SCHEMAS:
                conn.execute(f"DROP SCHEMA IF EXISTS {schema} CASCADE")
        for sql_file in sorted((ROOT / "sql").glob("[0-9][0-9]_*.sql")):  # not
demo_queries.sql
            conn.execute(sql_file.read_text(encoding="utf-8"))
```

```
    batch_id = str(uuid.uuid4())

    # The shadow copies are only loaded once: after that, retention decides what is left
of them.
    with psycopg.connect(**DB) as conn:
        loaded_before = conn.execute(
            "SELECT count(*) FROM audit.pipeline_run WHERE step_name =
'load_shadow_copies' AND status = 'SUCCESS'"
        ).fetchone()[0]
    if not loaded_before:
        run_step(batch_id, "load_shadow_copies", load_shadow_copies)

    for name, work in STEPS:
        run_step(batch_id, name, work)

    print("Done. Check the result with:  SELECT * FROM reports.checks;")


if __name__ == "__main__":
    main()
```

**`python/dashboard.py`** – Dashboard

```
"""Build the Wattlebrook data platform dashboard.

    python python/dashboard.py

Reads the report views (reports schema) and the gold star schema after a
pipeline run, and draws one dashboard page to dashboard/wattlebrook_dashboard.png.
It only reads data: nothing in the database is changed, and no personal data is
shown (gold holds no names, emails, phone numbers or dates of birth).

Design component: Figure 1, box 5 - Reports (dashboard over the report views).

Panels and the use case each one serves:
    KPI tiles            Operations, UC1, UC2, UC3
    Retention by policy  UC1  reports.retention_results
    People per systems   UC2  reports.identity_overlap
    Catalogue by layer   UC4  reports.catalogue
    Enrolment by level   UC5  gold.fact_enrolment + gold.dim_program
    Donors per systems   UC5  gold.fact_giving + gold.dim_person
    Pipeline step times  Operations  reports.pipeline_runs
"""

import os
from pathlib import Path

import matplotlib

matplotlib.use("Agg")  # draw to a file, no window needed
import matplotlib.pyplot as plt
import psycopg
from dotenv import load_dotenv

ROOT = Path(__file__).resolve().parent.parent
load_dotenv(ROOT / ".env")
OUT = ROOT / "dashboard" / "wattlebrook_dashboard.png"

DB = dict(
    host=os.getenv("POSTGRES_HOST", "localhost"),
    port=os.getenv("POSTGRES_PORT", "5432"),
    dbname=os.getenv("POSTGRES_DB"),
    user=os.getenv("POSTGRES_USER"),
    password=os.getenv("POSTGRES_PASSWORD"),
)

# Colours: one fixed order for series, muted ink for text and axes.
```

```
SURFACE, INK, INK_2, GRID = "#fcfcfb", "#0b0b0b", "#52514e", "#e4e3df"
SERIES = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100"]
GOOD, BAD = "#008300", "#c62828"



def rows(conn, sql):
    return conn.execute(sql).fetchall()



def fetch():
    """Run every query the dashboard needs and return the results by name."""
    with psycopg.connect(**DB) as conn:
        return {
            "checks": rows(conn, "SELECT check_name, result FROM reports.checks"),
            "accuracy": rows(conn, "SELECT real_people, people_found, precision_pct,
recall_pct "
                                   "FROM reports.identity_accuracy")[0],
            "records": rows(conn, "SELECT count(*) FROM silver.link_person")[0][0],
            "overdue": rows(conn, "SELECT count(*) FROM reports.retention_overdue")[0][0],
            "masked": rows(conn, "SELECT (SELECT count(*) FROM test_masked.employee)"
                                 " + (SELECT count(*) FROM test_masked.student)"
                                 " + (SELECT count(*) FROM test_masked.constituent)")[0]
[0],
            "leaks": rows(conn, "SELECT coalesce(sum(found), 0) FROM
reports.masking_check")[0][0],
            "retention": rows(conn, "SELECT policy_id, target, rows_affected, rows_after "
                                    "FROM reports.retention_results ORDER BY policy_id"),
            "overlap": rows(conn, "SELECT systems, people FROM reports.identity_overlap "
                                  "ORDER BY system_count, people DESC"),
            "catalogue": rows(conn, "SELECT layer, count(*) FILTER (WHERE real_pii), "
                                    "count(*) FILTER (WHERE NOT real_pii) "
                                    "FROM reports.catalogue GROUP BY layer ORDER BY
layer"),
            "enrolment": rows(conn, "SELECT p.study_level, f.enrolment_status, count(*) "
                                    "FROM gold.fact_enrolment f JOIN gold.dim_program p
USING (program_key) "
                                    "GROUP BY 1, 2"),
            "donors": rows(conn, "SELECT d.system_count, count(*),
round(sum(g.lifetime_donation_aud) / 1e6, 1) "
                                 "FROM gold.fact_giving g JOIN gold.dim_person d USING
(person_key) "
                                 "WHERE g.is_donor GROUP BY 1 ORDER BY 1"),
            "steps": rows(conn, "SELECT step_name, seconds, status FROM
reports.pipeline_runs ORDER BY run_id"),
        }



def style(ax, title):
    """Shared look for every chart panel: title, light grid, no box."""
    ax.set_title(title, loc="left", fontsize=11, fontweight="bold", color=INK, pad=22)
    ax.set_facecolor(SURFACE)
    for side in ("top", "right", "left"):
        ax.spines[side].set_visible(False)
    ax.spines["bottom"].set_color(GRID)
    ax.tick_params(colors=INK_2, labelsize=8.5, length=0)
    ax.grid(axis="x", color=GRID, linewidth=0.8)
    ax.set_axisbelow(True)



def tile(fig, x, label, value, note, colour=INK):
    """One headline number with a label above it and a note below it."""
    ax = fig.add_axes([x, 0.80, 0.18, 0.11])
    ax.axis("off")
    ax.add_patch(plt.Rectangle((0, 0), 1, 1, transform=ax.transAxes, facecolor="white",
                               edgecolor=GRID, linewidth=1))
    ax.text(0.06, 0.78, label, fontsize=9, color=INK_2, transform=ax.transAxes)
    ax.text(0.06, 0.36, value, fontsize=20, fontweight="bold", color=colour,
transform=ax.transAxes)
    ax.text(0.06, 0.10, note, fontsize=8, color=INK_2, transform=ax.transAxes)



def main():
```

```
    if not DB["dbname"]:
        raise SystemExit("Copy .env.example to .env first.")
    d = fetch()

    fig = plt.figure(figsize=(16, 11), dpi=150, facecolor=SURFACE)
    fig.text(0.03, 0.965, "Wattlebrook University governed data platform", fontsize=17,
             fontweight="bold", color=INK)
    fig.text(0.03, 0.940, "Live from the reports and gold schemas after the latest
pipeline run. "
                          "No names, emails, phone numbers or dates of birth are shown.",
             fontsize=9.5, color=INK_2)

    # Row 1: headline numbers
    passed = sum(1 for _, r in d["checks"] if r == "PASS")
    real, found, prec, rec = d["accuracy"]
    tile(fig, 0.03, "End-to-end checks", f"{passed} / {len(d['checks'])} PASS",
         "reports.checks", GOOD if passed == len(d["checks"]) else BAD)
    tile(fig, 0.225, "People resolved", f"{found:,}",
         f"from {d['records']:,} source records (true {real:,})")
    tile(fig, 0.42, "Matching accuracy", f"{float(prec):.2f}%",
         f"precision · recall {float(rec):.2f}%")
    tile(fig, 0.615, "Personal data past use-by", f"{d['overdue']:,} rows",
         "reports.retention_overdue", GOOD if d["overdue"] == 0 else BAD)
    tile(fig, 0.81, "Masked test rows", f"{d['masked']:,}",
         f"{d['leaks']} real values found (reports.masking_check)")

    # Row 2, left: UC1 retention - removed vs kept per policy
    ax = fig.add_axes([0.09, 0.45, 0.24, 0.27])
    style(ax, "UC1  Retention: records removed and kept")
    labels = [f"{p}\n{target.split('.')[-1]}" for p, target, *_ in d["retention"]]
    removed = [r[2] for r in d["retention"]]
    kept = [r[3] for r in d["retention"]]
    y = range(len(labels))
    ax.barh(y, removed, color=SERIES[1], height=0.6, label="Anonymised / deleted",
edgecolor=SURFACE, linewidth=1.5)
    ax.barh(y, kept, left=removed, color=SERIES[0], height=0.6, label="Kept",
edgecolor=SURFACE, linewidth=1.5)
    for i, (a, b) in enumerate(zip(removed, kept)):
        ax.text(a + b + 300, i, f"{a:,} removed", va="center", fontsize=8, color=INK_2)
    ax.set_yticks(list(y), labels, fontsize=8)
    ax.invert_yaxis()
    ax.set_xlim(0, max(a + b for a, b in zip(removed, kept)) * 1.35)
    ax.legend(loc="lower left", bbox_to_anchor=(0, 1.0), fontsize=8, frameon=False,
ncol=4,
              handlelength=1.2, columnspacing=1.0, borderaxespad=0.2)

    # Row 2, middle: UC2 identity overlap
    ax = fig.add_axes([0.43, 0.45, 0.22, 0.27])
    style(ax, "UC2  People by the systems they appear in")
    names = [s for s, _ in d["overlap"]]
    people = [p for _, p in d["overlap"]]
    ax.barh(range(len(names)), people, color=SERIES[0], height=0.6)
    for i, p in enumerate(people):
        ax.text(p + 150, i, f"{p:,}", va="center", fontsize=8, color=INK_2)
    ax.set_yticks(range(len(names)), names, fontsize=8)
    ax.invert_yaxis()
    ax.set_xlim(0, max(people) * 1.2)

    # Row 2, right: UC4 catalogue by layer
    ax = fig.add_axes([0.75, 0.45, 0.22, 0.27])
    style(ax, "UC4  Catalogued tables by layer")
    layers = [r[0] for r in d["catalogue"]]
    pii = [r[1] for r in d["catalogue"]]
    nopii = [r[2] for r in d["catalogue"]]
    ax.barh(range(len(layers)), pii, color=SERIES[1], height=0.6, label="Holds real
personal data",
            edgecolor=SURFACE, linewidth=1.5)
    ax.barh(range(len(layers)), nopii, left=pii, color=SERIES[0], height=0.6, label="No
real personal data",
            edgecolor=SURFACE, linewidth=1.5)
    ax.set_yticks(range(len(layers)), layers, fontsize=8)
    ax.invert_yaxis()
    ax.set_xlim(0, max(a + b for a, b in zip(pii, nopii)) + 1.5)
    ax.legend(loc="lower left", bbox_to_anchor=(0, 1.0), fontsize=8, frameon=False,
ncol=4,
```

```
              handlelength=1.2, columnspacing=1.0, borderaxespad=0.2)

    # Row 3, left: UC5 enrolment by study level and status
    ax = fig.add_axes([0.09, 0.07, 0.24, 0.27])
    style(ax, "UC5  Enrolments by study level and status")
    levels = ["Undergraduate", "Postgraduate", "Research"]
    statuses = ["Completed", "Enrolled", "Withdrawn", "Deferred"]
    counts = {(l, s): c for l, s, c in d["enrolment"]}
    left = [0] * len(levels)
    for k, s in enumerate(statuses):
        vals = [counts.get((l, s), 0) for l in levels]
        ax.barh(range(len(levels)), vals, left=left, color=SERIES[k], height=0.6, label=s,
                edgecolor=SURFACE, linewidth=1.5)
        left = [a + b for a, b in zip(left, vals)]
    for i, total in enumerate(left):
        ax.text(total + 120, i, f"{total:,}", va="center", fontsize=8, color=INK_2)
    ax.set_yticks(range(len(levels)), levels, fontsize=8)
    ax.invert_yaxis()
    ax.set_xlim(0, max(left) * 1.18)
    ax.legend(loc="lower left", bbox_to_anchor=(0, 1.0), fontsize=8, frameon=False,
ncol=4,
              handlelength=1.2, columnspacing=1.0, borderaxespad=0.2)

    # Row 3, middle: UC5 donors by number of systems
    ax = fig.add_axes([0.43, 0.07, 0.22, 0.27])
    style(ax, "UC5  Donors by number of systems they appear in")
    sys_labels = {1: "CRM only", 2: "2 systems", 3: "All 3 systems"}
    dn = [r[1] for r in d["donors"]]
    ax.barh(range(len(dn)), dn, color=SERIES[0], height=0.6)
    for i, (_, n, m) in enumerate(d["donors"]):
        ax.text(n + 20, i, f"{n:,} donors · ${float(m):.1f}M lifetime", va="center",
fontsize=8, color=INK_2)
    ax.set_yticks(range(len(dn)), [sys_labels.get(r[0], r[0]) for r in d["donors"]],
fontsize=8)
    ax.invert_yaxis()
    ax.set_xlim(0, max(dn) * 1.9)

    # Row 3, right: operations - seconds per pipeline step
    ax = fig.add_axes([0.75, 0.07, 0.22, 0.27])
    style(ax, "Operations  Seconds per pipeline step")
    steps = d["steps"]
    secs = [float(s or 0) for _, s, _ in steps]
    ax.barh(range(len(steps)), secs, height=0.6,
            color=[SERIES[0] if st == "SUCCESS" else BAD for *_, st in steps])
    for i, (_, s, st) in enumerate(steps):
        ax.text(float(s or 0) + 0.15, i, f"{float(s or 0):.1f}s {st}", va="center",
fontsize=7.5, color=INK_2)
    ax.set_yticks(range(len(steps)), [n for n, *_ in steps], fontsize=7.5)
    ax.invert_yaxis()
    ax.set_xlim(0, max(secs) * 1.5)

    OUT.parent.mkdir(exist_ok=True)
    fig.savefig(OUT, facecolor=SURFACE)
    print(f"Dashboard written to {OUT.relative_to(ROOT)}")



if __name__ == "__main__":
    main()
```

**`sql/demo_queries.sql`** – Demo and failure test queries

```
-- Demo queries for the presentation. Run them one at a time in psql after the pipeline.
-- They only read data, except the failure demo at the end, which undoes itself.


-- 1. The problem: Anh Le is in all three systems with three different IDs, emails and
phone formats.
SELECT 'HRMS' AS system, hr_employee_id AS id, first_name, last_name, email, phone FROM
src_hrms.employee WHERE hr_employee_id = 'EMP000001'
UNION ALL
SELECT 'SIS', sis_student_id, first_name, last_name, email, phone FROM src_sis.student
WHERE sis_student_id = 'STU0000001'
UNION ALL
```

```
SELECT 'ADV_CRM', crm_contact_id, first_name, last_name, email, phone FROM
src_crm.constituent WHERE crm_contact_id = 'CRM0000001';

-- 2. Identity matching linked the three records to one person, and says which rules
matched.
SELECT * FROM silver.link_person
 WHERE person_id = (SELECT person_id FROM silver.link_person WHERE source_record_id =
'STU0000001');

-- 3. The same person in the masked test data: fake name, email and phone, same person_ref
everywhere.
SELECT 'HRMS' AS system, hr_employee_id AS id, person_ref, first_name, last_name, email,
phone FROM test_masked.employee
 WHERE person_ref = (SELECT person_ref FROM test_masked.student WHERE sis_student_id =
test_masked.pseudo('STU0000001', 'STU'))
UNION ALL
SELECT 'SIS', sis_student_id, person_ref, first_name, last_name, email, phone FROM
test_masked.student
 WHERE sis_student_id = test_masked.pseudo('STU0000001', 'STU');

-- 4. Pipeline log, catalogue and lineage.
SELECT * FROM reports.pipeline_runs;
SELECT * FROM reports.catalogue;
SELECT * FROM audit.lineage ORDER BY lineage_id DESC LIMIT 20;

-- 5. Retention: what was removed, and proof nothing overdue is left.
SELECT * FROM reports.retention_results;
SELECT count(*) AS still_overdue FROM reports.retention_overdue;

-- 6. Shadow copies that are left (still within their approved date), by where they were
found.
SELECT environment, count(*) AS copies, min(retention_expiry_date) AS next_expiry
  FROM dev_sandbox.shadow_extracts GROUP BY environment ORDER BY copies DESC;

-- 7. Identity results and accuracy, masking check, and the end-to-end test.
SELECT * FROM reports.identity_overlap;
SELECT * FROM reports.identity_accuracy;
SELECT * FROM reports.masking_check;
SELECT * FROM reports.checks;

-- 8. Index demo: look up everything held about one email address.
EXPLAIN ANALYZE SELECT * FROM silver.sat_student WHERE email =
'anh.le1@student.wattlebrook.edu.au';

-- 9. Failure demo. Add a column the pipeline does not expect, run the pipeline (the
--    bronze_sis step fails and is rolled back), look at the log, then remove the column.
--    ALTER TABLE src_sis.student ADD COLUMN surprise TEXT;
--    (run: python python/run_pipeline.py)
--    SELECT * FROM reports.failed_steps;
--    ALTER TABLE src_sis.student DROP COLUMN surprise;
```

**`sql/demo_hubs_and_link.sql`** – Demo queries for hubs and the person link

```
-- Hubs and the link: demonstration queries.
--
-- Run after a clean pipeline run:  python python/run_pipeline.py --reset
-- Run them one at a time in psql. They only read data.
-- Anh Le (STU0000001, EMP000001, CRM0000001) is followed throughout.
--
-- The idea in one line:
--   A HUB holds one row per business key: the ID a system uses, plus an md5 hash of it.
--   The LINK (silver.link_person) says which source records belong to which person.
--   Personal details live in the satellites, so neither hubs nor the link hold any.
--
-- Numbers below are from a clean run on 13 Oct 2026. Retention uses today's date,
-- so a later run can give slightly different counts.



-- 1. The four hubs and how many keys each holds
SELECT 'hub_student' AS hub, count(*) AS keys, 'sis_student_id' AS business_key FROM
silver.hub_student
```

```
UNION ALL SELECT 'hub_employee',    count(*), 'hr_employee_id' FROM silver.hub_employee
UNION ALL SELECT 'hub_constituent', count(*), 'crm_contact_id' FROM silver.hub_constituent
UNION ALL SELECT 'hub_person',      count(*), 'person_id'      FROM silver.hub_person;
-- Expected: 15,000 students, 5,000 employees, 7,500 constituents and 20,399 people.
-- Talking point: the first three match the source record counts. hub_person is smaller
than
--                27,500 because one person can have records in several systems.



-- 2. What a hub row looks like: keys and load details only, nothing personal
SELECT * FROM silver.hub_student WHERE sis_student_id = 'STU0000001';

SELECT table_name, string_agg(column_name, ', ' ORDER BY ordinal_position) AS columns
  FROM information_schema.columns
 WHERE table_schema = 'silver' AND table_name IN ('hub_student', 'hub_employee',
'hub_constituent', 'hub_person')
 GROUP BY table_name ORDER BY table_name;
-- Expected: business key, hash key, load_dts and record_source. No name, email or date of
birth.



-- 3. The hash key is the md5 of the business key, so the same record always gets the same
key
SELECT sis_student_id, student_hk, md5(sis_student_id) AS recomputed, student_hk =
md5(sis_student_id) AS matches
  FROM silver.hub_student WHERE sis_student_id = 'STU0000001';

SELECT count(*) FILTER (WHERE student_hk = md5(sis_student_id)) AS matching_rows, count(*)
AS total_rows
  FROM silver.hub_student;
-- Expected: matches = true; 15,000 of 15,000 rows. This is why reloading a record never
creates
--           a duplicate hub row (the load uses ON CONFLICT DO NOTHING).



-- 4. Each source record has exactly one hub row (business keys are unique)
SELECT 'hub_student' AS hub, count(*) AS rows, count(DISTINCT sis_student_id) AS
distinct_keys FROM silver.hub_student
UNION ALL SELECT 'hub_employee',    count(*), count(DISTINCT hr_employee_id) FROM
silver.hub_employee
UNION ALL SELECT 'hub_constituent', count(*), count(DISTINCT crm_contact_id) FROM
silver.hub_constituent
UNION ALL SELECT 'hub_person',      count(*), count(DISTINCT person_id)      FROM
silver.hub_person;
-- Expected: rows equal distinct_keys on every line.



-- 5. Without the link, the three source hubs know nothing about each other
SELECT 'student hub' AS hub, sis_student_id AS business_key, student_hk AS hash_key
  FROM silver.hub_student WHERE sis_student_id = 'STU0000001'
UNION ALL
SELECT 'employee hub', hr_employee_id, employee_hk
  FROM silver.hub_employee WHERE hr_employee_id = 'EMP000001'
UNION ALL
SELECT 'constituent hub', crm_contact_id, constituent_hk
  FROM silver.hub_constituent WHERE crm_contact_id = 'CRM0000001';
-- Expected: three unrelated keys and hashes. Nothing here says they are the same person.



-- 6. The person hub: one row per person, with a stable ID and nothing else
SELECT * FROM silver.hub_person
 WHERE person_id = (SELECT person_id FROM silver.link_person WHERE source_record_id =
'STU0000001');
-- Expected: P-3EBCF1622F. The ID is built from the group of records that matched.



-- 7. The link: three rows, one person, and the rule that matched each record
SELECT * FROM silver.link_person
 WHERE source_record_id IN ('STU0000001', 'EMP000001', 'CRM0000001')
 ORDER BY system_name;
-- Expected: three rows with the same person_id, match_rule R2+R3, confidence HIGH.
```

```
-- Talking point: this is the whole idea. The source systems share no ID, yet the link
ties
--                three IDs to one person and records why.



-- 8. Following the link from the person hub to all three source hubs
SELECT p.person_id,
       l.system_name,
       l.source_record_id,
       l.match_rule,
       l.confidence,
       coalesce(hs.sis_student_id, he.hr_employee_id, hc.crm_contact_id) AS found_in_hub
  FROM silver.hub_person p
  JOIN silver.link_person l ON l.person_id = p.person_id
  LEFT JOIN silver.hub_student     hs ON l.system_name = 'SIS'     AND hs.sis_student_id =
l.source_record_id
  LEFT JOIN silver.hub_employee    he ON l.system_name = 'HRMS'    AND he.hr_employee_id =
l.source_record_id
  LEFT JOIN silver.hub_constituent hc ON l.system_name = 'ADV_CRM' AND hc.crm_contact_id =
l.source_record_id
 WHERE p.person_id = (SELECT person_id FROM silver.link_person WHERE source_record_id =
'STU0000001')
 ORDER BY l.system_name;
-- Expected: one person, three records, and found_in_hub filled in for each (none empty).



-- 9. Using the link: one person to everything held about them
SELECT l.person_id, l.system_name, l.source_record_id,
       coalesce(s.email, e.email, c.email) AS email_held,
       coalesce(s.program_name, e.position_title, c.contact_type) AS what_it_is
  FROM silver.link_person l
  LEFT JOIN silver.student_current     s ON l.system_name = 'SIS'     AND s.sis_student_id
= l.source_record_id
  LEFT JOIN silver.employee_current    e ON l.system_name = 'HRMS'    AND e.hr_employee_id
= l.source_record_id
  LEFT JOIN silver.constituent_current c ON l.system_name = 'ADV_CRM' AND c.crm_contact_id
= l.source_record_id
 WHERE l.person_id = (SELECT person_id FROM silver.link_person WHERE source_record_id =
'STU0000001')
 ORDER BY l.system_name;
-- Expected: three different emails for one person. The details come from the satellites;
--           the link only supplies the connection.



-- 10. The link has one row per source record, so rows are records, not people
SELECT system_name, count(*) AS link_rows FROM silver.link_person GROUP BY system_name
ORDER BY system_name;

SELECT count(*) AS link_rows, count(DISTINCT person_id) AS people FROM silver.link_person;
-- Expected: ADV_CRM 7,500, HRMS 5,000, SIS 15,000. In total 27,500 link rows for 20,399
people.
--           Every source record is linked, including those whose personal data retention
erased.



-- 11. How many records each person has
SELECT records_per_person, count(*) AS people
  FROM (SELECT person_id, count(*) AS records_per_person FROM silver.link_person GROUP BY
person_id) x
 GROUP BY records_per_person ORDER BY records_per_person;
-- Expected: 13,974 people with 1 record, 5,749 with 2, 676 with 3.

-- 11b. The same, by which systems the person appears in
SELECT systems, count(*) AS people
  FROM (SELECT person_id, string_agg(system_name, ' + ' ORDER BY system_name) AS systems
          FROM silver.link_person GROUP BY person_id) x
 GROUP BY systems ORDER BY people DESC;
-- Expected: the 676 with three records are the "ADV_CRM + HRMS + SIS" row.



-- 12. Why each link exists: the rule and its confidence
```

```
SELECT confidence, count(*) AS link_rows FROM silver.link_person GROUP BY confidence ORDER
BY confidence;


SELECT match_rule, confidence, count(*) AS link_rows
  FROM silver.link_person GROUP BY match_rule, confidence ORDER BY link_rows DESC;
-- Expected: SINGLE 13,974 (rule NONE: matched nothing), HIGH 10,576, MEDIUM 2,950.
-- Talking point: R1 is same email, R2 is same surname + date of birth + phone, R3 is same
--                name + date of birth in a different system. A record gets MEDIUM only
when
--                R3 is its only rule, so the weakest links are easy to find and review.



-- 13. A MEDIUM link going wrong, checked against the answer key
SELECT l.person_id, l.system_name, l.source_record_id, l.match_rule, l.confidence,
k.true_person_id
  FROM silver.link_person l
  JOIN qa.identity_answer_key k ON k.source_system = l.system_name AND k.source_record_id
= l.source_record_id
 WHERE l.confidence = 'MEDIUM'
   AND l.person_id IN (SELECT l2.person_id
                         FROM silver.link_person l2
                         JOIN qa.identity_answer_key k2
                           ON k2.source_system = l2.system_name AND k2.source_record_id =
l2.source_record_id
                        GROUP BY l2.person_id HAVING count(DISTINCT k2.true_person_id) >
1)
 ORDER BY l.person_id, l.system_name
 LIMIT 6;
-- Expected: pairs of records merged into one person_id but with DIFFERENT true_person_id
values:
--           different people who share a name and date of birth ("false friends").
-- Talking point: this is why R3 alone is only MEDIUM confidence. See
reports.identity_accuracy.



-- 14. Integrity: hubs and link agree with each other
SELECT 'links with no source hub row' AS check_name, count(*) AS problems
  FROM silver.link_person l
 WHERE NOT EXISTS (SELECT 1 FROM silver.hub_student     h WHERE l.system_name = 'SIS'
AND h.sis_student_id = l.source_record_id)
   AND NOT EXISTS (SELECT 1 FROM silver.hub_employee    h WHERE l.system_name = 'HRMS'
AND h.hr_employee_id = l.source_record_id)
   AND NOT EXISTS (SELECT 1 FROM silver.hub_constituent h WHERE l.system_name = 'ADV_CRM'
AND h.crm_contact_id = l.source_record_id)
UNION ALL
SELECT 'source hub rows with no link',
       (SELECT count(*) FROM silver.hub_student h WHERE NOT EXISTS
            (SELECT 1 FROM silver.link_person l WHERE l.system_name = 'SIS' AND
l.source_record_id = h.sis_student_id))
     + (SELECT count(*) FROM silver.hub_employee h WHERE NOT EXISTS
            (SELECT 1 FROM silver.link_person l WHERE l.system_name = 'HRMS' AND
l.source_record_id = h.hr_employee_id))
     + (SELECT count(*) FROM silver.hub_constituent h WHERE NOT EXISTS
            (SELECT 1 FROM silver.link_person l WHERE l.system_name = 'ADV_CRM' AND
l.source_record_id = h.crm_contact_id))
UNION ALL
SELECT 'link rows with no person hub row', count(*)
  FROM silver.link_person l WHERE NOT EXISTS (SELECT 1 FROM silver.hub_person p WHERE
p.person_id = l.person_id)
UNION ALL
SELECT 'people with no link rows', count(*)
  FROM silver.hub_person p WHERE NOT EXISTS (SELECT 1 FROM silver.link_person l WHERE
l.person_id = p.person_id);
-- Expected: 0 on every line. The end-to-end check "every source record is linked to one
person"
--           (reports.checks) tests the same thing.



-- 15. The link survives retention. Olivia Saito's student and CRM records were
anonymised,
--     yet the link still ties them to her staff record.
SELECT l.system_name, l.source_record_id, l.match_rule, l.confidence,
       CASE l.system_name
```

```
            WHEN 'SIS'     THEN (SELECT anonymised_at IS NOT NULL FROM
silver.student_current     WHERE sis_student_id = l.source_record_id)
            WHEN 'HRMS'    THEN (SELECT anonymised_at IS NOT NULL FROM
silver.employee_current    WHERE hr_employee_id = l.source_record_id)
            WHEN 'ADV_CRM' THEN (SELECT anonymised_at IS NOT NULL FROM
silver.constituent_current WHERE crm_contact_id = l.source_record_id)
       END AS personal_data_erased
  FROM silver.link_person l
 WHERE l.person_id = (SELECT person_id FROM silver.link_person WHERE source_record_id =
'STU0000006')
 ORDER BY l.system_name;
-- Expected: ADV_CRM CRM0000006 erased, HRMS EMP000006 not erased (still employed), SIS
STU0000006 erased.
-- Talking point: retention blanks the personal details in the satellites but keeps the
record and
--                its link, so you can still say how many people were affected without
keeping
--                their details.
```

**`docker-compose.yml`** – Database container

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: wbu_postgres_db
    restart: always

    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}

    ports:
      - "${POSTGRES_PORT}:5432"

    volumes:
      - pgdata:/var/lib/postgresql/data

    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  pgdata:
```

**`requirements.txt`** – Python packages

```
pandas==2.3.3
openpyxl==3.1.5
psycopg[binary]==3.2.10
python-dotenv==1.1.1
matplotlib==3.9.2
```

# 32113 Advances Database

## Project and Presentation
## Assignment 2

## Contribution Form & Marketing Sheet

**Overall Individual Contribution:**

| Student ID | Name | Project: Individual Contribution (0-100%) – | Presentation: Individual Contribution (0-100%) |
|---|---|---|---|
| 26184921 | Arya Mahesh Kaslikar | 100 | 100 |
| 26215537 | Abdul Mannan Mohammed | 100 | 100 |
| 26226264 | Aditi Mangroliya | 100 | 100 |
| 26240402 | Margi Govindbhai Vadodaria | 100 | 100 |
| 25620699 | Mihir Chauhan | 100 | 100 |
| 25630423 | Omkar Ravindran | 100 | 100 |

**Individual Contribution Rating Range Scale:**

WB: Well Below Average 0 - 20
BA: Below Average 20 - 40
AV: Average 40 - 60
AA: Above Average 60 - 80
WA: Well Above Average 80 – 100

**Individual Contribution Rating Range Scale:**

| Rating Name | Scale | Example scenarios |
|---|---|---|
| WB: Well Below Average | 0 - 20 | Team member has zero to limited contribution to the assigned task. |
| BA: Below Average | 20 - 40 | Team member partially completed their assigned task/ and required lot of support to complete their task/ improve the quality of work. |
| AV: Average | 40 - 60 | Team member completed their assigned task with average quality and required support to complete their task/ improve the quality of work. |
| AA: Above Average | 60 - 80 | Team member completed their assigned task/ and required little support to complete their task/improve the quality of work. |
| WA: Well Above Average | 80 - 100 | Team member fully completed their assigned task with no support and supported others to complete their task/ improve the overall quality of work/submission. |

**Notes:** Each group is required to provide the individual group member contributions ratings for both the project and presentation between 0-100%. **This will be multiplied to group marks of Project and Presentation to calculate individual student's marks.**

| Project Criteria Items | Max Marks | Obtained Marks |
|---|---|---|
| Coverage and quality of the advanced database solution design | 10 | |
| Coverage and quality of the working prototype implementation using Snowflake.<br>*Non-working prototype will get zero marks. | 20 | |
| Effective communication (clear title, executive summary, introduction, overall report summary and quality i.e. structure, grammar, clarity) | 5 | |
| Total | 35 | |

| Presentation Criteria Items | Max Marks | Obtained Marks |
|---|---|---|
| Effective communication: presentation and articulation | 2.5 | |
| Correctness of answers to questions | 2.5 | |
| Total | 5 | |

## Project Detailed Marking

**Effective Communication (5)**

(i) Title of the report (1)

(ii) Executive Summary (100-250 Words) (1)

(iii) Introduction - Overview of description of the problem that you are trying to solve with your project. That is, what are the high-level goals of the code that you have implemented in Snowflake (300-400 Words) (1)

(iv) Report Summary/ Conclusion (200-300 Words)

a. Present an overall summary of the contents presented in his report. (1)
b. Overall grammar/ clarity (1)

**Solution Design (10)**

(v) Solution Design Details (No Words Limit)

a. Solution Architecture (2)
b. Conceptual Model of Data Entities (2)
c. Logical Data Model of Data Entities (2)
d. Design Rationale (why you choose the given design e.g. use case fit, performance, complexity, cost) (2)
e. Design Trade-offs (2)

**Working Prototype Implementation (20)**

(vi) Implementation (No Words Limit)

a. Databases and schemas of source systems (at least 3 data sources) (4)
b. Integrated data warehouse database and schema (at least 1) (4)
c. SQL Scripts with synthetic data for creating databases, schemas, extracting data from source systems and transforming/loading into

the Datawarehouse. This should be fully commented, ready for execution without any errors. This should be clearly mapped to the solution design components. (4)

d. Reports and dashboard as per data use cases (at least 3) for your project with synthetic data. (4)
e. End-to-end testing and provide a link in the report to access the recorded end-to-end testing demo video of the working prototype. You may post video demo to your GitHub or YouTube. (4)

**Presentation (5)**

This assessment task includes an oral/visual presentation (e.g. PowerPoint) of the project report/prototype. Each team member must participate in the presentation. Each presentation is expected to last approx. 10 minutes, and maximum no more than 15 minutes. Allow 5 minutes for questions.

**Additional Comments and Remarks:**

All six of us attended every group meeting (Meetings 1 to 5, held between 5 September and 10 October 2026), and everyone finished the part of the work they were given. As a group we agreed to give each member 100% for both the project and the presentation. Below is a short note on what each person did, taken from our meeting minutes.

**Arya Mahesh Kaslikar (group lead):** Arya set up our Notion board in the first meeting and worked with Omkar on the architecture flowchart. From Meeting 2 onwards Arya looked after the start of the pipeline: loading the HRMS, SIS and CRM data and the shadow copies, and building the bronze and silver layers. Arya also set up the GitHub repository, kept the scripts rerunnable and helped with the pipeline details in the report.

**Abdul Mannan Mohammed:** Abdul drafted the first report outline with Mihir. On the prototype, Abdul built the gold star schema and the reports that run on it. From Meeting 4, Abdul and Omkar took the lead on writing and putting together the final report.

**Aditi Mangroliya:** Aditi shortlisted the dashboard tools with Margi at the start. Aditi's part of the prototype was the masked test data, meaning the masked copies of the three systems and the leak check that makes sure no real personal data gets through. Aditi and Margi then built the dashboard for the retention, control coverage and exposure reports.

**Margi Govindbhai Vadodaria:** Margi worked on the governance side of the prototype: the data catalogue, lineage, the personal-data column tags and the discovery scan that picked up the unregistered shadow copies. Margi also worked with Aditi on the dashboard, from choosing the tool in Meeting 1 to finishing it by Meeting 5.

**Mihir Chauhan:** Mihir drafted the report outline with Abdul in the first meeting. Mihir then built the identity matching, including the person hub, the matching rules and the accuracy check against the answer key, and reviewed the MEDIUM confidence matches. Mihir also wrote the test video script with Omkar and recorded the end-to-end demo video.

**Omkar Ravindran:** Omkar worked with Arya on the solution design and the architecture flowchart. Omkar's part of the prototype was the retention rules, the purge and the end-to-end test, which meant rebuilding from an empty database until every check passed. Omkar also co-wrote the test video script with Mihir and put the report together with Abdul.

# UNIVERSITY OF TECHNOLOGY SYDNEY

## 32113 – Meeting Minutes

| GROUP NR.: | MEETING NR.: | DATE/TIME: | TEACHER SIGNATURE: |
|---|---|---|---|
| 2 | 1 | 05/09/2026<br>4:00PM–4:45PM (45 minutes) | Saba, 08/09/2026 |

**ATTENDEES:**

- Arya Kashlikar 26184921
- Abdul Mannan Mohammed 26215537
- Aditi Mangroliya 26226264
- Margi Govindbhai Vadodaria 26240402
- Mihir Chauhan 25620699
- Omkar Ravindran 25630423

**ABSENT:**

None. All six members were present.

**ACTION ITEMS:**

- All members: complete feasibility checks before Meeting 7.
- Arya and Omkar: finalise the architectural flowchart.
- Mihir and Abdul: draft the report outline.
- Aditi and Margi: shortlist a dashboard tool.
- All members: set up a local development environment.
- Arya: create the Assignment 2 board in Notion.

**MEETING NOTES:**

- Opened Assignment 2 and identified the overall structure.
- Established the architectural flowchart for the implementation.
- Split the work into three streams: design, report and dashboard.
- Confirmed all six members work on the implementation.
- Began feasibility studies on the proposed tools and platforms.
- Design stays provisional until the feasibility results are in.

**ASSIGNED RESPONSIBILITIES:**

Advanced Database Assignment 2 | Notion

**All members**
Implementation: each member builds and tests one component.

**Arya and Omkar**
Solution design: architectural flowchart, data stores and data flow.

**Mihir and Abdul**
Report: structure, written analysis and rubric mapping.

**Aditi and Margi**
Dashboard: tool selection, layout, metrics and data connection.

**ACCOMPLISHED TASKS:**

- Structure for Assignment 2 identified.
- Architectural flowchart established.
- Work split into three streams.
- Roles assigned across design, report and dashboard.
- Feasibility studies started.

**TEACHER FEEDBACK / NEXT STEPS:**

# UNIVERSITY OF TECHNOLOGY SYDNEY

## 32113 - Meeting Minutes

| GROUP NR.: | MEETING NR.: | DATE/TIME: | TEACHER SIGNATURE: |
|---|---|---|---|
| 2 | 2 | 23/09/2026<br>4:00PM to 4:45PM (45 minutes) | |

**ATTENDEES:**

– Arya Kashlikar 26184921
– Abdul Mannan Mohammed 26215537
– Aditi Mangroliya 26226264
– Margi Govindbhai Vadodaria 26240402 – Mihir Chauhan 25620699
– Omkar Ravindran 25630423

**ABSENT:**

None. All six members present.

**ACTION ITEMS:**

– Each member to write the SQL scripts for their assigned problem in the course Docker Postgres environment.
– Abdul to build the gold star schema and at least 3 dashboard reports.
– Omkar to run the end-to-end test from an empty database and record the test video.
– All members write their report section in their own words and add their trade-offs.
– All members to bring working scripts to the next meeting for integration.

**MEETING NOTES:**

– Agreed the Assignment 2 prototype will address the five shadow data problems from Assignment 1. – Problem 1, no governed integration: replace manual copies with one logged medallion pipeline (bronze, silver, gold, audit) and a Data Vault silver layer.
– Problem 2, no test data management: give developers a masked clone (pseudonyms, fake names, shifted birth dates, TFN and bank nulled) plus a leak check.
– Problem 3, no metadata or lineage: add a dataset catalogue, lineage graph and PII column tags in the audit schema.
– Problem 4, no retention policy: store rules in a policy table, list overdue records in a due-list view and purge with a dry run and log.
– Problem 5, one person with three IDs: build a person hub using email and name plus DOB matching, with MEDIUM matches sent for human review.
– Agreed the brief mapping: 3 source systems (HRMS, SIS, CRM), a bronze/silver/gold warehouse, numbered ETL scripts, at least 3 reports and an end-to-end test.
– Agreed to discuss the design trade-offs in the report, including pseudonyms not being fully anonymous, placeholder retention periods and matching errors.
– Reminder: GenAI must not be used to write the report or SQL scripts.

| **ASSIGNED RESPONSIBILITIES:** | **ACCOMPLISHED TASKS:** |
| --- | --- |
| Advanced Database Assignment 2 \| Notion **Arya**<br>Problem 1: governed pipeline and audit log **Mihir**<br>Problem 5: person hub and identity matching **Aditi**<br>Problem 2: masked test data and leak check **Abdul**<br>Gold star schema and dashboard reports **Margi**<br>Problem 3: catalogue, lineage and PII tags **Omkar** | – Walked through all five problems and agreed a solution for each.<br>– Confirmed the prototype architecture and how each fix feeds the next.<br>– Chose the reports: retention due-list, real PII outside production, identity overlap and masking leak check.<br>– Split the prototype work across all six members. |



| | |
| --- | --- |
| Problem 4: retention policy and purge, plus end-to end test | |

| **TEACHER FEEDBACK / NEXT STEPS:** |
| --- |
| |

# UNIVERSITY OF TECHNOLOGY SYDNEY

## 32113 - Meeting Minutes

| GROUP NR.: | MEETING NR.: | DATE/TIME: | TEACHER SIGNATURE: |
|---|---|---|---|
| 2 | 3 | 30/09/2026<br>4:00PM to 4:45PM (45 minutes) | |

**ATTENDEES:**

- Arya Kashlikar 26184921
- Abdul Mannan Mohammed 26215537
- Aditi Mangroliya 26226264
- Margi Govindbhai Vadodaria 26240402
- Mihir Chauhan 25620699
- Omkar Ravindran 25630423

**ABSENT:**

None. All six members present.

**ACTION ITEMS:**

- Each member to finish their script using their implementation guide and keep it rerunnable.
- Omkar to rebuild from empty with run_all.sh until all 19 end-to-end checks pass, then record the test video.
- Mihir to review the 8 MEDIUM identity matches in the review queue.
- Abdul to connect R1, R5 and R6 to a dashboard.
- All members to draft their report section, including findings and trade-offs.
- All members to bring passing scripts and draft sections to Meeting 6.

**MEETING NOTES:**

- Agreed to build the prototype on the Uwaterbrook synthetic shadow data workbook: HRMS 5,000, SIS 15,000, CRM 7,500 and 27,500 shadow extracts across 8 environments.
- Agreed person_master_id is removed from the source tables and kept only as an answer key for testing.
- Agreed one run order (scripts 00 to 09) and a single run_all.sh that rebuilds from an empty database.
- Data quality: 1,675 future birth dates and 2,624 staff born under 15 years before their start date. Agreed to flag these, not drop them.
- Agreed five retention policies: delete expired copies, delete unused unmasked copies, mask copies still in use, anonymise old student contact details, expire test data after 30 days.
- Agreed identity rules: same email is HIGH, same name and DOB is MEDIUM, and MEDIUM matches go to human review.
- Agreed R1 (retention due-list), R5 (control coverage) and R6 (exposure by access group) as the core three reports.

**ASSIGNED RESPONSIBILITIES:**

Advanced Database Assignment 2 | Notion

**Arya**
00_sources, 01_before, 03_pipeline: source load, shadow copies, bronze and silver

**Mihir**
05_identity: person hub, matching accuracy, breach sizing

**Aditi**
04_masking: masked clone and leak check

**Abdul**
08_gold_reports: star schemas and dashboard reports

**Margi**
02_catalogue, 06_discovery: catalogue, lineage, discovery scan

**Omkar**

**ACCOMPLISHED TASKS:**

- Reviewed progress on each member's Meeting 4 action items.
- Converted the workbook sheets to CSV for loading into Postgres.
- Mapped each of the five problems to a numbered script and owner.
- Agreed the run order, retention policies, matching rules and core reports.

| | |
|---|---|
| 07_retention, 09_e2e_test: retention purge and end-to-end test | |

| |
|---|
| **TEACHER FEEDBACK / NEXT STEPS:** |

# UNIVERSITY OF TECHNOLOGY SYDNEY

## 32113 - Meeting Minutes

| GROUP NR.: | MEETING NR.: | DATE/TIME: | TEACHER SIGNATURE: |
|---|---|---|---|
| 2 | 4 | 3/10/2026<br>4:00PM to 4:45PM (45 minutes) | Saba 6/10/26 |

**ATTENDEES:**

- Arya Kashlikar 26184921
- Abdul Mannan Mohammed 26215537
- Aditi Mangroliya 26226264
- Margi Govindbhai Vadodaria 26240402
- Mihir Chauhan 25620699
- Omkar Ravindran 25630423

**ABSENT:**

None. All six members present.

**ACTION ITEMS:**

- Each member to finish their script using their implementation guide and keeps it rerunnable.
- Omkar and Mihir to rebuild from empty with run_all.sh until all 19 end-to-end checks pass, then record the test video.
- Mihir to review the 8 MEDIUM identity matches in the review queue.
- Omkar and Abdul to begin report creation.
- Abdul to connect R1, R5 and R6 to a dashboard.
- All members to draft their report section, including findings and trade-offs.
- All members to bring passing scripts and draft sections to Meeting 6.

**MEETING NOTES:**

- Agreed to build the prototype on the Waterbrook (University Name) synthetic shadow data workbook: HRMS 5,000, SIS 15,000, CRM 7,500 and 27,500 shadow extracts across 8 environments.
- Agreed person_master_id is removed from the source tables and kept only as an answer key for testing.
- Agreed one run order (scripts 00 to 09) and a single run_all.sh that rebuilds from an empty database.
- Data quality: 1,675 future birth dates and 2,624 staff were born under 15 years before their start date. Agreed to flag these, not drop them.
- Agreed five retention policies: delete expired copies, delete unused unmasked copies, mask copies still in use, anonymise old student contact details, expire test data after 30 days.
- Agreed identity rules: same email is HIGH, same name and DOB is MEDIUM, and MEDIUM matches go to human review.
- Agreed R1 (retention due-list), R5 (control coverage) and R6 (exposure by access group) as the core three reports.

**ASSIGNED RESPONSIBILITIES:**

Advanced Database Assignment 2 | Notion

**Arya**
00_sources, 01_before, 03_pipeline: source load, shadow copies, bronze and silver.

**Mihir**
05_identity: person hub, matching accuracy, breach sizing

**Aditi**
04_masking: masked clone and leak check

**Abdul**
08_gold_reports: star schemas and dashboard reports

**Margi**
02_catalogue, 06_discovery: catalogue, lineage, discovery scan

**ACCOMPLISHED TASKS:**

- Reviewed progress on each member's Meeting 4 action items.
- Converted the workbook sheets to CSV for loading into Postgres.
- Mapped each of the five problems to a numbered script and owner.
- Agreed with the run order, retention policies, matching rules and core reports.
- Set up the repository, completed the DB and started with the Python pipelines

| | |
|---|---|
| **Omkar**<br><br>07_retention, 09_e2e_test: retention purge and end-to-end test | |

**TEACHER FEEDBACK / NEXT STEPS:**

When writing minutes, we go around the table and ask what we have done, where we are stuck and what is the next step.
Avoid abbreviations. If using, mention at the top of the document.

# UNIVERSITY OF TECHNOLOGY SYDNEY

## 32113 - Meeting Minutes

| GROUP NR.: | MEETING NR.: | DATE/TIME: | TEACHER SIGNATURE: |
|---|---|---|---|
| 2 | 5 | 10/10/2026<br>4:00PM to 4:45PM (45 minutes) | Saba, 13/10/26 |

**ATTENDEES:**
- Arya Kashlikar 26184921
- Abdul Mannan Mohammed 26215537
- Aditi Mangroliya 26226264
- Margi Govindbhai Vadodaria 26240402
- Mihir Chauhan 25620699
- Omkar Ravindran 25630423

**ABSENT:**
None. All six members present.

**ACTION ITEMS:**
- Mihir to finish recording the test video.
- Omkar and Abdul to complete the report.
- Aditi and Margi to send dashboard screenshots to Omkar and Abdul for the report.
- All members to proofread the report and remove or define any abbreviations.
- All members to bring the final report and test video to the next meeting.

**MEETING NOTES:**
We went around the table and each member shared what they have done, where they are stuck and their next step.

**Arya**
- **Done:** source load, shadow copies, bronze and silver scripts finished and rerunnable.
- **Stuck:** nothing.
- **Next:** support the report with the data pipeline details.

**Mihir**
- **Done:** wrote the test video script with Omkar.
- **Stuck:** nothing.
- **Next:** finish recording the test video.

**Aditi**
- **Done:** finished the dashboard with Margi.
- **Stuck:** nothing.
- **Next:** send dashboard screenshots for the report.

**Margi**
- **Done:** finished the dashboard with Aditi.
- **Stuck:** nothing.
- **Next:** send dashboard screenshots for the report.

**Abdul**
- **Done:** working on the report with Omkar.
- **Stuck:** nothing.
- **Next:** complete the report with Omkar.

**Omkar**
- **Done:** completed the local end-to-end checks; wrote the test video script with Mihir.
- **Stuck:** nothing.
- **Next:** complete the report with Abdul.

| **ASSIGNED RESPONSIBILITIES:** | **ACCOMPLISHED TASKS:** |
|---|---|
| Advanced Database Assignment 2 \| Notion<br>**Arya**<br>Source data load, shadow copies, and bronze and silver data layers.<br>**Mihir**<br>Identity matching: person hub, matching accuracy and breach sizing. Test video.<br>**Aditi**<br>Data masking: masked clone and leak check. Dashboard (with Margi).<br>**Margi**<br>Data catalogue, lineage and discovery scan. Dashboard (with Aditi).<br>**Abdul**<br>Gold reporting layer: star schemas. Report (with Omkar).<br>**Omkar**<br>Retention purge and end-to-end testing. Report (with Abdul). | – Reviewed each member's action items from the last meeting around the table.<br>– Dashboard for the retention due-list, control coverage and exposure by access group reports completed (Aditi and Margi).<br>– Local end-to-end checks completed (Omkar).<br>– Test video script written (Omkar and Mihir).<br>– Report in progress (Omkar and Abdul). |

| **TEACHER FEEDBACK / NEXT STEPS:** |
|---|
| |